

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

Цэдэн-Ишийн Биндэрцэцэг

**Холимог Микрофронтенд Архитектур: React ба
JSF ашигласан кейс судалгаа**
**(Hybrid Microfrontend Architecture: A React and JSF Case
Study)**

Программ Хангамж (D061302)
Баклаврын судалгааны ажил

Улаанбаатар хот

2026 оны 5 сар

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

Холимог Микрофронтенд Архитектур: React ба JSF
ашигласан кейс судалгаа
(Hybrid Microfrontend Architecture: A React and JSF Case
Study)

Программ Хангамж (D061302)
Баклаврын судалгааны ажил

Удирдагч: _____ Ph.D. Б. Батням

Гүйцэтгэсэн: _____ Ц. Биндэрцэцэг (22B1NUM0027)

Улаанбаатар хот

2026 оны 5 сар

Зохиогчийн баталгаа

Миний бие Цэдэн-Ишийн Биндэрцэцэг нь ”Холимог Микрофронтенд Архитектур: React ба JSF ашигласан кейс судалгаа” сэдэвтэй судалгааны ажлыг гүйцэтгэсэн болохыг зарлаж дараах зүйлсийг баталж байна:

- Ажил нь бүхэлдээ эсвэл ихэнхдээ Монгол Улсын Их Сургуулийн зэрэг горилохоор дэвшүүлсэн болно.
- Энэхүү ажлын аль нэг хэсгийг эсвэл бүхлээр нь ямар нэг их, дээд сургуулийн зэрэг горилохоор оруулж байгаагүй.
- Бусдын хийсэн ажлаас хуулбарлаагүй, ашигласан бол ишлэл, зүүлт хийсэн.
- Ажлыг би өөрөө (хамтарч) хийсэн ба миний хийсэн ажил, үзүүлсэн дэмжлэгийг дадлагын ажилд тодорхой тусгасан.
- Ажилд тусалсан бүх эх сурвалжид талархаж байна.

Гарын үсэг: _____

Огноо: _____

ГАРЧИГ

БҮЛГҮҮД	1
1. СУДАЛГААНЫ АЖЛЫН ТУХАЙ.....	1
1.1 Сэдэв сонгосон үндэслэл	1
1.2 Зорилго	1
1.3 Зорилт	1
1.4 Сэдвийн судлагдсан байдал	1
1.5 Судалгааны шинэлэг тал	2
1.6 Судалгааны ажлын үр дүн	2
1.7 Судалгааны ажлын арга зүй	3
2. СУДАЛГАА	6
2.1 Онолын судалгаа	6
2.2 Технологийн судалгаа	12
2.3 Сэдвийн судалгаа	17
2.4 ISO 25010 стандарт	21
3. ТУРШИЛТИЙН ЗАГВАР	26
3.1 Туршилтын загварын бүтэц	26
3.2 Асинхрон скрипт ачаалалт	27
3.3 CSS загварын давхцал	28
4. СИСТЕМИЙН ШИНЖИЛГЭЭ	32
4.1 Системийн алсын хараа	32
4.2 Функциональ загвар	33
4.3 Динамик загвар	36
4.4 Бүлгийн дүгнэлт	40
5. СИСТЕМИЙН ЗОХИОМЖ	41
5.1 Архитектурын үндэслэл	41

5.2	Микрофронтенд архитектур	42
5.3	Микросервис Архитектур	45
5.4	Өгөгдлийн давхарга	48
5.5	Бүлгийн дүгнэлт	50
6.	СИСТЕМИЙН ХЭРЭГЖҮҮЛЭЛТ	53
6.1	Фронтенд Хэрэгжүүлэлт	53
6.2	Бэкенд Хэрэгжүүлэлт	59
6.3	Өгөгдлийн Сан	62
6.4	Аюулгүй байдал	64
6.5	Бүлгийн дүгнэлт	65
7.	ТУРШИЛТЫН ҮР ДҮН	66
7.1	Туршилтын Орчин	66
7.2	Статик Хэмжилт	67
7.3	Хөгжүүлэлтийн Хурд	69
7.4	Ачааллын туршилтын үр дүн	71
7.5	Сүлжээний ачаалал	73
7.6	Кодын чанарын шинжилгээ	76
7.7	Бүлгийн дүгнэлт	77
8.	ДҮГНЭЛТ	81
	НОМ ЗҮЙ	82
	ХАВСРАЛТ	86
	А. НЭР ТОМЬЁОНЫ ТАЙЛБАР	86
	В. МИКРОФРОНТЕНД ХУУДСУУД	89
	С. КОДЫН ХЭРЭГЖҮҮЛЭЛТ	93

ЗУРГИЙН ЖАГСААЛТ

1.1	Судалгааны үйл явцын үе шатууд	5
2.1	Цул архитектур	6
2.2	Микросервис архитектур	8
2.3	Микрофронтенд архитектур	10
2.4	JSF хүсэлт боловсруулах амьдралын мөчлөг, Эх сурвалж: [14]	14
2.5	React амьдралын мөчлөг, Эх сурвалж: [6]	15
2.6	Бүтээгдэхүүний чанарын загвар ба сонгосон чанарын үзүүлэлтүүд	22
3.1	Туршилтын загварын бүтэц	26
3.2	Асинхрон скрипт ачаалалтын эхлүүлэх логик	27
3.3	Асинхрон скрипт ачаалалтын динамик дуудлага	28
3.4	CSS Modules-ийн хэш нэмэгдсэн класс нэрүүд (Browser DevTools)	29
3.5	Одоо байгаа микрофронтенд архитектурын шийдэл	30
3.6	Санал болгож буй холимог микрофронтенд архитектурын шийдэл	31
4.1	БСА-ын сэдэв дэвшүүлэх процесс	33
4.2	БСА-ын сэдэв дэвшүүлэх процессын өргөтгөл	34
4.3	БСА-ын сэдэв сонгох процесс	34
4.4	БСА-ын үечилсэн төлөвлөгөөг тэнхимээр батлуулах процесс	35
4.5	БСА гүйцэтгэх, тайлан хураалгах процесс	35
4.6	БСА-ыг хамгаалах, дүгнэх процесс	36
4.7	Дипломын ажлын удирдлагын системийн динамик загвар	37
5.1	Микрофронтендүүд хоорондын харилцаа	45
5.2	Микросервисүүд хоорондын харилцаа	48
5.3	Системийн ER диаграмм	50
5.4	Системийн бүрэн архитектурын нэгдсэн бүдүүвч	51
6.1	JWT нэвтрэлтийн дараалал	56

6.2	Зургаан талт архитектур	60
7.1	JSF болон React MFE архитектурын Lead Time харьцуулалт	69
7.2	Vite HMR болон JSF дахин ачаалах хугацааны харьцуулалт	71
7.3	Зэрэгцээ хэрэглэгчийн тоо болон JVM heap ашиглалтын харьцаа	72
7.4	P50, P75, P95, P99, P99.9 латенцийн перцентилийн харьцуулалт	73
7.5	Хуудас шилжилтэд дамжих өгөгдлийн хэмжээний харьцуулалт	74
7.6	Хуудас шилжилтийн тоо ба сүлжээний ачааллын хамаарал	75
7.7	HTTP кэшийн нөлөө: JSF болон MFE кэштэй хувилбарын харьцуулалт	75
7.8	ISO/IEC 25010 стандартын 7 үзүүлэлтээрх радарын харьцуулалт	79
B.1	Оюутны илгээсэн тайланг хянах хэсэг	89
B.2	Системд нэвтрэх цонх	90
B.3	Комиссын гишүүн оюутны ажлыг үнэлэх хэсэг	90
B.4	Тэнхимийн хянах самбар	91
B.5	Оюутны хянах самбар	91
B.6	Комисс үүсгэх хэсэг	92
B.7	Оюутан сэдэв дэвшүүлэх хэсэг	92

ХҮСНЭГТИЙН ЖАГСААЛТ

2.1	Интеграцийн төрлүүдийн харьцуулалт	12
2.2	Сонгож авсан судалгааны ажлын тойм	20
2.3	ISO/IEC 25010 үзүүлэлт ба харгалзах хэмжилтийн хэрэгсэл	25
3.1	Туршилтын загварын техникийн орчин	27
4.1	Системийн алсын харааны тодорхойлолт	32
4.2	Дипломын ажлын удирдлагын системийн ажлын явцын тайлбар	37
5.1	Архитектурын стратегийн харьцуулалт (Туршилтын загвараар)	42
5.2	MFE модулиудын зохион байгуулалт	44
5.3	Микросервисүүдийн зохион байгуулалт	46
6.1	Дипломын ажлын статус шилжилтийн зөвшөөрөгдсөн матриц	61
6.2	R2DBC Connection Pool-ийн тохиргооны параметрууд	62
7.1	MFE модуль тус бүрийн dist хэмжээ	67
7.2	Технологи тус бүрийн кодын мөрийн тоо (LOC)	68
7.3	Build паплайны хугацааны харьцуулалт (секунд)	68
7.4	Багш сэдэв дэвшүүлэх даалгаврын хугацааны задаргаа (цагаар)	69
7.5	HMR болон JSF дахин ачаалах хугацааны харьцуулалт (миллисекунд) ..	70
7.6	Зэрэгцээ хэрэглэгчийн тоо болон JVM heap ашиглалтын харьцаа	72
7.7	HTTP хариу латенцийн перцентил харьцуулалт (мс, 1,000 зэрэгцээ хэрэглэгч)	73
7.8	Хуудас шилжилт тус бүрийн өгөгдлийн хэмжээ (КБ)	74
7.9	Шилжилтийн тооноос хамаарсан нийт сүлжээний ачаалал (КБ)	75
7.10	SonarQube кодын чанарын хэмжүүрүүдийн нэгтгэсэн үзүүлэлт	76
7.11	Хэмжилтэд суурилсан ISO/IEC 25010 чанарын харьцуулалт	78
7.12	Архитектурын хэмжилтийн нэгтгэсэн үр дүн	80

КОДЫН ЖАГСААЛТ

6.1	module-thesis-ийн vite.config.ts тохиргооны бүтэц	53
6.2	JSF .xhtml хуудасны MFE injection механизм	54
6.3	Shared Singleton загварын зохицуулалт. Antd нь нэгдсэн singleton хэлбэрт ачааллагдаж, Remote модуль бүр энэ нэг instance-ийг ашиглана	58
6.4	build-modules.sh шелл скрипт	58
6.5	Cross-service мэдээллийн нийцлийн механизм. Retry болон Idempotency Key-ийн хослол нь eventual consistency-г хангана	63
C.1	DefenseSessionController.java	93
C.2	CorsConfig.java	96
C.3	vite.config.ts	97
C.4	AuthContext.tsx	98
C.5	authGuard.ts	101

1. СУДАЛГААНЫ АЖЛЫН ТУХАЙ

Энэ бүлэгт судалгааны ажлын үндэслэл, хүрэх зорилго болон зорилтуудыг танилцуулна.

1.1 Сэдэв сонгосон үндэслэл

Вэб аппликейшн хөгжүүлэлтэд орчин үеийн JavaScript фреймворкууд өргөн ашиглагдаж байгаа хэдий ч, зарим тохиолдолд серверийн талд рендер хийгддэг уламжлалт технологитой хослуулан ашиглах шаардлага гардаг. Ийм ялгаатай технологийн парадигмуудыг нэгтгэж нэгдсэн хэрэглэгчийн интерфейс үүсгэх нь архитектурын хувьд хүндрэлтэй байдаг. Иймд шинээр хөгжүүлэгдэж буй МУИС-ийн Дипломын ажлын удирдлагын системийн хөгжүүлэлтийн нэгэн хувилбар болгон холимог микрофронтенд архитектурыг сонгон авч, React, JSF фреймворкуудыг хэрхэн уялдуулан ажиллуулж болохыг бодитоор хэрэгжүүлэн, туршилт хийж судлах нь энэхүү сэдвийг сонгох үндэслэл болсон болно.

1.2 Зорилго

МУИС-ийн Дипломын ажлын удирдлагын системийн хэрэглэгчийн интерфейс дээр React болон JSF технологийг хослуулсан микрофронтенд архитектурыг туршина.

1.3 Зорилт

Дээрх зорилгод хүрэхийн тулд дараах зорилтуудыг дэвшүүлэн ажиллана. Үүнд:

1. Судалгаа хийх зорилтоор микрофронтенд архитектурын онол, арга зүйг судлах;
2. Туршилтын загвар боловсруулах зорилтоор React, JSF ашиглан холимог архитектур бүхий системийн туршилтын загварыг хөгжүүлэх;
3. Туршилт ба үнэлгээ хийх зорилтоор боловсруулсан загвар дээрээ туршилт хийж, ISO/IEC 25010 стандарт болон DORA, ATAM аргачлалаар үнэлж, дүгнэнэ.

1.4 Сэдвийн судлагдсан байдал

Энэхүү судалгаанд Scopus болон DiVA сангаас микрофронтенд архитектурын чиглэлээрх 10 гол бүтээлийг сонгон авч шинжиллээ. Тэдгээрийн үндсэн чиглэлийг Хүснэгт 2.2-д

НЭГТГЭВ.

- Интеграцийн шийдэл: [35, 43] нар систем задлах онолыг судалсан ч Strangler Fig үлгэр загварын шилжилтийн үе болох төлөв хадгалдаг болон төлөвгүй архитектурыг зэрэгцүүлэн ажиллуулах техникийн зааварчилгааг орхигдуулсан байна.
- Нөөцийн давхардал: [4, 48] зэрэг судалгаанууд сүлжээний ачааллыг хэмжсэн боловч холимог орчин дахь серверийн нөөцийн давхардал, түүний нөлөөллийг тооцоолоогүй байна.
- Парадигм хоорондын төлөв: [45, 40] нар төлөв хуваалцахыг гол сорилт гэж онцолсон ч ялгаатай парадигм бүхий системүүдийг уялдуулах бодит техникийн шийдлийг санал болгоогүй.
- Танин мэдэхүйн ачаалал: [8, 50] зэрэг бүтээлүүд хөгжүүлэлтийн хурдыг хэмжсэн ч хоёр өөр технологийг нэгэн зэрэг удирдах үед хөгжүүлэгчид үүсэх танин мэдэхүйн ачааллыг үнэлээгүй орхисон байна.

1.5 Судалгааны шинэлэг тал

Энэ бакалаврын судалгааны ажлын шинэлэг тал нь дараах хоёр хэсгээс бүрдэнэ. Үүнд:

1. Серверийн талын JSF болон клиент талын React фреймворкийг `<script>` интеграци ашиглан DOM түвшинд нэгтгэсэн туршилтын загвар хөгжүүлсэн.
2. Хэрэгжүүлсэн холимог микрофронтенд архитектурыг ISO/IEC 25010 чанарын стандартын дагуу үнэлж дүгнэсэн.

1.6 Судалгааны ажлын үр дүн

Үр дүн

Энэ судалгааны хүрээнд JSF, React фреймворкуудыг хослуулсан микрофронтенд архитектурын туршилтын загварыг хэрэгжүүлж, модулиудыг бие даан ажиллуулж, интеграци хийх боломжтойг харуулна. Мөн, үнэлгээний үр дүнг гаргана.

Ач холбогдол

Энэ судалгаа нь өөр өөр технологийн стэк бүхий хэрэглээний программуудыг микрофронтенд архитектураар хэрхэн нэгтгэх техникийн шийдлийг бодитоор харуулсан

1.7. СУДАЛГААНЫ АЖЛЫН АРГА ЗҮЙ БҮЛЭГ 1. СУДАЛГААНЫ АЖЛЫН ТУХАЙ

кейс судалгаа болно. Цаашид ижил төстэй технологийн хоршил ашиглах хэрэгцээ гарсан оюутнуудад техникийн лавлагаа, харьцуулалт хийх суурь мэдээлэл болох ач холбогдолтой юм.

1.7 Судалгааны ажлын арга зүй

Судалгааны үйл явцыг илүү сайн ойлгохын тулд 5 үе шатанд хуваасан. Үе шат бүрд дараагийн шатанд бэлтгэх, хэрэгжүүлэлтийн талаар туршлага хуримтлуулах, үүссэн өгөгдлийг хэмжиж, цуглуулах тодорхой үйлдлүүдийг гүйцэтгэнэ. Зураг 1.1-д судалгааны үйл явцын үе шатууд болон тэдгээрийг хэрэгжүүлэх дарааллыг харуулав.

Сэдвийн судлагдсан байдал, хүрээг тодорхойлох

Энэ үе шатны зорилго нь сэдвийн ойлголттой танилцахад оршино. Шаардлагатай мэдлэгийн түвшинд хүрэхийн тулд Scopus, DiVA (Дэд бүлэг 2.3-ыг үзнэ үү) өгөгдлийн сангаас гадна төрөл бүрийн блог, форум, фреймворкүүдийн техникийн баримт бичгүүдийг ашигласан. Энд ISO/IEC 25010 стандартыг судалж, МУИС-ийн Дипломын ажлын удирдлагын системд шаардлагатай байж болох программ хангамжийн чанарын үзүүлэлтүүдийн тодорхойлолтыг гаргасан.

Туршилтын загвар

Фронтенд хөгжүүлэлтэд микрофронтенд архитектурыг хэрэгжүүлэх техникийн мэдлэг, туршлага хуримтлуулах зорилгоор туршилтын загварыг (Дэд бүлэг 3.1-ыг үзнэ үү) хөгжүүлсэн. Туршлага хуримтлуулахаас гадна, хэрэгжүүлэлтийн явцад төрөл бүрийн сорилт, бэрхшээлүүд гарч ирсэн болно. Туршилтын загвар нь React,JSF гэсэн хоёр өөр фреймворкоор хөгжүүлэгдсэн, микрофронтенд хэрэглээний программуудыг ачаалдаг жижиг хэмжээний хэрэглээний программ юм.

Туршилт

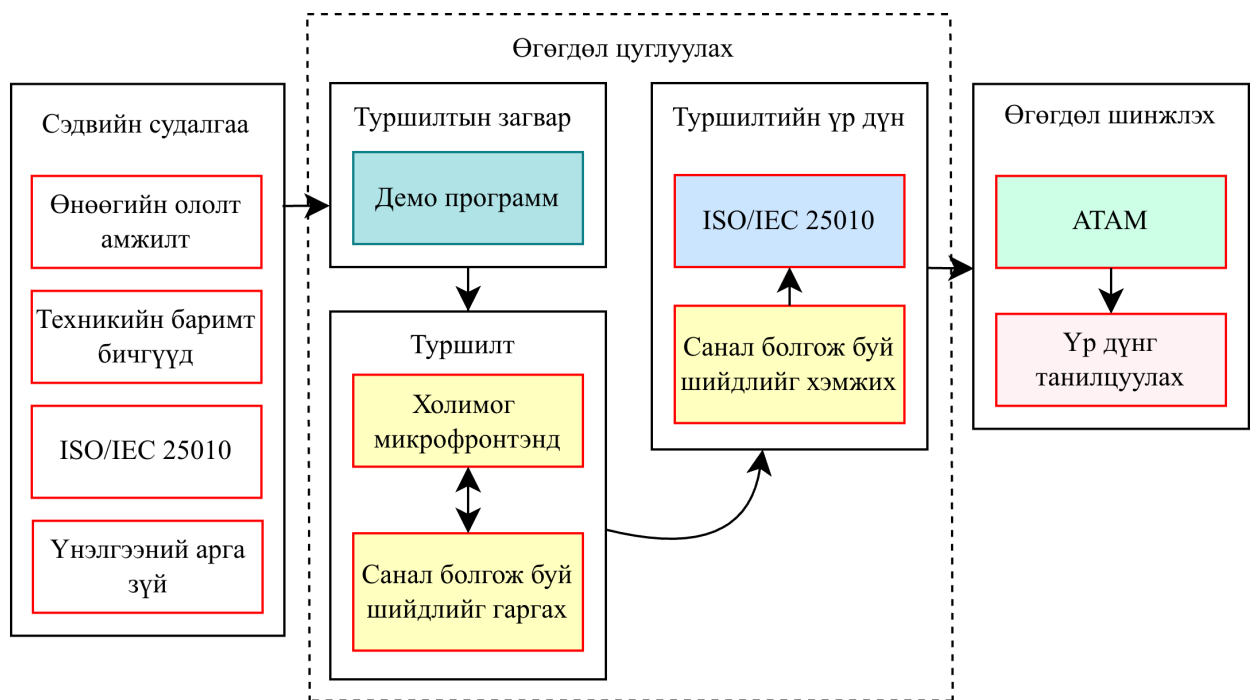
Туршилтын үе шат нь МУИС-ийн Дипломын ажлын удирдлагын системийн хэрэглэгчийн интерфэйсийг цэвэр микрофронтенд болон холимог микрофронтенд архитектураар хөгжүүлэх үйл явцаас бүрдэнэ. Хөгжүүлэлтийн давтамж бүрийн дараа санал болгож буй шийдлийг сонгогдсон ISO/IEC 25010 чанарын үзүүлэлтүүд болон микрофронтенд архитектурын үндсэн зарчмуудтай уялдуулан үнэлэв.

Кэйс судалгаа

Кэйс судалгааны объект нь МУИС-ийн Дипломын ажлын удирдлагын систем бөгөөд харьцуулах хоёр хувилбарыг тодорхойлов: нэгдүгээрт, JSF 2.2 монолит архитектур; хоёрдугаарт, Vite Module Federation ашиглан хэрэгжүүлсэн React MFE болон Spring WebFlux реактив микросервис архитектур. Судалгааны нэгж нь нэг системийн хоёр архитектурын хувилбарыг ижил функциональ шаардлага, ижил туршилтын орчинд (Apple M2 Pro, macOS 15.3, localhost) хэмжсэн нэг кэйс зохиомж юм. Хэмжигдэх үзүүлэлтүүдийг дөрвөн бүлэгт хуваав: хөгжүүлэлтийн хурд (Lead Time, HMR/reload), ажлын ачааллын гүйцэтгэл (RAM, латенц), сүлжээний зардал (payload, кэш), кодын чанар (Cyclomatic Complexity, duplication).

Өгөгдлийн шинжилгээ

Цуглуулсан өгөгдлийг хоёр аргачлалаар шинжлэв. Нэгдүгээрт, ISO/IEC 25010 стандартын 7 чанарын үзүүлэлт (Хугацааны төлөв, Харилцан ажиллах чадвар, Модульлаг байдал, Дахин ашиглах боломж, Шинжилж болохуйц байдал, Өөрчлөх боломж, Тестлэх боломж)-ийг Google Lighthouse, SonarQube, madge, Jest, Cypress хэрэгслүүдийн тоон гаралтаар хэмжиж, JSF болон React MFE архитектурыг харьцуулан оноолов. Хоёрдугаарт, АТАМ (Architecture Trade-off Analysis Method) аргачлалаар архитектурын гол шийдвэрүүдийн мэдрэмтгий цэг (Sensitivity Point) болон буулт цэгүүдийг (Trade-off Point) тодорхойлж, чанарын үзүүлэлтүүдийн хоорондох өрсөлдөөнт харилцааг шинжиллэв.



Зураг 1.1. Судалгааны үйл явцын үе шатууд

2. СУДАЛГАА

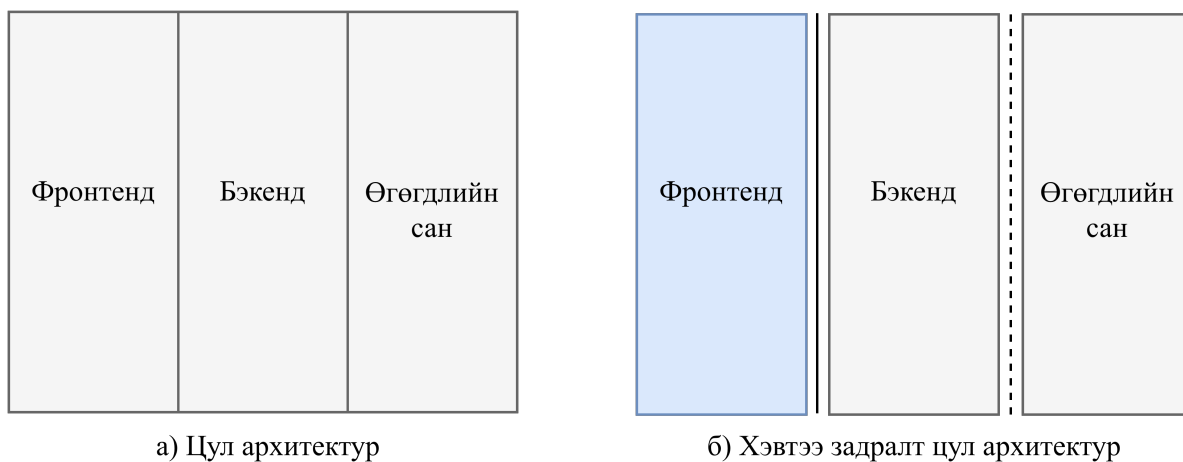
Энэ бүлэгт судалгааны ажлын онолын үндэслэл, холбогдох бүтээлүүдийн тоймыг танилцуулна. Үүний хүрээнд микрофронтенд архитектурын ойлголт, ISO/IEC 25010 стандарт, мөн сүүлийн үеийн ижил төстэй судалгаануудыг тус тус авч үзэх болно.

2.1 Онолын судалгаа

Энэ хэсэгт судалгааны сэдэв, асуудал, даалгаврыг ойлгоход шаардлагатай үндсэн ойлголтуудын тодорхойлолт, тайлбарыг танилцуулах болно. Микрофронтенд гэх ойлголтыг таниулахын тулд вэб хэрэглээний программ хөгжүүлэлтийн одоогийн хандлагууд, микросервис архитектурын үндсэн зарчмуудыг тайлбарласнаас гадна тестийн үр дүнг үнэлэхэд анхаарч үзсэн ISO/IEC 25010 стандарт, чанарын үзүүлэлтүүдийг мөн дурдсан болно.

2.1.1 Цул архитектур

Төслийн цар хүрээ тэлэхийн хэрээр модулиуд хоорондоо улам их хамааралтай болж эхлэн, нарийн төвөгтэй байдал нэмэгдэж, аль нэг модульд өөрчлөлт оруулах нь бусад модулиудад урьдчилан таамаглах аргагүй үр дагавар авчрах эрсдэлтэй болдог. Энд зөвхөн код цул болоод зогсохгүй, бусад хөгжүүлэгчидтэй хамтран ажиллах эсвэл шинэ хөгжүүлэгч багт нэмэгдэхэд хүндрэлтэй болдог. Зураг 2.1.а-д цул архитектуртай хэрэглээний программын давхаргуудыг харуулав.



Зураг 2.1. Цул архитектур

Үүний улмаас хөгжүүлэлтийн процессыг фронтенд, бэкенд, өгөгдлийн сангийн давхарга

[20] гэх мэтээр хэвтээ чиглэлд задалдаг. Ингэж задалснаар зөвхөн код өөр өөр давхаргад хуваагдаад зогсохгүй, хөгжүүлэлтийн багууд ч тухайн хөгжүүлэлтийн чиглэл болон ашиглаж буй технологиороо хуваагдах боломжтой болдог. Зураг 2.1.б-д Geers-ийн [20] тайлбарласан хэвтээ задлалтыг харуулсан болно. Өөр өөр давхаргад задалсан хэдий ч хэрэглээний программ томорч, нарийн төвөгтэй болох тусам цул программ хангамжийн хөгжүүлэлт байсаар байдаг. [13]-т дурдсанаар цул архитектур нь дараах олон асуудлыг дагуулдаг:

- Засвар үйлчилгээ, хөгжүүлэх, алдааг хянахад хүндрэлтэй.
- Сан нэмэх, шинэчлэхэд код ажиллахгүй болох, тааварлашгүй байдлаар ажиллах эрсдэлтэй.
- Өөрчлөлт оруулахын тулд бүхэл хэрэглээний программыг дахин нэвтрүүлэх шаардлагатай болдог.
- Байршуулалтийн тохиргоо нь бүх бүрэлдэхүүн хэсэгт тохирсон байх ёстой.
- Өргөтгөх боломж хязгаарлагдмал байдаг.
- Сонгосон технологи, фреймворкийг л үргэлжлүүлэн ашиглах шаардлагатай болдог.

Нөгөөтэйгүүр Atlassian [5] цул архитектурын дараах давуу талуудыг онцолсон:

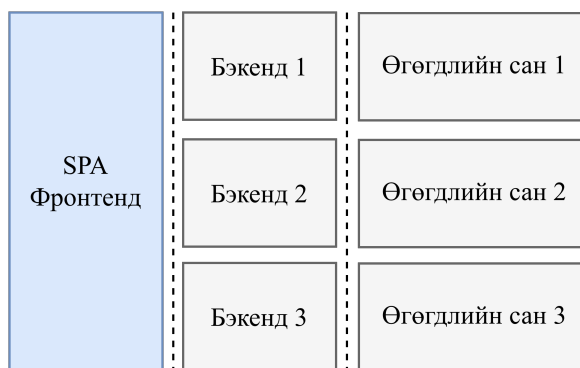
- Бүхэл бүтэн хэрэглээний программыг нэг л файл ашиглан нэвтрүүлдэг.
- Хэрэглээний программ нэг кодын сан дээр суурилан бүтээгддэг.
- Модулиуд хоорондоо нэг процесс дотор ажилладаг тул API дуудлага хийхэд илүү хурдан.
- Нэг кодын санд байдаг тул тестлэхэд хялбар бөгөөд E2E тестлэхэд тохиромжтой байдаг.
- Хэрэглээний программ илүү энгийн үед хүсэлтүүдийг мөшгөх, асуудлыг олоход хялбар байдаг.

2.1.2 Микросервис архитектур

Микросервис архитектур нь бие даан нэвтрүүлэх боломжтой үйлчилгээнүүд дээр тулгуурладаг архитектур юм [5]. Эдгээр үйлчилгээ бүр нь өгөгдлийн сантай байж болно [13]. Үйлчилгээнүүд хоорондоо хөнгөн протоколууд ашиглан харилцдаг [9]. Микросервисийг ойлгохын тулд энэхүү архитектурыг бүрдүүлдэг дараах хэд хэдэн үндсэн зарчмуудыг мэдэх хэрэгтэй:

1. Сул холбоост буюу бусад микросервисүүдэд нөлөөлөхгүйгээр аль нэг микросервист өөрчлөлт оруулах боломжтой байх.
2. SOLID[9] зарчмын дагуу үйлчилгээ бүр зөвхөн нэг л үүрэг хариуцлагатай байх ёстой.
3. Үйлчилгээнүүд нь бие даасан бөгөөд бүх хамаарлуудыг өөртөө агуулсан байдаг.
4. Үйлчилгээний төгсгөлийн цэгүүд нь API хэлбэрээр ил гарсан байдаг. API нь хийсвэрлэл үүсгэдэг бөгөөд бусад үйлчилгээнүүд нь тухайн үйлчилгээнд ашиглагдаж буй технологи, бүтэц, логик хэрэгжүүлэлтийн талаар мэдэх шаардлагагүй байдаг.

Kubernetes [13], Docker [44] гэх мэт хэрэгслүүд нь бэкенд хөгжүүлэгчдэд микросервисүүдийг ашиглах, удирдан зохион байгуулахад тусалдаг. Geers [20] хөгжүүлэлтийн өнөөгийн чиг хандлага бол микросервис бэкенд дээр суурилсан SPA (Single Page Application) фронтенд гэж тайлбарласан байдаг (Зураг 2.2). [5]-р нийтлэлд



Зураг 2.2. Микросервис архитектур

Atlassian компани дараах давуу талуудыг онцолжээ:

- Жижиг багуудаар Шалмаг аргачлалаар ажиллах.
- Үйлчилгээний инстанц үүсгэх нь маш хурдан байдаг.
- Инстанц бүрийг бие даан нэвтрүүлдэг тул бусад бүрэлдэхүүн хэсгүүдийг дахин нэвтрүүлэх шаардлагагүй. Энэ нь хөгжүүлэлтийн амьдралын мөчлөгийг хурдасгадаг.
- Алдааг тусгаарлаж, засахад хялбар байдаг. Кодын шинэчлэлт хийх, шинэ боломжууд нэвтрүүлэх нь хамаагүй хурдан болдог.
- Шинэ фичер бүхий бие даасан нэгжүүдийг хялбархан нэвтрүүлж болно.

- Багууд өөрсдийн хүссэн хэрэгсэл, технологийг сонгох боломжтой.
- Өөрчлөлтийг нэвтрүүлэх нь бүхэл бүтэн хэрэглээний программыг унагах эрсдэл үүсгэдэггүй.
- Багууд илүү бие даасан болдог.

Нөгөө талаас, Харрис өөрийн нийтлэлдээ [9] микросервис архитектурын сул талыг дурджээ:

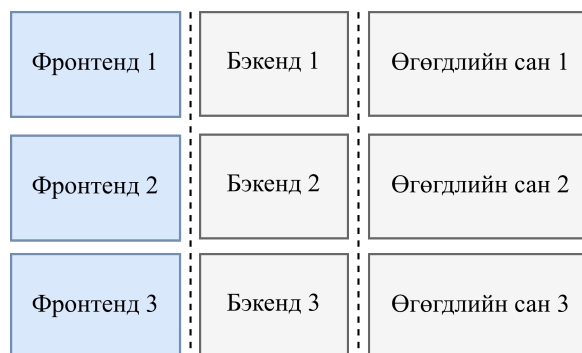
- Олон тооны микросервисүүдийг удирдах нь бэрхшээлтэй бөгөөд үүнийг зөв хийхгүй бол хөгжүүлэлтийг удаашруулах эрсдэлтэй.
- Микросервис бүр тест, байршуулалт, хостинг, хяналт зэрэг өөрийн гэсэн зардалтай байдаг.
- Шинэчлэлтүүдийг зохицуулахын тулд багууд харилцаа холбоо, хамтын ажиллагааны шинэ түвшнийг нэмэх шаардлагатай болдог.
- Нэг бизнесийн логик нь хэд хэдэн инстанц дундуур дамжин ажиллах боломжтой бөгөөд энэ нь алдааг олж засахад хүндрэл учруулдаг.
- Микросервис бүрд зориулсан нийтлэг платформ байдаггүй тул лог хөтлөх стандарт, хяналт зэргийг ижил түвшинд хэрэгжүүлэхэд хэцүү байдаг.
- Нэг баг олон микросервис эзэмшиж болдог тул тухайн микросервисийг аль баг хариуцдаг нь тодорхойгүй байх үед дэмжлэг үзүүлэхтэй холбоотой асуудлуудыг үүсгэдэг.

2.1.3 Нэг хуудаст хэрэглээний программ (SPA)

Миковски болон Пауэлл нар өөрсдийн номондоо [39] SPA-г вэб хэрэглээний программ хөгжүүлэлтийн шинэ, орчин үеийн архитектурын арга барил хэмээн тодорхойлсон байдаг. SPA нь агуулгаа динамикаар шинэчилдэг вэб хэрэглээний программ юм. Энэ нь өөрчлөгдсөн харагдацын хэсгүүдийг л шинэчилж, харуулдаг. SPA нь серверээс өгөгдөл татахын тулд AJAX хүсэлтүүдийг ашигладаг. SPA архитектур нь Angular, Vue, React гэх мэт хамгийн алдартай фреймворкуудад хэрэгжсэн байдаг. SPA архитектурын сул тал нь кодын сан томрох тусам түүнийг арчлах, шинэ боломжууд нэмэхэд илүү хэцүү болдог. Энэ нь ихэвчлэн бүхэл бүтэн хэрэглээний программыг дахин бичих эсвэл дахин зохион байгуулахад хүргэдэг.

2.1.4 Микрофронтенд архитектур

Микросервис архитектурыг фронтенд хөгжүүлэлтэд нэвтрүүлэх нь байршуулалт, хөгжүүлэлт, багийн зохион байгуулалт зэрэгт шинэ боломжуудыг олгодог. Микрофронтенд архитектурыг бий болгохын тулд тэдгээр Микросервис архитектурын зарчмуудыг (Дэд бүлэг 2.1.2-ыг үзнэ үү) хэрэгжүүлэх нь чухал юм. Зураг 2.3-т микрофронтенд хөгжүүлэлтийн хандлагыг харуулав. Энэ хандлага нь фронтендийг удирдах боломжтой бүрэлдэхүүн хэсгүүдэд задлах бөгөөд бүрэлдэхүүн хэсэг бүр нь тодорхой бизнесийн логикийг хариуцдаг. Фронтенд бүрэлдэхүүн хэсэг бүр API ашиглан бэкендтэй харилцдаг бөгөөд энэ нь өөр өөрийн фронтенд кодын сантай байдаг. Микрофронтенд архитектурт *хэвтээ* задлалтаас зайлсхийх хэрэгтэй [17]. Энд багуудыг микрофронтенд хэрэглээний программуудаар нь хуваах хэрэгтэйг онцолсон байдаг. Багууд нь *босоо* чиглэлд бүрэлдэх ёстой бөгөөд баг бүр нэг микро хэрэглээний программыг хариуцаж, тодорхой нэг фичерийг хөгжүүлж нийлүүлэх үүрэгтэй байдаг.



Зураг 2.3. Микрофронтенд архитектур

Мартин Фаулер [17] микрофронтенд архитектурыг ашиглахын давуу талуудыг дурджээ:

- Хэрэглэгчдэд шинэ боломжуудыг үргэлжлүүлэн хүргэхийн тулд хуучин кодын санг хөгжүүлэх явцдаа хэсэг хэсгээр нь шинэчлэн бичих боломжтой.
- Жижиг кодын сангууд дээр ажиллахад илүү хялбар байдаг. Санамсаргүй хамаарал үүсгэхгүйн тулд бүрэлдэхүүн хэсгүүдийн хоорондох хил хязгаарыг тодорхой болгодог.
- Кодын сан бүр өөрийн гэсэн нийлүүлэлтийн дамжлагатай бөгөөд энэ нь кодыг бүтээх, тестлэх, продакшн руу нэвтрүүлэх ажлыг хийдэг. Нэг бүрэлдэхүүн хэсгийг унагах боломжтой тул алдаа гарах үед учрах эрсдэл бага байдаг.

- Багууд нь бие даасан бөгөөд бүх давхаргаа (фронтенд, бэкенд, өгөгдлийн сан) бүрэн хариуцдаг. Энэ нь хөгжүүлэлт болон шинэ боломжуудыг нэвтрүүлэх процессыг илүү хялбар, хурдан болгодог.

Товчхондоо микрофронтенд гэдэг нь том кодын санг жижиг, удирдаж болохуйц хэсгүүдэд хувааж, тэдгээрийн хоорондын хамаарлыг тодорхой болгохыг хэлнэ. Харин сул талуудыг дараах байдлаар онцолжээ:

- Давхардсан сангууд нь өөр өөр микрофронтенд хэрэглээний программуудад ачаалагддаг тул илүү их нөөц зарцуулах эрсдэлтэй.
- Локал хөгжүүлэлт болон продакшн орчны хоорондох ялгаатай байдал.
- Олон микрофронтенд хэрэглээний программтай болох нь удирдах, арчлах олон домэйн, сервер, хэрэгслүүдтэй болоход хүргэдэг.

2.1.5 Интеграцийн төрлүүд

Мартин Фаулерын [38] нийтлэлд интеграци хийх хугацааны хувьд хоёр төрлийн интеграцийг ялгаж үздэг. Интеграцийн хугацаа гэдэг нь микрофронтенд ашиглаж буй хэрэглээний программ тухайн микрофронтендийн эх код руу нэвтрэх эрхтэй болох цаг мөчийг хэлнэ. Микрофронтендийг дээрх хоёр төрлийн цаг хугацааны интеграцийн аль алинд нь хэрэгжүүлэх боломжтой. Гэхдээ тэдгээр нь бүгд микросервисийн үндсэн зарчмуудыг хангадаггүй.

Угсралтын үеийн интеграци (Build-time integration)

Угсралтын үеийн интеграци нь заримдаа компиляци интеграци (compile-time) гэж бас нэрлэгддэг бөгөөд энэ нь микрофронтендийг импортолж буй хэрэглээний программ хөтөч дээр ачаалагдахаас өмнө ашиглагдах микрофронтендийн эх код нь нэгтгэгдсэн байхыг хэлнэ. Энэхүү интеграцийг микрофронтендийг prj багц хэлбэрээр нийтэлж, дараа нь үндсэн хэрэглээний программ рүү хамаарал болгон импортлох замаар хэрэгжүүлэх боломжтой. Микрофронтенд архитектурыг нэвтрүүлэхэд энэ хандлагыг ашиглах нь ойлгоход илүү хялбар боловч микрофронтенд архитектурын үндсэн бүх зарчмыг хангаж чаддаггүй. Аль нэг prj багцад өөрчлөлт орох бүрд тэдгээр багцыг импортолсон микрофронтенд хэрэглээний программуудыг мөн дахин нэвтрүүлэх шаардлагатай болдог. Энэ үед микрофронтендүүд сул холбоост байж чадахгүй бөгөөд импортолсон микрофронтендүүдээсээ илүү их хамааралтай, нягт холбоотой болдог.

Ажиллах үеийн интеграци (Run-time integration)

Ажиллах үеийн интеграцийг сервер талын (server-side), клиент талын (client-side) болон захын (edge-side) интеграци гэж гурван дэд ангилалд хуваадаг [38]. Ажиллах үеийн интеграци гэдэг нь микрофронтендийг импортолж буй хэрэглээний программ хөтөч дээр ачаалагдсаны дараа ашиглагдах микрофронтендийн эх код нэгтгэгдэхийг хэлнэ. Энэ төрлийн интеграцийг тохируулахад илүү төвөгтэй байдаг. Микрофронтендүүдийн хувьд илүү их тохиргоо шаарддаг. Нөгөө талаас, хэрэгжүүлэлтийн энэ арга нь микрофронтендүүдийн сул холбоост байдлыг бий болгодог. Микрофронтендүүдийн байршуулалтыг бие даан хийх боломжтой бөгөөд бүхэл бүтэн хэрэглээний программыг дахин нэвтрүүлэх шаардлагагүй болдог. Захын болон сервер талын интеграци нь илүү их нөөц шаарддаг тул энэхүү ажилд бэлтгэсэн демо загвар болон шилжүүлэлтэд клиент талын интеграцийг ашиглах болно.

Тодорхойлсон интеграцийн төрлүүд дээр үндэслэн микрофронтенд архитектурын аль үндсэн зарчмуудыг хангаж байгааг Хүснэгт 2.1-д харуулав.

Хүснэгт 2.1. Интеграцийн төрлүүдийн харьцуулалт

Үндсэн зарчмууд	Угсралтын үе	Ажиллах үе
Нэг хариуцлагатай (Single responsibility)	x	x
Бие даан нэвтрүүлэх боломжтой (Independently deployable)		x
Фреймворкоос үл хамаарах (Framework agnostic)	x	x
Сул холбоост (Loosely coupled)		x

2.2 Технологийн судалгаа

Энэ хэсэгт судалгааны ажилд сонгосон үндсэн технологиудын онолын үндэслэл, техникийн онцлог, сонголтын шалтгааныг товч авч үзнэ.

2.2.1 JavaServer Faces (JSF)

JavaServer Faces нь Java EE платформын нэг хэсэг болсон серверийн талд бүрэлдэхүүнд суурилсан вэб фреймворк юм. JSF нь хуудасны бүрэлдэхүүн хэсэг бүрийн төлөвийг серверт HttpSession-д ViewState объект хэлбэрт хадгалдаг [14]. Хэрэглэгч хуудастай харилцах бүрт сервер нь ViewState-ийг сэргээж, шаардлагатай үйлдлийг гүйцэтгэн, шинэчилсэн HTML-ийг хариу болгон илгээдэг. Энэхүү request-response мөчлөг нь JSF-ийн амьдралын мөчлөгөөр удирдагддаг бөгөөд дараах зургаан үе шаттайгаар (Зураг 2.4) явагддаг:

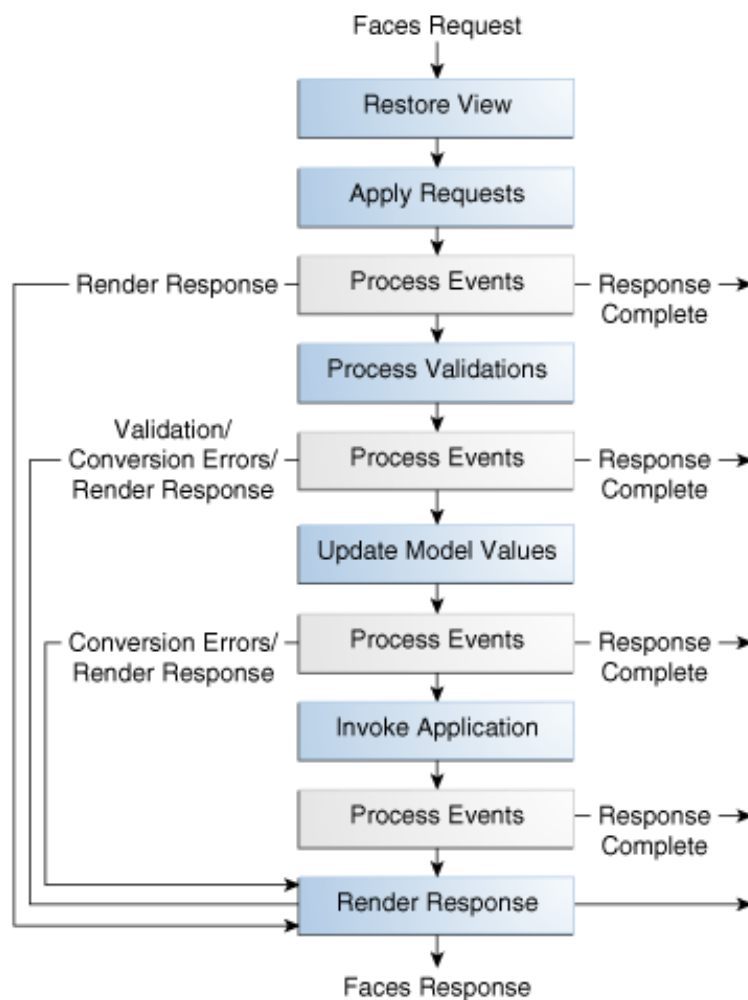
- **Restore View** (Харагдацыг сэргээх): Хэрэглэгчээс хүсэлт ирэх үед сервер тухайн хуудасны бүрэлдэхүүн хэсгүүдийн модыг (component tree) шинээр үүсгэх эсвэл өмнөх хадгалагдсан төлөвөөс (*ViewState*) сэргээдэг.
- **Apply Request Values** (Хүсэлтийн утгуудыг хэрэглэх): Хэрэглэгчээс ирсэн өгөгдлүүдийг (жишээлбэл, формын талбарт оруулсан утгууд) модонд байгаа харгалзах бүрэлдэхүүн хэсгүүдэд оноож өгдөг.
- **Process Validations** (Баталгаажуулалтыг боловсруулах): Оруулсан өгөгдлийн төрлийг зохих хэлбэрт хувиргах (conversion) болон өгөгдсөн дүрэм, шалгууруудад нийцэж байгаа эсэхийг баталгаажуулан шалгана (validation). Хэрэв алдаа гарвал дараагийн шатуудыг алгасаж, шууд хариу дүрслэх шат руу шилждэг.
- **Update Model Values** (Загварын утгуудыг шинэчлэх): Баталгаажсан өгөгдлүүдийг серверийн талын загвар объектууд (backing beans) болон дотоод хувьсагчид руу хуулж, утгуудыг шинэчилнэ.
- **Invoke Application** (Хэрэглээний программын логикийг дуудах): Хэрэглэгчийн хийсэн үйлдэлтэй (жишээ нь, товчлуур дарах, холбоос дээр дарах) холбоотой серверийн талын бизнес логик, функцууд энэ шатанд ажиллана.
- **Render Response** (Хариуг дүрслэх): Бүх шатны үр дүнд шинэчлэгдсэн төлөв болон өгөгдлийг ашиглан эцсийн HTML хариуг үүсгэж, хэрэглэгчийн хөтөч рүү буцаан илгээдэг.

JSF нь энэхүү судалгаанд хост системийн үүрэгтэйгээр ашиглагдаж байгаа шалтгаан нь JSF дээр суурилсан монолог системийг бүрэн дахин бичихгүйгээр орчин үеийн архитектур руу аажмаар шилжих стратегитэй холбоотой юм [18].

2.2.2 *React*

React нь Facebook (Meta)-ийн боловсруулсан, бүрэлдэхүүнд суурилсан JavaScript UI сан бөгөөд Virtual DOM механизмаар хэрэглэгчийн интерфэйсийг үр ашигтайгаар шинэчлэдэг [6]. React-ийн бүрэлдэхүүн хэсэг нь өгөгдлийн оролтоор UI-ийн дүрслэлтийг буцаадаг цэвэр функц хэлбэрт тодорхойлогддаг. Дотоод төлөвийн удирдлагыг *useState*, *useReducer* hook-уудаар, нэмэлт логикийг *useEffect*, *useMemo* зэрэгт тусгасан байдаг.

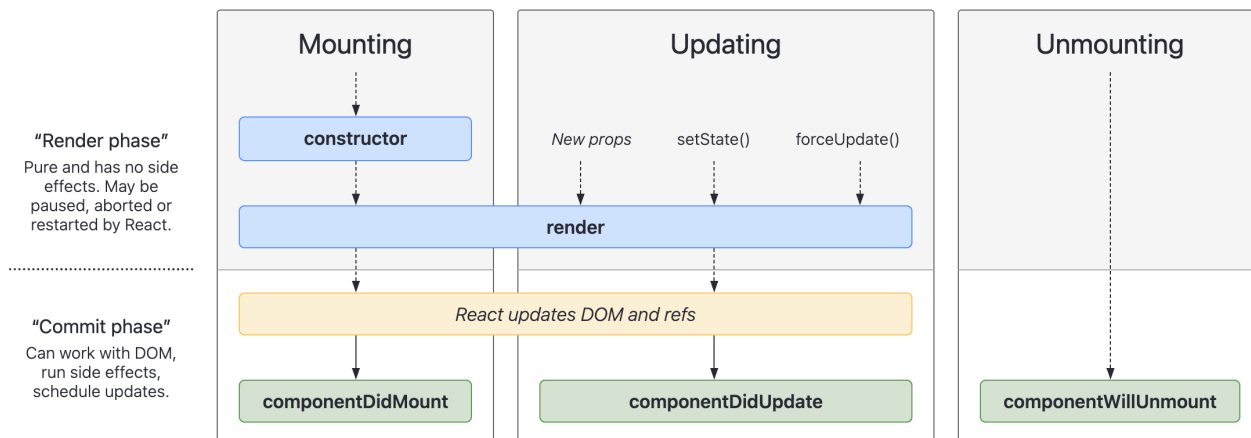
React-ийн бүрэлдэхүүн хэсгийн амьдралын мөчлөг нь үндсэн гурван үе шатаас бүрдэнэ: угсрах (mounting), шинэчлэх (updating), болон салгах (unmounting). Угсрах үе шатанд бүрэлдэхүүн хэсэг анх удаа DOM-д үүсгэгдэн байршдаг бол, шинэчлэх үе шат нь тухайн



Зураг 2.4. JSF хүсэлт боловсруулах амьдралын мөчлөг, Эх сурвалж: [14]

бүрэлдэхүүний дотоод төлөв (state) эсвэл гаднаас дамжигдах өгөгдөл (props) өөрчлөгдөх үед идэвхжиж, UI-г шаардлагатай хэсгүүдэд дахин дүрсэлдэг (re-render). Харин салгах үе шатанд бүрэлдэхүүн хэсэг DOM-оос устгагдах бөгөөд энэ үед санах ойн алдагдлаас сэргийлэх цэвэрлэгээний (cleanup) үйлдлүүд гүйцэтгэгддэг (Зураг 2.5). Орчин үеийн функциональ бүрэлдэхүүн хэсгүүдэд эдгээр амьдралын мөчлөгийн үе шатуудыг дээр дурдсан `useEffect` `hook`-ийн тусламжтайгаар нэгтгэн удирдах бүрэн боломжтой байдаг.

Энэхүү судалгаанд React-ийг Remote MFE-ийн технологи болгон сонгосон үндэслэл нь гурван хүчин зүйлд тулгуурлана: нэгдүгээрт, React-ийн экосистем нь TypeScript, Ant Design, Vite зэрэг хэрэгслүүдтэй нягт нийцдэг; хоёрдугаарт, бүрэлдэхүүнд суурилсан загвар нь MFE-ийн бие даасан байдлын зарчимтай онолын хувьд нийцдэг [26]; гуравдугаарт, Virtual DOM-ийн өөрчлөлт олж тогтоох алгоритм (reconciliation) нь JSF-ийн бүтэн хуудасны дахин дүрслэлттэй харьцуулахад хэсэгчилсэн UI шинэчлэлтийг илүү үр ашигтай гүйцэтгэдэг [4].



Зураг 2.5. React амьдралын мөчлөг, Эх сурвалж: [6]

2.2.3 Webpack Module Federation

Webpack Module Federation нь Webpack 5-д нэвтрүүлсэн залгаас бөгөөд бие даан угсарч, байршуулсан JavaScript хэрэглээний программуудыг ажиллах үед динамикаар хоорондоо кодоо хуваалцах боломж олгодог [55]. Уг механизмд хоёр үндсэн дүр тоглогч оролцдог: *Host* (хэрэглэгч) болон *Remote* (нийлүүлэгч). *Remote* нь өөрийн экспортлох бүрэлдэхүүнийг `remoteEntry.js` файлаар зарладаг бөгөөд *Host* нь энэ файлыг ачаалж, шаардлагатай бүрэлдэхүүнийг динамикаар импортолдог.

`shared` тохиргооны хэсэгт хуваалцах сангуудыг зарлаж, `singleton: true` тэмдэг тавьснаар React зэрэг сан нэг л хувилбарт ажиллах баталгааг хангадаг [47]. Энэхүү “shared singleton” механизм нь холимог MFE орчинд хэд хэдэн React instance зэрэгцэн ажиллахад үүсдэг hook-ийн зөрчилдөөний асуудлыг шийддэг тул онолын хувьд чухал ач холбогдолтой.

Энэхүү судалгаанд `@originjs/vite-plugin-federation` хэрэгслийг ашигласан бөгөөд энэ нь Webpack 5-ийн Module Federation-ийн зарчмыг Vite-ийн угсралтын орчинд (esbuild + Rollup) хэрэгжүүлсэн хувилбар юм [8]. Уг хэрэгсэл нь Webpack-тай жинхэнэ нийцтэй биш тул зарим боломжийн хязгаарлалттай байдаг, ялангуяа nested federation (алслагдсан модуль дахь алслагдсан модуль) топологи нь статик хүргэлтийн орчинд runtime алдаа үүсгэдэг нь энэ судалгааны явцад тодорхойлогдсон.

2.2.4 Холимог микрофронтенд архитектур

Холимог микрофронтенд архитектур гэдэг нь серверийн талд дүрслэлт (SSR) хийдэг уламжлалт фреймворк болон клиентийн талд дүрслэлт (CSR) хийдэг орчин үеийн фреймворкийг нэг нэгдсэн хэрэглээний программ дотор зэрэгцүүлэн ажиллуулах

архитектурын загвар юм [18]. Энэ загварт SSR хост систем нь бүрхүүл буюу үндсэн аппликейшний үүрэг гүйцэтгэж, нэг буюу хэд хэдэн CSR-д суурилсан алслагдсан модулиудыг ачаалж, хөтчийн DOM дотор тодорхой оруулах цэгт байрлуулдаг [37]. JSF зэрэг SSR-д суурилсан фреймворк нь серверт бүрэн бэлтгэгдсэн HTML хуудсыг хариу болгон илгээдэг бол React зэрэг CSR-д суурилсан фреймворк нь хөнгөн HTML-ийг татан авч, UI-г хөтчид JavaScript-ийн тусламжтайгаар дүрсэлдэг [22]. Эдгээр хоёр фреймворкыг хослуулах нь дараах онолын сорилтуудыг үүсгэдэг:

1. **Чиглүүлэлт:** SSR фреймворкийн сервер талын URL хандалтын дүрмүүд нь CSR модулийн клиент талын чиглүүлэгчтэй зөрчилддөг. JSF-ийн FacesServlet нь *.html хэлбэрийн хүсэлтийг хариуцдаг бол React Router нь бүх зам дор URL шилжилтийг хянахыг хичээдэг [26] тул холимог орчинд CSR модуль нь Hash-based routing хэрэглэхэд хүрдэг.
2. **Сейшн хяналт:** SSR систем нь хэрэглэгчийн нэвтрэлтийн мэдэгдлийг серверийн HttpSession-д хадгалдаг бол CSR модуль нь JWT токенийг localStorage-д тохируулдаг [35]. Хоёр хадгалалтын механизм зэрэгцэж байх нь өгөгдлийн нэг эх сурвалжтай байдлыг дутагдуулж, нэвтрэлтийн мэдээллийн зөрчилдөөн үүсгэж болдог.
3. **CSS тусгаарлалт:** SSR фреймворкийн глобал CSS дүрмүүд нь CSR модулийн компонентийн загварт санамсаргүйгээр нөлөөлж болдог. Shadow DOM эсвэл CSS Modules хэрэглэхгүйгээр ийм загварын цутгалт (style bleeding) зайлшгүй тохиолддог [48].
4. **Дундын сан:** SSR хост болон CSR нийлүүлэгч нь ижил JavaScript санг (жишээ нь React, Ant Design) тус тусдаа ачаалах эрсдэлтэй бөгөөд энэ нь сүлжээний хэт ачааллыг нэмэгдүүлж, санах ойд давхар instance үүсгэдэг [4]. Module Federation-ийн shared singleton механизм нь энэ асуудлыг онолын хувьд шийддэг боловч тохиргооны алдаа гарах нь нийтлэг байдаг.
5. **Угсралтын хэрэгсэл:** SSR систем нь Maven, Gradle зэрэг JVM-д суурилсан угсралтын хэрэгслийг ашиглах бол CSR модуль нь Vite, Webpack зэрэг JavaScript-д суурилсан хэрэгслийг ашигладаг. Эдгээр хоёр угсралтын мөчлөгийг нэгдсэн CI/CD паплайнд уялдуулах нь нэмэлт зохицуулалт шаарддаг [8].

Эдгээр сорилтуудыг шийдвэрлэх нийтлэг онолын хандлага нь Fowler [18]-ийн *Strangler Fig* загвар юм. Энэ загварт хуучин SSR системийг нэгэн зэрэг орхилгүйгээр шинэ CSR

модулиудыг аажмаар сольж, эцэстээ хуучин системийг бүрэн орлуулна. Уг загварын давуу тал нь бизнесийн тасралтгүй байдлыг хангадагт оршдог бол сул тал нь шилжилтийн хугацаанд хоёр технологийн давхаргыг нэгэн зэрэг засвар үйлчилгээ хийх шаардлагатай байдаг явдал юм [45].

2.2.5 *Spring Boot ба Реактив Микросервис*

Spring Boot нь Spring Framework-д суурилсан Java-д зориулсан микросервис хөгжүүлэлтийн тулгуур хэрэгсэл бөгөөд “конвенц дагасан тохиргоо” (convention over configuration) зарчмаар autoconfiguration, embedded server, production-ready хяналтын боломжуудыг нэгдсэн байдлаар хангадаг [42]. Уг судалгаанд Spring Boot 3.2.x болон Spring WebFlux-ийн хослолыг ашиглав.

Spring WebFlux нь Reactor-д суурилсан реактив програмчлалын загвар дээр ажиллах бөгөөд Mono<T> (нэг элемент) болон Flux<T> (олон элемент) реактив stream-ийг API болгон ашигладаг [9]. Уламжлалт Spring MVC-ийн request-per-thread загвараас ялгаатай нь WebFlux нь Netty-ийн Non-blocking I/O event-loop дээр суурилж, CPU цөмийн тоотой тэнцэх цөөн thread-ээр олон зэрэгцэх холболтыг удирдах боломжтой. Энэ архитектурын шийдэл нь ялангуяа өндөр зэрэгцэх ачааллын үед санах ойн хэрэглээг мэдэгдэхүйц бууруулдаг [13].

2.3 Сэдвийн судалгаа

Энэ хэсэгт микрофронтенд архитектур зохиомж, түүний янз бүрийн шинж чанар, хэрэгжүүлэлт болон өмнө нь хийгдсэн судалгааны ажлуудын тоймыг хүргэх болно. Одоо байгаа ном зохиол, судалгааг тоймлохын гол зорилго нь микрофронтенд архитектур зохиомжтой холбоотой орчин үеийн тэргүүлэх шийдлүүд, холбогдох хэрэгсэл, багц, санг судлахад оршино.

2.3.1 *Судалгааны ажлын хайлт*

Холбогдох өгүүллүүдийг олохын тулд Scopus, DiVA өгөгдлийн санг ашигласан. Хайлтын утгад микрофронтенд архитектур зохиомжийн нэршлийн өөр өөр хувилбаруудыг ашигласан болно. Үүнд: micro- OR microfrontend OR "micro" OR "*"** гэсэн хайлтын утгаар Scopus өгөгдлийн сангаас нийт 47 илэрц, microfrontend OR micro OR micro - гэсэн хайлтын утгаар DiVA сангаас 5 илэрц олдсон. Өгүүллийн гарчиг, хураангуй, эсвэл түлхүүр үгсэд хайлтын утга орсон бүтээлүүдийг сонгож авсан. Олдсон өгүүллүүдийг уншиж танилцсаны дараа судалгааны арга зүй, сэдвийн хамаарал, сэдвийг хэрхэн хөндсөн байдал зэрэгт нь үндэслэн 10 өгүүллийг тоймлон үзэхээр сонгосон. Эдгээрийн 1 нь DiVA сангаас,

үлдсэн 9 нь Scopus өгөгдлийн сангаас авагдсан болно.

2.3.2 Сонгож авсан судалгааны ажил

[40] дугаартай бүтээлд зохиогчид нь одоо ашиглагдаж буй QL4POMR бэкенд программ хангамжийн шийдэл дээр тулгуурлан эмч, өвчтөн хоорондын харилцан яриаг загварчлах зориулалттай чатботыг хэрхэн хэрэгжүүлсэн талаар тайлбарласан байдаг. Энэхүү харилцан ярианы систем нь patientMFE болон physicianMFE гэж нэрлэгдсэн хоёр микрофронтед хэрэглээний программаас бүрдэнэ. Уг бүтээлд микрофронтед хэрэгжүүлэхэд ашиглагддаг Piral, SystemJS, Single SPA, Webpack Module Federation, Bit.Dev зэрэг зарим хэрэгслүүдийг дурджээ. Хэрэгжүүлэлтийн техникийн шийдэл нь зөвхөн тухайн системийн тохиргоонд зориулан ашигласан технологиудаар хязгаарлагдсан байна.

[47] дугаартай бүтээлд Стефановска болон Трайковик нар домэйн суурилсан зохиомж нь микрофронтед архитектурын үндсэн гол ойлголт болохын хувьд түүний тоймоор судалгаагаа эхлүүлсэн байна. Тэд одоо байгаа техникийн шийдлүүдийг чанарын үзүүлэлтүүд ашиглан үнэлжээ. Үнэлгээ хийсний дараа тэд кодын дахин ашиглалтыг чухалчилсан өөрсдийн техникийн шийдлээ санал болгосон. Энэхүү техникийн шийдэл нь 2 ажлын явцаас бүрдсэн. Эхний ажлын явцад вэб компонент нь фронтенд хэрэглээний программыг бүхэлд нь агуулах бүрхүүл байдлаар хэрхэн ажилладгийг тайлбарласан. Хоёр дахь ажлын явц нь модулийн шинж чанарт төвлөрсөн бөгөөд үүнийг Module Federation Plugin ашиглан хэрэгжүүлсэн байна. Тэдний дурдсан Module Federation Plugin-ийг ашигласнаар олж авах боломжтой нэг шинж чанар нь өргөжүүлэх боломж юм. Учир нь микрофронтед хэрэглээний программуудыг статик JavaScript файл хэлбэрээр татаж авдаг байна. Санал болгосон хэрэгжүүлэлтийн үр дүнд хэрэглээний программын хөгжүүлэлтээс үйлдвэрлэл рүү шилжих амьдралын мөчлөгийн хугацаа илүү хурдан болсон байна.

[43] дугаартай бүтээлд вэб хөгжүүлэлт дэх микрофронтед архитектурын ойлголтуудыг нарийвчлан судалсан байдаг. Зохиогчид үүнийг микросервис архитектурын дараагийн салшгүй, зүй ёсны хөгжил гэж үзсэн. Уг судалгаанд вэб болон бэкенд хөгжүүлэлтийн уламжлалт болон микросервис архитектурын хандлагуудыг харьцуулж, микрофронтедийн өнөөгийн туршлагуудыг судалжээ. Хөгжүүлэгчдийн туршлагад үндэслэн кэйс судалгаа хийсэн бөгөөд чиглүүлэлт, статик файл дамжуулалт, репозиторын зохион байгуулалт зэрэгт тулгардаг сорилтуудыг онцолсон байна. Микрофронтед нь том баг, нарийн төвөгтэй, томоохон хэмжээний төслүүдэд илүү тохиромжтой гэж дүгнэсэн. Харин цөөн хөгжүүлэгчтэй багийн хувьд уг архитектурыг дэмжиж ажиллахад нэмэлт хүчин чармайлт шаарддаг бөгөөд төслийн бүтэц, хөгжүүлэлтийн процессыг нэлээд төвөгтэй болгодог байна. Мөн уг архитектурын давуу талыг бүрэн ашиглахын тулд хөгжүүлэгчдээс тодорхой

хэмжээний туршлага шаарддаг төдийгүй нийгэмлэгийн зүгээс үүнийг бүрэн дэмжих “бэлэн шийдлүүд” хомс байгааг дурджээ.

[50] дугаартай өгүүллийн зорилго нь хөгжүүлэлтийн багийн бүтээмжийг нэмэгдүүлэх, нөөцийн зарцуулалтыг сайжруулахад чиглэсэн платформын шийдлийг танилцуулахад оршино. Уг платформ нь нүсэр, нийлмэл программ хангамжийн хөгжүүлэлтийг төвлөрсөн бус арга зүйгээр хөнгөвчлөх бөгөөд микросервис болон микрофронтендийг үүсгэх, ашиглах процессыг дэмждэг. Энэхүү судалгаа нь микросервис болон микрофронтенд архитектурын давуу тал болон сорилтуудыг ерөнхийд нь тоймлон харуулсан. Зэрэгцээ хөгжүүлэлт болон өргөтгөх боломж нь эдгээр архитектурын хамгийн том давуу тал бөгөөд ингэснээр нөөцийг илүү үр дүнтэй ашиглах боломж бүрддэг байна. Мөн фронтендийн хэсгүүдийг алхам алхмаар шинэчлэх боломж нь уламжлалт цул архитектуртай харьцуулахад хөгжүүлэгчийн бүтээмжийг эрс нэмэгдүүлдэг. Эцэст нь, зохиогчид бүтээгч, хэрэглэгч болон администраторуудад зориулсан гурван хэсэгтэй платформыг санал болгосон нь системийг хөгжүүлэхэд тулгардаг хүндрэлүүдийг илүү сайн ойлгох, цаашлаад платформын үйл ажиллагааг сайжруулахад чухал түлхэц болжээ.

[41] дугаартай бүтээлд микрофронтендийн талаарх хайгуулын судалгаа хийж, хөгжүүлэгчдээс ярилцлага авсан байна. Энэхүү судалгаа нь системийн өөрчлөгдөх боломж болон ерөнхийлөн ашиглагдах байдал зэрэг чанарын үзүүлэлтүүд дээр төвлөрч, микрофронтендийн давуу тал, тулгарч буй асуудлуудыг тодруулахыг зорьжээ. Судалгааны хүрээнд нэг нь SPA, нөгөө нь микрофронтенд архитектур ашигласан хоёр өөр фронтенд хэрэглээний программ хөгжүүлж туршсан байна. Үүний дараа уламжлалт болон микрофронтенд архитектурын аль алинд нь туршлагатай хөгжүүлэгчдээс ярилцлага авч, өөрчлөгдөх боломжийг үнэлэхийн тулд хэд хэдэн хэмжүүр ашиглажээ. Эдгээр өгөгдөлд үндэслэн микрофронтенд архитектурыг сонгох үед анхаарах ёстой эрсдэлүүдийг хэлэлцсэн байна. Эцсийн дүгнэлтдээ Монтелиус ярилцлагын үр дүнг туйлын үнэн гэхээсээ илүүтэйгээр өөр өнцгөөс харсан хандлага гэж үзэх нь зүйтэйг онцолсон бөгөөд туршилтын загваруудаас гарсан үр дүнг бүх SPA болон микрофронтендүүдэд шууд хамааруулан ерөнхийлөн дүгнэх боломжгүйг сануулжээ.

[4] дугаартай бүтээлд Амеер болон түүний нөхөд SPA болон микрофронтенд архитектурын гүйцэтгэлийн үзүүлэлтүүдийг бодит туршилтаар харьцуулан судалсан байна. Тэд LCP болон TPI зэрэг вэб гүйцэтгэлийн хэмжүүрүүдийг ашиглан микрофронтенд нь багийн бие даасан байдлыг хангадаг боловч сүлжээний ачаалал нь уламжлалт SPA-ээс муу байх эрсдэлтэйг нотолжээ. Энэхүү судалгаа нь микрофронтенд архитектурыг сонгохдоо гүйцэтгэлийн золиосыг зайлшгүй тооцоолох шаардлагатайг сануулснаараа онцлог юм.

[45] дугаартай өгүүлэлд Силва нар микрофронтенд архитектурын чиглэлээр хэвлэгдсэн

100 гаруй эрдэм шинжилгээний бүтээлд системчилсэн тойм хийжээ. Уг судалгаагаар микрофронтендийн хамгийн гол давуу тал нь системийг тусдаа байршуулах чадамж болохыг тодорхойлсон байна. Харин цаашид шийдвэрлэвэл зохих гол сорилтуудад хэрэглээний программ хооронд төлөв хуваалцах, аюулгүй байдал болон хэрэглэгчийн баталгаажуулалт зэрэг асуудлууд багтаж байгааг тэд онцлон тэмдэглэжээ.

[8] дугаартай бүтээлд Бжелотомич нар микрофронтенд боловсруулалтын орчин үеийн хэрэгслүүд болох Webpack болон Vite-ийн гүйцэтгэлийг харьцуулан шинжилсэн байна. Тэд хөгжүүлэгчийн бүтээмж болон дотоод орчны билд хийх хурдыг хэмжсэн бөгөөд React орчинд Vite ашиглах нь хөгжүүлэлтийн процессыг эрс хурдасгадаг болохыг харуулсан. Гэсэн хэдий ч энэ нь зөвхөн хөгжүүлэлтийн орчинд хэмжигдсэн бөгөөд үйлдвэрлэлийн ачааллыг бүрэн тооцоолоогүй гэсэн хязгаарлалттай байв.

[35] дугаартай судалгаанд хуучин уламжлалт цул хэрэглэгчийн интерфэйсийг задалж, микрофронтенд архитектур руу шилжүүлэх практик туршлагыг тайлбарласан байдаг. Маннисто нар системийг үе шаттайгаар салгах нь кодын модульчлалыг хэрхэн сайжруулж байгааг үнэлжээ. Уг бүтээл нь хуучин технологиос орчин үеийн архитектур руу шилжих үед тулгардаг саад бэрхшээлүүдийг бодит төсөл дээр харуулснаараа практик ач холбогдолтой юм.

[48] дугаартай өгүүлэлд Тайби болон Меззалира нар микрофронтенд архитектурыг программхангамжийн архитектурын нарийвчилсан өнцгөөс судалсан байна. Тэд микрофронтендүүдийн хоорондын чиглүүлэлт болон өгөгдөл дамжуулах үед үүсэх сүлжээний ачаалал нь системийн нийт гүйцэтгэлд хэрхэн нөлөөлдгийг шинжилжээ. Мөн системийг хэт олон жижиг хэсгүүдэд хуваах нь засвар үйлчилгээний зардлыг нэмэгдүүлж, бүтцийг нийлмэл болгодог болохыг анхааруулсан байна.

2.3.3 Сонгож авсан судалгааны ажлын тойм

Сонгож авсан 10 судалгааны ажлын хөндсөн үндсэн сэдвүүдийг Хүснэгт 2.2-д нэгтгэн харуулав.

Хүснэгт 2.2. Сонгож авсан судалгааны ажлын тойм

Судалгааны чиглэл	Түлхүүр үгс	Эх сурвалж
Модульчлал	Monolith to MFE, Reusability, Migration	[35], [43]
Гүйцэтгэл	ISO 25010, LCP, TTI, Network overhead	[4], [48], [47]
Аюулгүй байдал	State management, Auth, Session sharing	[45], [40]
Хөгжүүлэгчийн бүтээмж	DX, Parallel development, Build time	[8], [50], [41]

Судалгааны ажлуудыг шинжлэн үзэхэд хэд хэдэн нийтлэг ололт болон хязгаарлагдмал

байдлууд ажиглагдаж байна. Тухайлбал, [35], [43] зэрэг бүтээлүүдэд уламжлалт цул системийг задлах онолын арга зүйг судалсан ч хуучин төлөв хадгалдаг системийг орчин үеийн микрофронтендтэй уялдуулах бодит техникийн шийдэл дутмаг байна. [4], [48], [47] зэрэг судалгаанууд нь микрофронтендийн сүлжээний саатлыг амжилттай хэмжсэн ч холимог орчин дахь серверийн нөөцийн давхардлыг тооцоолоогүй орхигдуулжээ. [45], [40] зэрэг судалгаанууд микрофронтенд хооронд төлөв хуваалцахыг гол сорилт гэж онцолсон ч төлөв хадгалдаг болон төлөвгүй архитектур хоорондын интеграцийн бодит шийдлийг санал болгоогүй. [8], [50], [41] зэрэг судалгаанууд нь зэрэгцээ хөгжүүлэлтийн хурдыг хэмжсэн ч хоёр өөр технологийг нэгэн зэрэг удирдах үед хөгжүүлэгчид үүсэх танин мэдэхүйн ачааллыг үнэлээгүй орхисон байна.

2.4 ISO 25010 стандарт

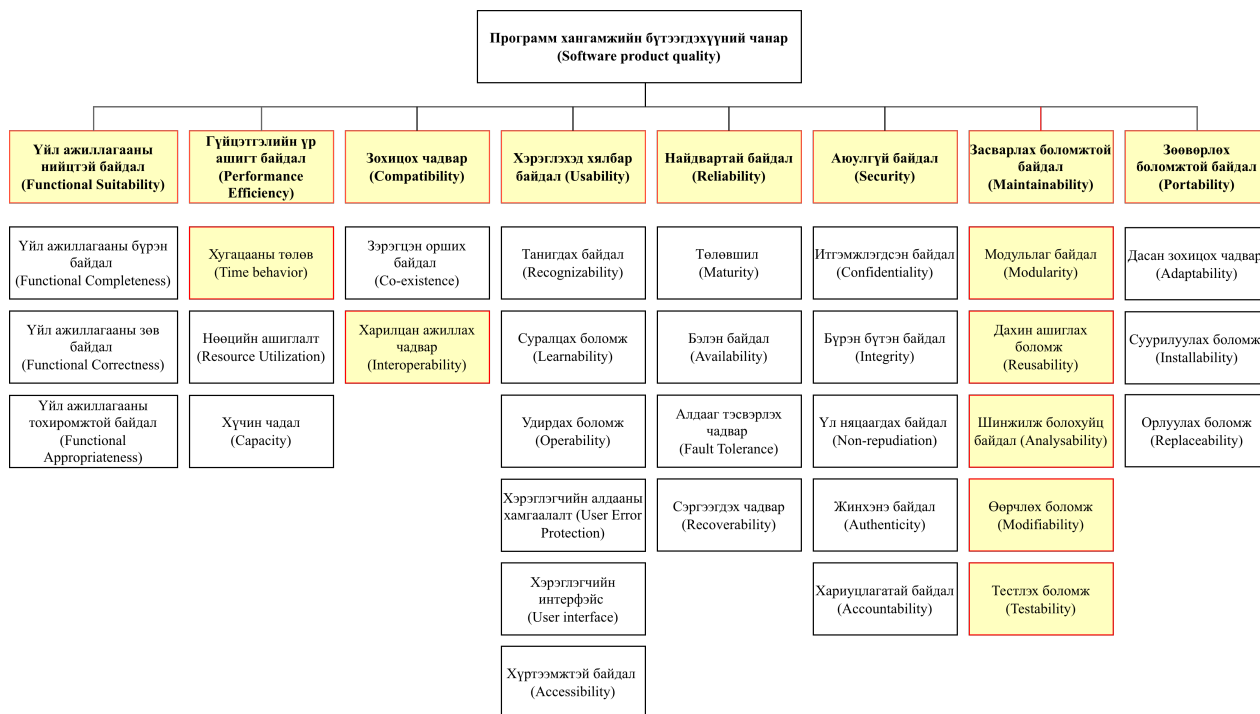
Программ хангамж болон компьютерын системүүдэд хамаарах бүтээгдэхүүний чанарын үзүүлэлтүүдийг ISO/IEC 25010 стандартад заасан байдаг. Чанарын үзүүлэлтүүд (Quality attributes буюу QA) нь олон төрлийн программ хангамжийн шийдэл, хэрэглээний программыг тодорхойлох, хэмжих болон үнэлэхэд ашиглагддаг давтагдашгүй, нарийн тодорхой нэр томъёонууд юм. Энэхүү судалгааны тухайд, тестийн үр дүнг хэмжихийн тулд тус стандартаас хэсэг бүлэг чанарын үзүүлэлтүүдийг сонгож авсан. Зураг 2.6-т бүтээгдэхүүний чанарын загварыг нийт үзүүлэлт, тэдгээрийн ангиллын хамтаар харуулахын сацуу тусгайлан сонгосон үзүүлэлтүүдийг мөн багтаан харууллаа. Энэхүү судалгаанд ашиглах сонгосон чанарын үзүүлэлтүүдийн талаар илүү сайн ойлголт өгөхийн тулд тэдгээрийн тодорхойлолт болон тайлбарыг дараах дэд хэсгүүдэд орууллаа.

2.4.1 Хугацааны төлөв (Time behavior)

[25]-т энэ үзүүлэлтийг “тухайн бүтээгдэхүүн эсвэл систем нь өөрийн үйл ажиллагааг гүйцэтгэхдээ хариу үйлдэл үзүүлэх хугацаа, боловсруулалтын хурд болон нэвтрүүлэх чадвар нь тавигдсан шаардлагыг хэр зэрэг хангаж буй түвшин” гэж тодорхойлсон байдаг.

2.4.2 Харилцан ажиллах чадвар (Interoperability)

Энэ үзүүлэлт нь “хоёр буюу түүнээс дээш системүүд хоорондоо мэдээлэл солилцож, уг солилцсон мэдээллээ ашиглаж чадах түвшин”-г илэрхийлнэ [25].



Зураг 2.6. Бүтээгдэхүүний чанарын загвар ба сонгосон чанарын үзүүлэлтүүд

2.4.3 Модульлаг байдал (Modularity)

Энэ үзүүлэлтийг “систем эсвэл компьютерын программ нь аль нэг бүрэлдэхүүн хэсэгтээ өөрчлөлт оруулахад бусад хэсгүүддээ хамгийн бага нөлөө үзүүлэхүйц салангид хэсгүүдээс бүрдсэн байх хэмжүүр” гэж тодорхойлдог [25]. Энэ нь микрофронтенд архитектур руу амжилттай шилжихэд чухал гүйцэтгэх гол үзүүлэлтүүдийн нэг юм.

2.4.4 Дахин ашиглах боломж (Reusability)

Энэ үзүүлэлтийг “аливаа нэг нөөц, бүрэлдэхүүнийг нэгээс олон системд, эсвэл өөр шинэ нөөц бүтээхэд ашиглаж болох түвшин” хэмээн тодорхойлдог [25]. Энэ нь микрофронтенд архитектурт шилжих процессад ихээхэн эерэг нөлөө үзүүлнэ гэж үздэг.

2.4.5 Шинжилж болохуйц байдал (Analysability)

Энэ үзүүлэлтийг “бүтээгдэхүүн эсвэл системийн аль нэг хэсэгт өөрчлөлт оруулахад гарах нөлөөллийг үнэлэх, доголдол болон алдааны шалтгааныг оношлох, эсвэл өөрчлөх шаардлагатай хэсгүүдийг тодорхойлох процесс хэр үр дүнтэй, үр ашигтай байх түвшин” гэж тодорхойлжээ [25]. Энэхүү үзүүлэлт нь кодоод дүн шинжилгээ хийх болон алдааг олж

засварлахад хэр хялбар байгаагаар илэрдэг.

2.4.6 Өөрчлөх боломж (Modifiability)

Энэ үзүүлэлтийг “бүтээгдэхүүн, системд шинээр алдаа доголдол үүсгэхгүйгээр, эсвэл одоо байгаа чанарыг нь бууруулахгүйгээр үр дүнтэй бөгөөд үр ашигтай өөрчлөлт оруулах боломжтой түвшин” хэмээн тодорхойлдог [25]. Хэрэглээний программын өөрчлөх боломж нь кодын санд өөрчлөлт оруулахад хэр хялбар байгааг харуулдаг үзүүлэлт юм.

2.4.7 Тестлэх боломж (Testability)

Энэ үзүүлэлтийг “систем, бүтээгдэхүүн эсвэл түүний бүрэлдэхүүн хэсгийн хувьд тестийн шалгуурыг тогтоож, уг шалгуурыг хангасан эсэхийг баталгаажуулах тестийг гүйцэтгэх боломжтой байдлын түвшин” гэж тодорхойлсон байдаг [25]. Өөрөөр хэлбэл, энэ нь тухайн бүрэлдэхүүн хэсэг, модулийг янз бүрийн тестийн аргуудаар хэр хялбар тестэлж болохыг илтгэнэ.

2.4.8 Туршилтын багаж хэрэгслийн судалгаа

ISO/IEC 25010 стандартаас сонгосон чанарын үзүүлэлтүүдийг объектив байдлаар хэмжихийн тулд тухайн үзүүлэлт бүрт тохирсон тусгай хэмжилт, шинжилгээний хэрэгслийг ашиглах шаардлагатай. Дараах дэд хэсэгт сонгосон хэрэгсэл тус бүрийн үндэслэл, хэмжих чадавхийг товч тайлбарлав.

Google Lighthouse

Google Lighthouse нь вэб хэрэглээний программын гүйцэтгэлийн үзүүлэлтүүдийг автоматаар хэмждэг нээлттэй эхийн шинжилгээний хэрэгсэл бөгөөд Chrome хөтчид болон командын мөрт ажиллана [33]. Уг хэрэгсэл нь ISO/IEC 25010-ийн “Хугацааны төлөв” үзүүлэлтэд шууд харгалзах дараах хэмжүүрүүдийг гаргадаг:

- First Contentful Paint (FCP): Хөтөч хуудасны анхны агуулгыг дэлгэцэд гаргах хүртэлх хугацаа.
- Largest Contentful Paint (LCP): Хуудасны хамгийн том харагдах элемент дэлгэцэд бүрэн бэлтгэгдэх хугацаа.
- Time to Interactive (TTI): Хуудас хэрэглэгчийн оролтод бүрэн хариу үзүүлэх болтол өнгөрөх хугацаа.

- Total Blocking Time (TBT): FCP болон TTI хооронд JavaScript-ийн гол урсгалыг блокловсон нийт хугацаа.

Энэ судалгааны ажлаар JSF дээрх хуудас болон React MFE модулийг нэгэн ижил нөхцөлд Lighthouse-аар хэмжиж, “Хугацааны төлөв” үзүүлэлтийн дагуу харьцуулалт хийнэ. Хэмжилтийг `--throttling-method=simulate` тохиргоотойгоор гурав давтаж, дундаж утгыг авна [4].

SonarQube

SonarQube нь кодын чанарын статик шинжилгээний платформ бөгөөд Java, TypeScript, JavaScript зэрэг олон программчлалын хэл дэмждэг [46]. ISO/IEC 25010-ийн “Шинжилж болохуйц байдал” болон “Өөрчлөх боломж” үзүүлэлтүүдийг хэмжихэд SonarQube-ийн дараах хэмжүүрүүдийг ашиглана:

- Cyclomatic Complexity: Функц, метод бүрийн мөчлөгийн нарийн төвөгтэй байдлыг тоогоор илэрхийлнэ. Бага утга нь шинжилж болохуйц байдал өндөр гэсэн утгатай.
- Code Duplication (%): Кодын санд давхардсан мөрийн хувь. Өндөр давхардал нь өөрчлөх боломжийг бууруулдаг.
- Technical Debt (цаг): Кодыг стандартын дагуу болгоход зарцуулах тооцоолсон хугацаа.
- Code Smells: Засварлавал зохих архитектурын дутагдлуудын тоо.

`mvn sonar:sonar` командаар бэкенд Java сервисүүдийг, `sonar-scanner` командаар модулиудыг тус тус шинжилж, хоёр архитектурын нэгтгэлтийн хэмжүүрүүдийг харьцуулна [47].

madge

madge нь JavaScript болон TypeScript кодын хамаарлын график дүрслэх нээлттэй эхийн хэрэгсэл юм [34]. ISO/IEC 25010-ийн “Модульлаг байдал” үзүүлэлтэд тохирох дараах шинжилгээг гүйцэтгэнэ:

- Цикл хамаарал (Circular dependency) илрүүлэлт: Модулиуд хоорондоо дугуй хамааралтай байх нь модульчлалын зарчмыг зөрчдөг.
- Хамаарлын гүн (Dependency depth): Нэг модуль хэдэн давхаргын модульд тулгуурлаж байгааг хэмжинэ. Гүн их байх нь тусгаарлалт муу гэдгийг илтгэнэ.

`madge --circular src/` командаар циклийн хамааралыг шалгаж, `madge --image deps.png src/` командаар харагдахуйц граф үүсгэн архивлана.

Jest ба Cypress

Jest нь JavaScript болон TypeScript-д зориулсан нэгжийн тестийн хэрэгсэл бөгөөд React компонентийн тусгаарлагдсан тестэд өргөн ашиглагддаг [27]. Cypress нь end-to-end (E2E) тестийн хэрэгсэл бөгөөд бодит хөтөч дээр хэрэглэгчийн үйлдлийг дуурайн шалгадаг [11]. ISO/IEC 25010-ийн “Тестлэх боломж” үзүүлэлтийг хэмжихэд дараах хэмжүүрүүдийг ашиглана:

- Нэгжийн тестийн хамралт (%): Jest-ийн `--coverage` тэмдэгээр гаргасан мөр, салаа, функцийн хамралтын хувь.
- E2E тестийн тоо: Cypress-д бичигдсэн тестийн тоо болон тест бичихэд тулгарсан бэрхшээлийн субъектив үнэлгээ (1–5 оноо).
- Mock шаардлага: Jest-д нэгжийн тест бичихэд шаардагдах mock, stub-ийн тоо, тоо их байх нь тусгаарлалт муу, тестлэх боломж бага гэсэн дохио юм.

Хүснэгт 2.3. ISO/IEC 25010 үзүүлэлт ба харгалзах хэмжилтийн хэрэгсэл

ISO 25010 үзүүлэлт	Хэрэгсэл	Гол хэмжүүр
Хугацааны төлөв	Google Lighthouse	FCP, LCP, TTI, TBT
Харилцан ажиллах чадвар	Chrome DevTools	Network request, CORS алдаа
Модульлаг байдал	madge	Циклийн хамаарал, хамаарлын гүн
Дахин ашиглах боломж	SonarQube	Code duplication %
Шинжилж болохуйц байдал	SonarQube	Cyclomatic complexity, Technical Debt
Өөрчлөх боломж	SonarQube	Code Smells, Modifiability индекс
Тестлэх боломж	Jest, Cypress	Хамралт %, E2E нэвтрэх боломж

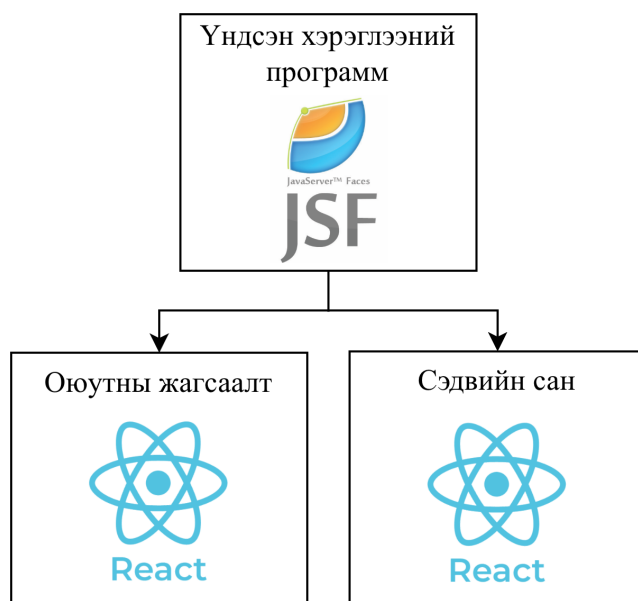
Хүснэгт 2.3-д нэгтгэн харуулсан хэрэгслийн сонголт нь судалгааны хэмжилтийн объектив байдлыг хангах зорилготой. Хэрэгсэл бүр нь нээлттэй эхийнх эсвэл чөлөөтэй ашиглах боломжтой бөгөөд тогтсон нийгэмлэгийн баталгаажсан аргачлалтай тул судалгааны дахин сэргээгдэх байдлыг хангана [25].

3. ТУРШИЛТИЙН ЗАГВАР

Энэ хэсэгт холимог микрофронтед архитектурын хэрэгжүүлэлтийг тайлбарласан. Вэб хөгжүүлэлт нь HTML, CSS, JavaScript ашигладаг үеэс хол зам туулсан тул шаардлагатай хэрэгсэл, нөөц, хамаарал, статик файлууд илүү их эрэлт хэрэгцээтэй болсон. Иймд хялбараар вэб хөгжүүлэх, байршуулахын тулд боловсруулсан шинэ хэрэгслүүд нь орчин үеийн вэб хэрэглээний программ бүрийн чухал хэсэг бөгөөд энэ судалгааны ажилд тусгагдсан шийдлүүд юм.

3.1 Туршилтын загварын бүтэц

Туршилтын загвар нь Оюутны жагсаалт микрофронтед болон Сэдвийн сан микрофронтедээс бүрдэнэ. Микрофронтед бүрийг Үндсэн аппликейшн рүү ачаалдаг. Зураг 3.1-т туршилтын загварын бүтцийг харуулав. Туршилтын загварыг боловсруулахын тулд хийгдсэн алхмуудыг төслийн бүтэц, Webpack тохиргоо, техникийн орчин хүртэл дэлгэрэнгүй тайлбарласан.



Зураг 3.1. Туршилтын загварын бүтэц

Туршилтын загварын бүтэц нь Үндсэн хэрэглээний программ, Микрофронтед модулиуд гэсэн үндсэн бүрэлдэхүүн хэсгүүдээс бүрдэнэ. Үндсэн хэрэглээний программ нь дотроо React хэрэглээний программуудыг ачааллах зориулалттай хоосон DOM элементүүдийг бэлтгэж өгнө. Туршилтын загварыг хөгжүүлэхэд ашигласан технологиудын хувилбарыг

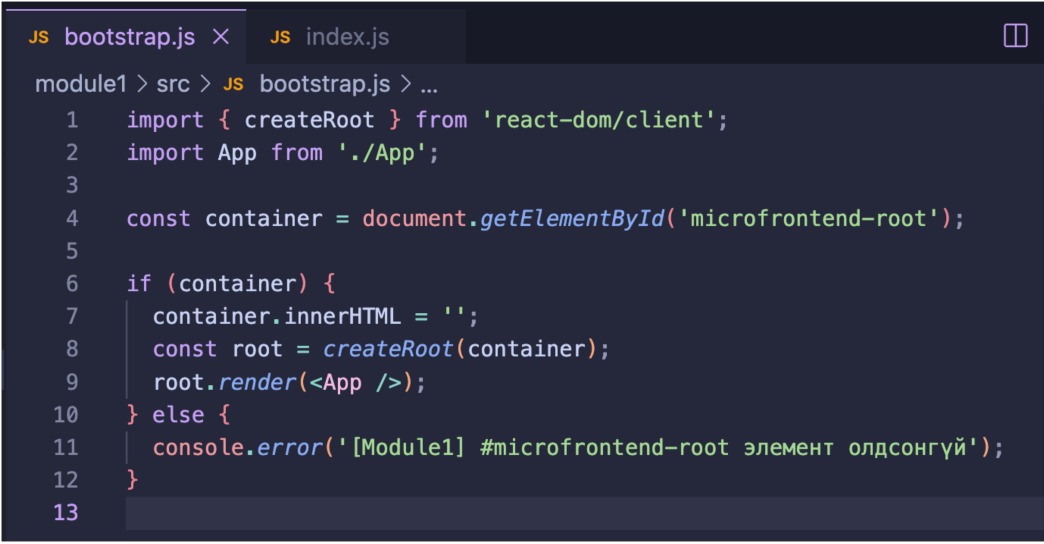
Хүснэгт 3.1-д харуулав.

Хүснэгт 3.1. Туршилтын загварын техникийн орчин

Технологи	Үндсэн хэрэглээний программ	Модуль 1	Модуль 2
Node.js	-	18.18.0	18.18.0
Webpack	-	5.81.0	5.81.0
JSF	2.3	-	-
React	-	18.2.0	18.2.0

3.2 Асинхрон скрипт ачаалалт

Микрофронтендүүдийг ачааллах үед шаардлагатай хамаарал бүрэн татагдаж дуусаагүй байхад хэрэглээний программ уншигдаж эхэлбэл алдаа гарах эрсдэлтэй. Үүнээс сэргийлэхийн тулд асинхрон скрипт ачаалалтыг ашигласан. Хэрэглээний программыг эхлүүлэх логикийг bootstrap.js файл дотор бичиж, index.js нь зөвхөн `import('./bootstrap');` гэсэн динамик дуудлагыг агуулна. Webpack энэ динамик дуудлагыг хөрвүүлэх явцад кодыг хэд хэдэн chunk файл болгон хуваадаг: main.js нь оролтын файл бөгөөд bootstrap.js-г агуулсан chunk болон React vendor chunk-уудыг динамикаар дуудна. Иймд webpack.config.js файл дотор output .publicPath-ийг зөв тохируулах шаардлагатай бөгөөд энэ нь хөтчид chunk файлуудыг зөв замаас татах боломжийг олгоно.

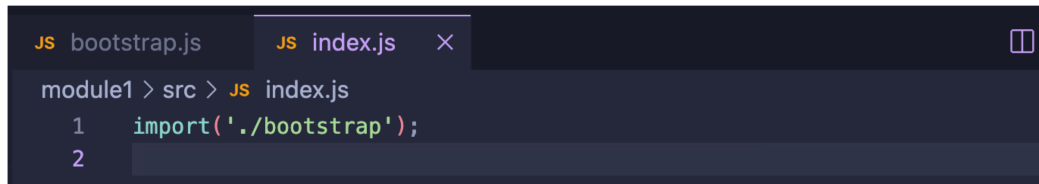


```

JS bootstrap.js × JS index.js
module1 > src > JS bootstrap.js > ...
1  import { createRoot } from 'react-dom/client';
2  import App from './App';
3
4  const container = document.getElementById('microfrontend-root');
5
6  if (container) {
7    container.innerHTML = '';
8    const root = createRoot(container);
9    root.render(<App />);
10 } else {
11   console.error('[Module1] #microfrontend-root элемент олдсонгүй');
12 }
13

```

Зураг 3.2. Асинхрон скрипт ачаалалтын эхлүүлэх логик



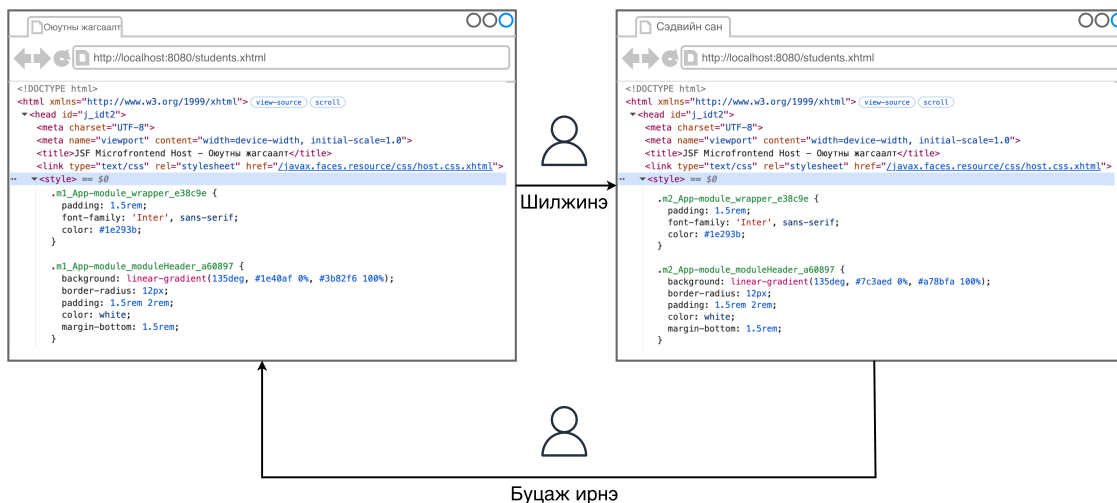
Зураг 3.3. Асинхрон скрипт ачаалалтын динамик дуудлага

3.3 CSS загварын давхцал

Хэрэглэгч нэг микрофронтендийн хуудаснаас нөгөө рүү шилжих үед хөтөч нь тухайн модулиудын CSS загваруудыг DOM-ийн <head> хэсэгт нэмж ачаалдаг бөгөөд ижил нэртэй CSS классууд нэгнийгээ дарж бичин, UI-ийн хэв гажилт үүсгэдэг байна. Энэ асуудлаас сэргийлэхийн тулд туршилтын загварт CSS Modules аргачлалыг ашигласан. Модуль бүрийн загварын файлыг .module.css өргөтгөлтэйгээр үүсгэж, webpack.config.js дотор css-loader-ын modules тохиргоог идэвхжүүлсэн. Webpack хөрвүүлэлтийн явцад CSS класс бүрийн нэрэнд [name]_[local]_[hash:6] загварын дагуу давтагдахгүй хэш утга залгадаг тул JSF үндсэн хэрэглээний программ болон бусад React модулиудын загвартай мөргөлдөхөөс бүрэн хамгаалагддаг.

Хэдийгээр CSS Modules нь React компонентүүдийн класс нэрийг дахин давтагдашгүй болгож, тэднийг бусад модуль болон JSF хэсэгтэй мөргөлдөхөөс бүрэн хамгаалдаг боловч нэгэн техникийн хязгаарлалт байдгийг дурдах нь зүйтэй. JSF хост хэрэглээний программын талд тодорхойлогдсон глобал загварууд нь React компонентүүд рүү доошоо уламжлагдан орж ирж нөлөөлөх эрсдэл бий. Учир нь JSF болон React нь хөтөч дээр нэг л DOM модонд хамтдаа байрладаг.

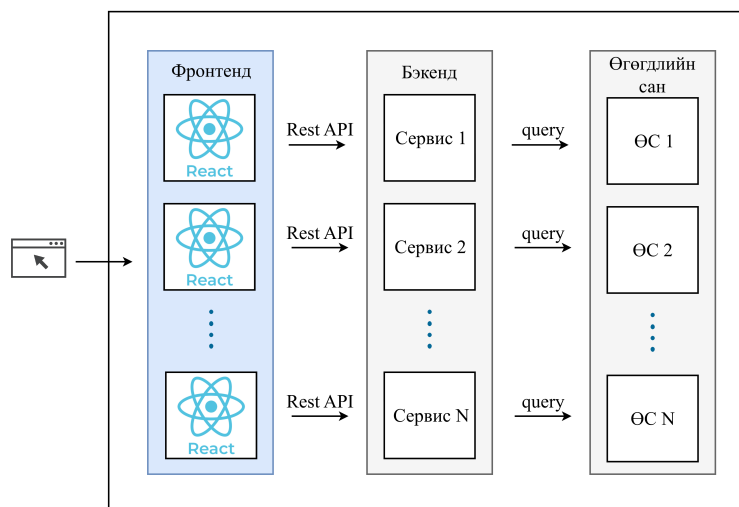
Энэ глобал нөлөөллөөс сэргийлж тусгаарлахын тулд Web Components стандартын нэг хэсэг болох *Shadow DOM* ашиглах хувилбарыг онолын хувьд давхар судалсан. Гэвч Shadow DOM-ийг React-тэй хоршуулан ашиглах үед Webpack-ийн style-loader-ийн тохиргоог хэт хүндрүүлдэг төдийгүй, дэлгэцийн хүрээнээс гадуур дүрслэгддэг React Portals (жишээ нь Modal, Dialog, Tooltip) суурьтай 3-дагч UI компонентүүдийн загвар алдагдаж эвдрэх өндөр магадлалтай байдаг. Тиймээс хост болон микрофронтендийн кодыг нэг дор удирдан хөгжүүлж буй энэхүү туршилтын нөхцөлд CSS Modules аргачлал нь архитектурын хувьд хавьгүй оновчтой бөгөөд хөнгөн шийдэл болно гэж дүгнэн сонгосон болно.



Зураг 3.4. CSS Modules-ийн хэш нэмэгдсэн класс нэрүүд (Browser DevTools)

3.3.1 Одоо байгаа шийдэл

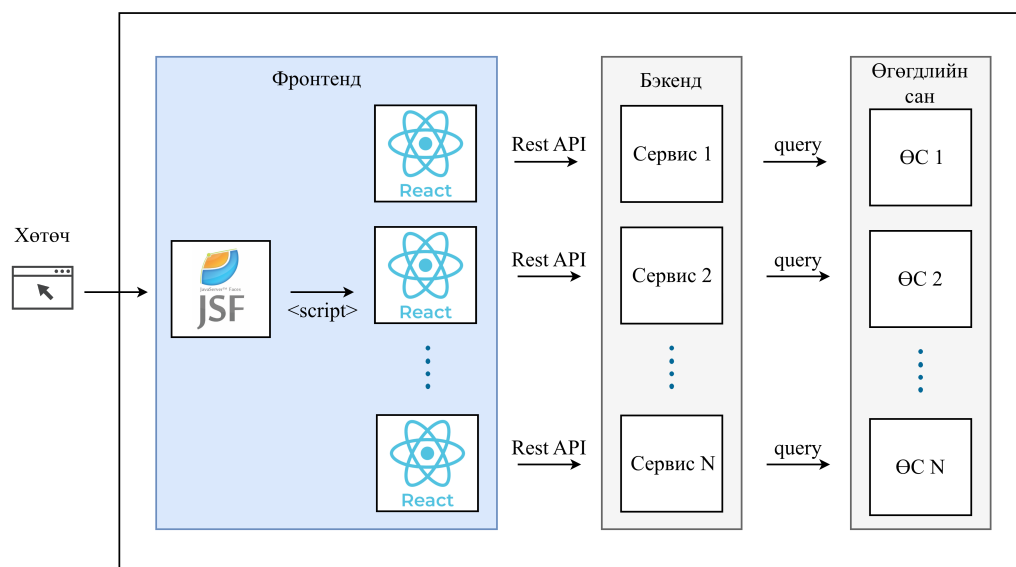
Орчин үеийн цэвэр микрофронтед архитектурын нийтлэг шийдлийг Зураг 3.5-т харуулав. Энэхүү бүтэц нь дан ганц React гэх мэт орчин үеийн фреймворк дээр суурилсан үндсэн хэрэглээний программ болон бие даасан микрофронтед модулиудаас бүрддэг. Модуль тус бүр нь бэкенд дэх өөрийн харгалзах микросервисээр дамжуулан тусгаарлагдсан өгөгдлийн сантай харилцдаг. Энэ нь цоо шинээр хөгжүүлэгдэж буй системүүдэд маш тохиромжтой хэдий ч, олон жил ашиглагдсан, бизнесийн цогц логик бүхий JSF суурьтай цул системийг тэр чигт нь энэхүү архитектур руу шууд шилжүүлэхэд хөгжүүлэлтийн цаг хугацаа болон нөөцийн хувьд хэт өндөр зардалтай юм.



Зураг 3.5. Одоо байгаа микрофронтенд архитектурын шийдэл

3.3.2 Санал болгож буй шийдэл

Уламжлалт системийг шинэчлэхэд үүсдэг дээрх хүндрэлийг шийдвэрлэхийн тулд Зураг 3.6-т үзүүлсэн холимог микрофронтенд архитектурыг энэхүү судалгаагаар санал болгож байна. Энэхүү шийдэл нь хуучин JSF хэрэглээний программыг бүрэн устгахын оронд, түүнийг хэрэглэгчийн интерфэйсийн эх бие буюу хост болгон ашиглах зарчимд тулгуурлана. Шинээр хөгжүүлэх шаардлагатай модулиудыг React фреймворкоор бие даан хөгжүүлж, JSF-ийн хуудсанд `<script>` тагаар динамикаар нэгтгэнэ. Ингэснээр хуучин системийн тогтвортой ажиллагааг хадгалахын зэрэгцээ, орчин үеийн технологийн давуу талыг ашиглан системийг үе шаттайгаар, эрсдэл багатайгаар шинэчлэх боломж бүрдэх юм.



Зураг 3.6. Санал болгож буй холимог микрофронтенд архитектурын шийдэл

4. СИСТЕМИЙН ШИНЖИЛГЭЭ

Энэ бүлэгт Дидломын ажлын удирдлагын системийн шинжилгээг хийж, системийн алсын хараа, суурь нөхцөл, бизнес процесс, динамик болон статик загварыг тодорхойлсон.

4.1 Системийн алсын хараа

МУИС-ын дипломын ажлын удирдлагын систем нь их, дээд сургуулийн оюутан, удирдагч багш, сургалтын албанд зориулагдсан дипломын ажлын автоматжуулалтын нэгдсэн систем юм. Энэхүү систем нь дипломын сэдэв сонголт, баталгаажуулалт, удирдлага, үнэлгээ болон гүйцэтгэлийн бүх үйл явцыг ил тод, төвлөрсөн байдлаар удирдах боломжийг олгоно. Уламжлалт цаасан болон салангид системд тулгуурласан гар ажиллагаатай арга барилаас ялгаатай нь, уг систем нь цаг хугацаа хэмнэх, процессыг хялбаршуулах, бүх оролцогч талуудын ажлын уялдаа холбоог сайжруулах цогц шийдлийг санал болгож байна.

Хүснэгт 4.1. Системийн алсын харааны тодорхойлолт

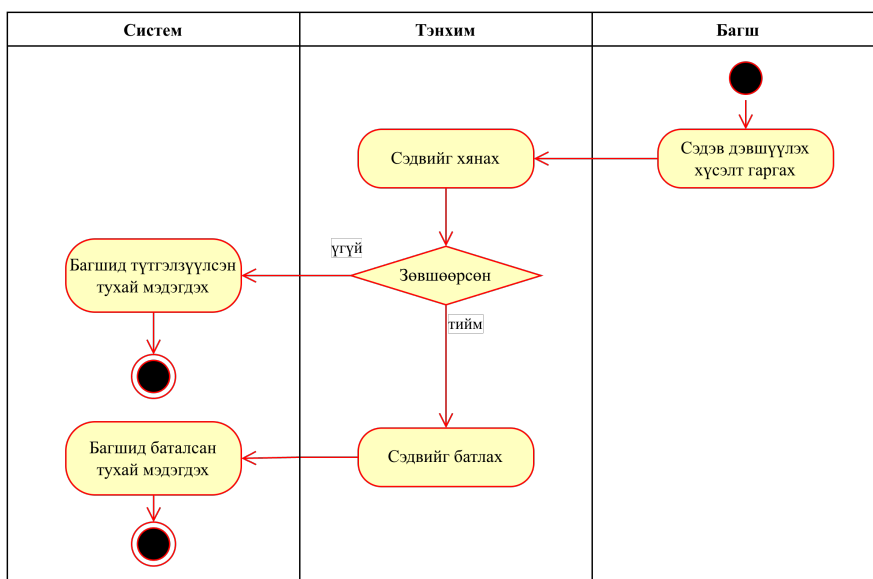
Бүтэц	Тодорхойлолт
FOR (Хэнд)	Их, дээд сургуулийн оюутан, удирдагч багш, сургалтын албаны ажилтнуудад
WHO (Ямар хэрэгцээтэй)	Дипломын ажлын сэдэв сонголт, баталгаажуулалт, удирдлага, үнэлгээ болон явцын хяналтыг илүү үр ашигтай, нэгдсэн байдлаар хэрэгжүүлэх шаардлагатай байгаа хэрэглэгчдэд
THE (Юу)	thesis.num.edu.mn нь дипломын ажлын автоматжуулалтын нэгдсэн вэб систем бөгөөд
THAT (Ямар шийдэлтэй)	Дипломын ажлын бүхий л процессыг (сэдэв дэвшүүлэх, сонгох, батлах, удирдах, үнэлэх, гүйцэтгэлийг хянах) нэг платформоор удирдах боломж олгож, мэдээллийн урсгалыг ил тод болгож, цаг хугацаа болон гар ажиллагааг эрс багасгана
UNLIKE (Юугаар ялгаатай)	Уламжлалт цаасан суурьтай болон салангид, гар ажиллагаанд тулгуурласан системүүдээс ялгаатай нь
OUR PRODUCT (Бидний бүтээгдэхүүн)	Дипломын ажлын бүх үе шатыг бүрэн автоматжуулж, төвлөрсөн удирдлага, хяналт, тайлагнал бүхий өргөтгөх боломжтой дижитал орчныг бүрдүүлнэ

4.2 Функциональ загвар

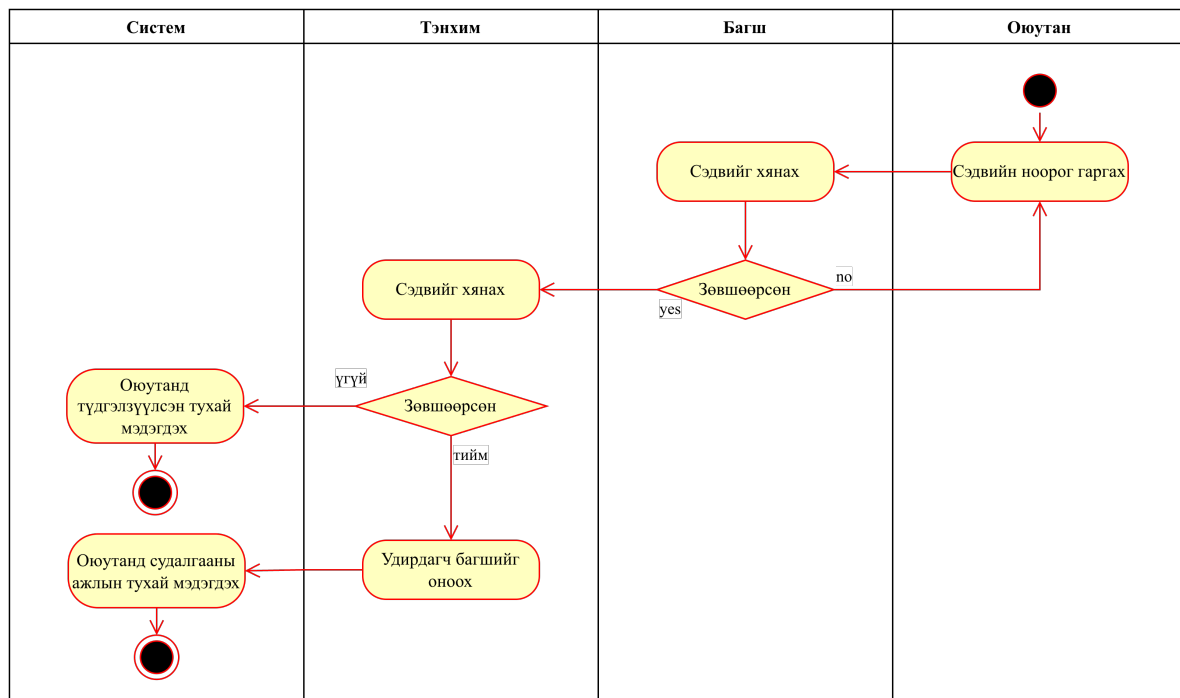
МУИС-ийн Дипломын ажлын удирдлагын системд дараах үндсэн хэрэглэгчид оролцоно.
Үүнд:

- Оюутан
- Багш
- Тэнхим (Сургалтын алба / Тэнхимийн туслах ажилтан)
- Гадны мэргэжилтэн

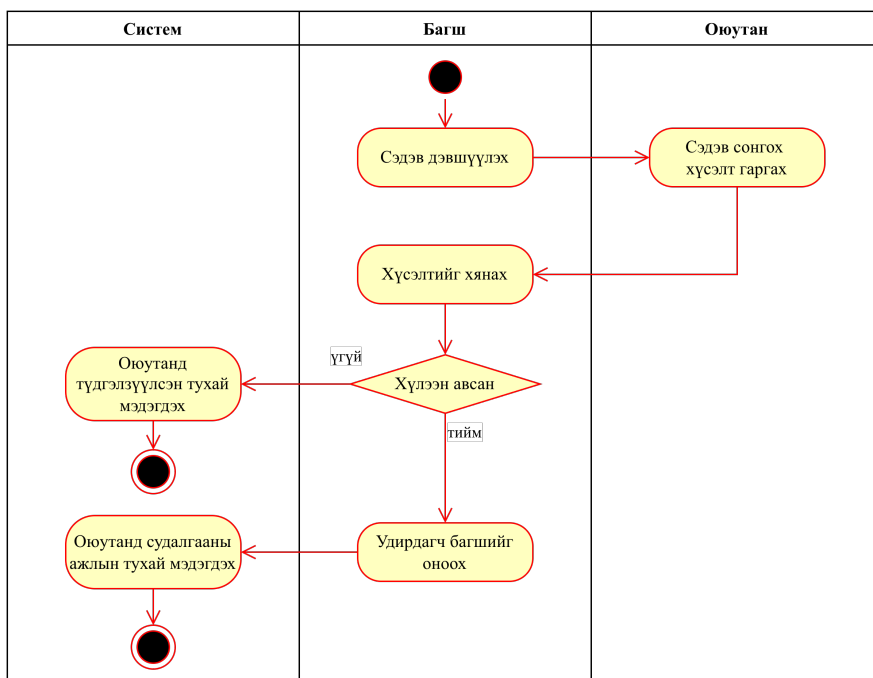
Эдгээр оролцогчдын харилцан үйлдэл болон дипломын ажлын үе шатуудад тулгуурлан МУИС-ийн Дипломын ажлын удирдлагын системийн үндсэн бизнес процессуудыг тодорхойлсон. Зураг 4.1-т сэдэв дэвшүүлэх процесс, зураг 4.2-т оюутан сэдэв дэвшүүлэх процесс, зураг 4.3-т БСА-ын сонгох процесс, зураг 4.4-т БСА-ын үечилсэн төлөвлөгөөг тэнхимээр батлуулах процесс, 4.5-т БСА гүйцэтгэх, тайлан хураалгах процесс, 4.6-т БСА-ыг хамгаалах, дүгнэх процессыг харуулав.



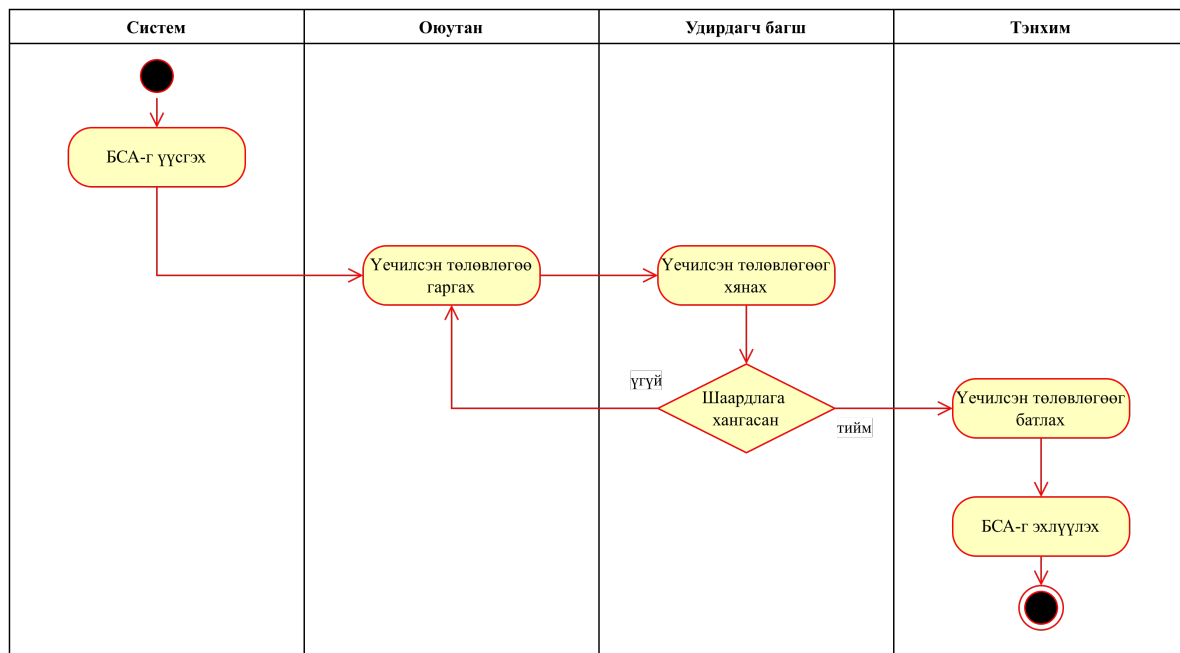
Зураг 4.1. БСА-ын сэдэв дэвшүүлэх процесс



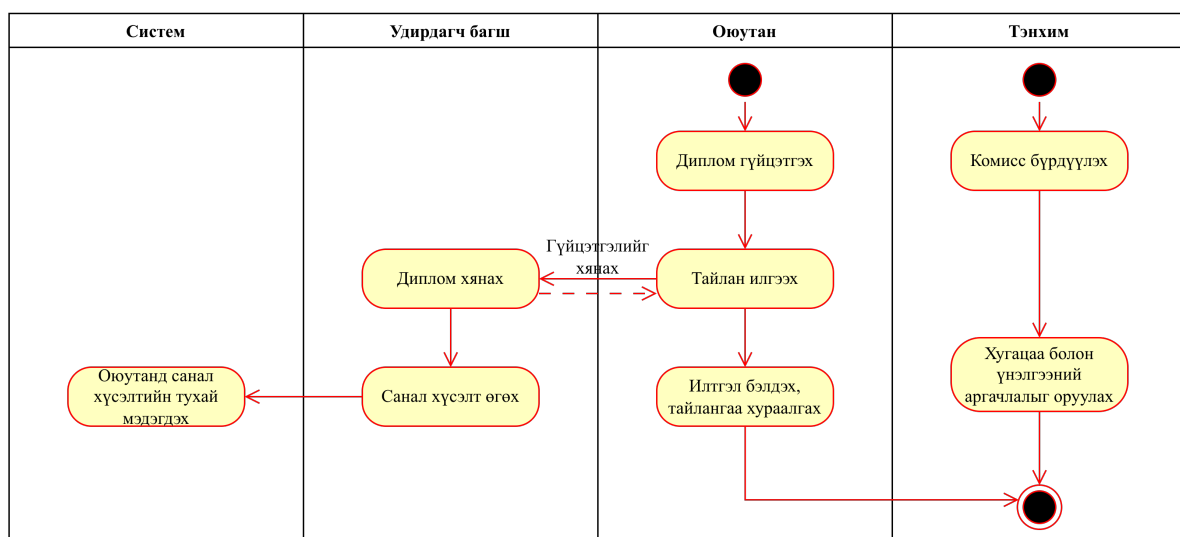
Зураг 4.2. БСА-ын сэдэв дэвшүүлэх процессын өргөтгөл



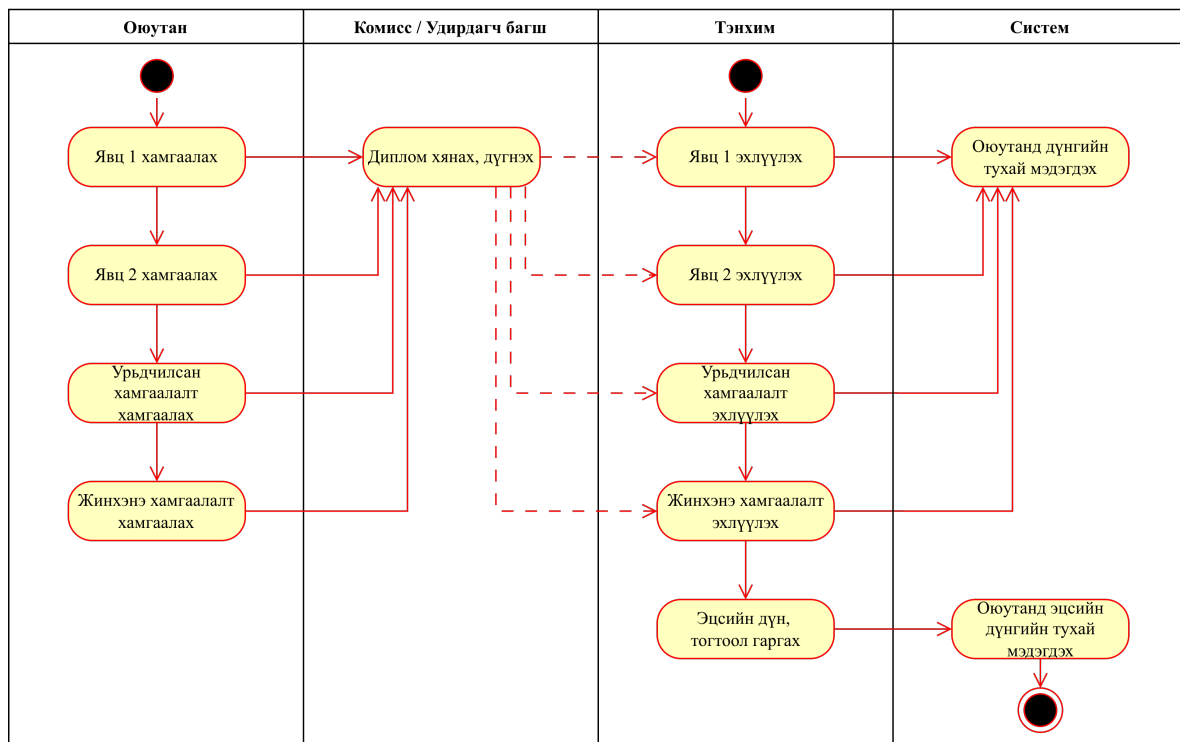
Зураг 4.3. БСА-ын сэдэв сонгох процесс



Зураг 4.4. БСА-ын үчилсэн төлөвлөгөөг тэнхимээр батлуулах процесс



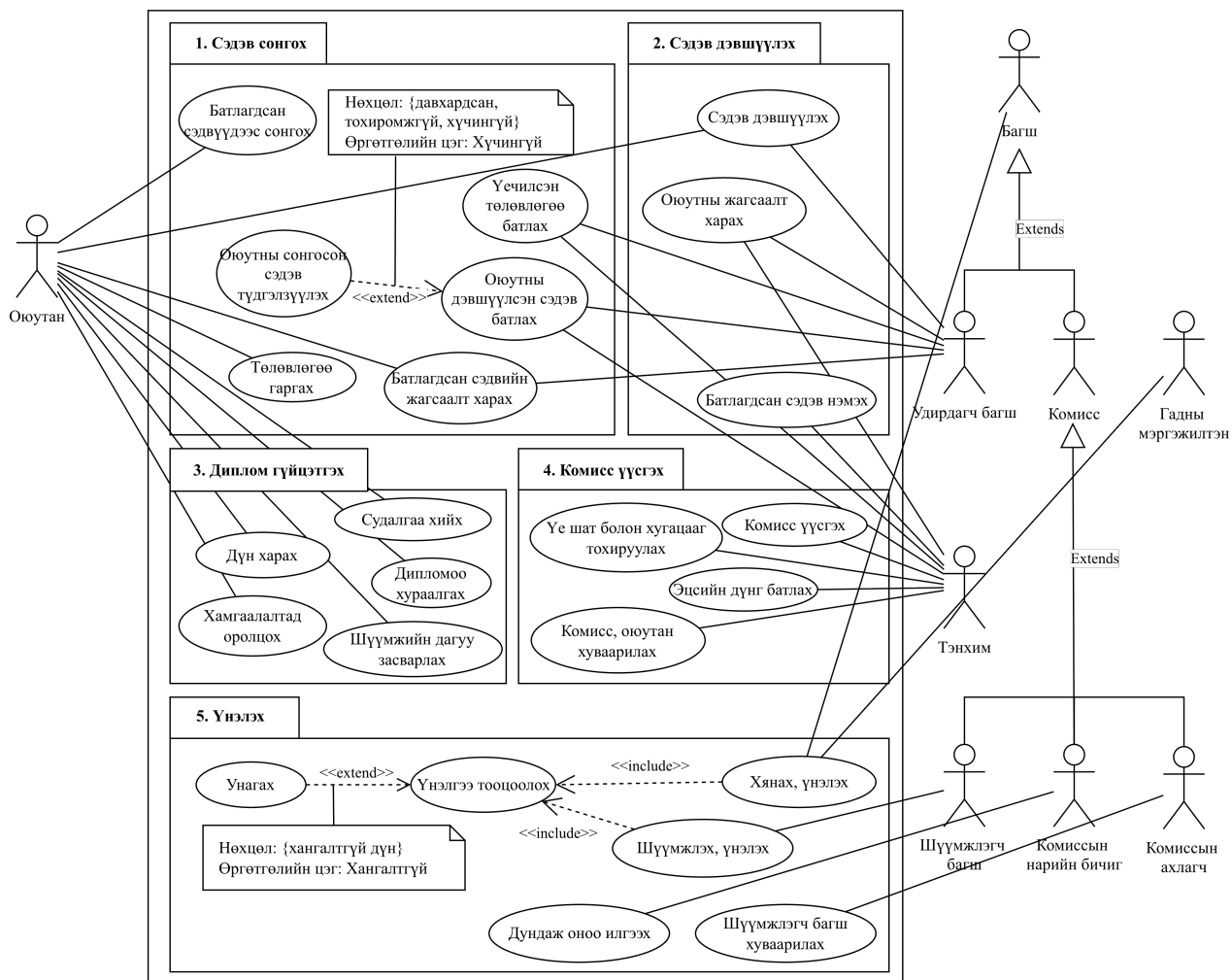
Зураг 4.5. БСА гүйцэтгэх, тайлан хураалгах процесс



Зураг 4.6. БСА-ыг хамгаалах, дүгнэх процесс

4.3 Динамик загвар

МУИС-ийн Дипломын ажлын удирдлагын системийн динамик загварыг зураг 4.7-т харуулав. Динамик загварт оюутан, багш, тэнхим, гадны мэргэжилтэн гэсэн тоглогчид оролцох бөгөөд тоглогч тус бүрийн ажлын явцын задаргааг дүрслэв. Хүснэгт 4.2-т тоглогч тус бүрийн ажлын явцын тодорхой тайлбарыг өгсөн.



Зураг 4.7. Дипломын ажлын удирдлагын системийн динамик загвар

Хүснэгт 4.2. Дипломын ажлын удирдлагын системийн ажлын явцын тайлбар

Ажлын явц	Тоглогч	Тайлбар
Батлагдсан сэдвүүдээс сонгох	Оюутан	Тэнхимээс баталж, санал болгосон дипломын ажлын сэдвүүдээс өөрийн сонирхол, чиглэлд нийцүүлэн сонголт хийх.
Батлагдсан сэдвийн жагсаалт харах	Оюутан	Системд нийтлэгдсэн, сонгох боломжтой дипломын ажлын сэдвүүдийн дэлгэрэнгүй жагсаалт болон шаардлагуудтай танилцах.

Ажлын явц	Тоглогч	Тайлбар (үргэлжлэл)
Төлөвлөгөө гаргах	Оюутан	Дипломын ажил гүйцэтгэх үе шат, цаг хугацааны хуваарийг нарийвчлан гаргаж системд оруулах.
Сэдэв дэвшүүлэх	Оюутан	Өөрийн санаачилсан шинэ дипломын ажлын сэдвийг удирдагч багш болон тэнхимээр батлуулахаар санал болгож илгээх.
Судалгаа хийх	Оюутан	Сонгосон сэдвийн хүрээнд хийж буй онолын болон практик судалгааны ажлын явц, үр дүнг системд бүртгэж тайлагнах.
Дипломоо хураалгах	Оюутан	Эцсийн байдлаар боловсруулсан дипломын ажлын тайлан болон холбогдох файлуудыг системд байршуулж хүлээлгэн өгөх.
Хамгаалалтад оролцох	Оюутан	Товлосон хугацаанд дипломын ажлын урьдчилсан болон жинхэнэ хамгаалалтад оролцоход шаардлагатай мэдээллийг системээс авах, батлах.
Шүүмжийн дагуу засварлах	Оюутан	Удирдагч болон шүүмжлэгч багш нараас системээр дамжуулан өгсөн санал, шүүмжийн дагуу ажилдаа засвар оруулж дахин илгээх.
Дүн харах	Оюутан	Дипломын ажлын үнэлгээ, комиссын нэгдсэн дүн болон эцсийн шийдвэртэй танилцах.
Оюутны жагсаалт харах	Тэнхим	Тухайн хичээлийн жилд дипломын ажил бичиж буй нийт оюутнуудын нэгдсэн жагсаалт, тэдний ажлын явцтай танилцах.
Батлагдсан сэдэв нэмэх	Тэнхим	Оюутнууд сонгох боломжтой дипломын ажлын сэдвүүдийн санг шинэчлэх, шинээр сэдэв нэмж оруулах.
Үечилсэн төлөвлөгөө батлах	Тэнхим	Оюутнуудын гаргасан дипломын ажил гүйцэтгэх төлөвлөгөө, цаглабарыг хянаж, баталгаажуулах.

Ажлын явц	Тоглогч	Тайлбар (үргэлжлэл)
Оюутны дэвшүүлсэн сэдэв батлах	Тэнхим	Оюутнаас бие даан дэвшүүлсэн сэдвийг судлан үзэж, шаардлага хангасан тохиолдолд албан ёсоор батлах.
Комисс үүсгэх	Тэнхим	Дипломын ажил хамгаалуулах зөвлөл буюу комиссын бүрэлдэхүүнийг томилж, системд бүртгэх.
Үе шат болон хугацааг тохируулах	Тэнхим	Дипломын ажлын үйл явцын (сэдэв сонгох, хураалгах, хамгаалах) ерөнхий хуваарь, системийн хугацааг тохируулах.
Эцсийн дүнг батлах	Тэнхим	Хамгаалалтын комиссоос гарсан оюутны нэгдсэн дүнг системд эцсийн байдлаар баталгаажуулах.
Комисс, оюутан хуваарилах	Тэнхим	Хамгаалалтад орох оюутнуудыг холбогдох комисст болон хамгаалах цагийн хуваарьт хуваарилж батлах.
Сэдэв дэвшүүлэх	Удирдагч багш	Өөрийн удирдах боломжтой, оюутнуудад санал болгох дипломын ажлын сэдвийг системд бүртгүүлж батлуулах.
Оюутны жагсаалт харах	Удирдагч багш	Өөрийн удирдаж буй оюутнуудын жагсаалт, тэдний өнөөгийн явцтай системээс танилцах.
Үечилсэн төлөвлөгөө батлах	Удирдагч багш	Өөрийн удирдаж буй оюутны гаргасан нарийвчилсан ажлын төлөвлөгөөг хянан баталгаажуулах.
Оюутны дэвшүүлсэн сэдэв батлах	Удирдагч багш	Оюутны санал болгосон сэдвийг удирдах боломжтой эсэхээ шийдвэрлэж, батлах процессийг эхлүүлэх.
Батлагдсан сэдвийн жагсаалт харах	Удирдагч багш	Тэнхим болон бусад багш нарын санал болгосон, системд нийтлэгдсэн нийт сэдвүүдтэй танилцах.
Хянах, үнэлэх	Удирдагч багш	Оюутны ажлын явцыг тогтмол хянаж, үе шат бүрд санал хүсэлт өгөх, явцын болон эцсийн үнэлгээ хийх.

Ажлын явц	Тоглогч	Тайлбар (үргэлжлэл)
Шүүмжлэх, үнэлэх	Шүүмжлэгч багш	Оюутны хураалгасан дипломын ажилтай танилцаж, мэргэжлийн шүүмж бичиж, оноо бүхий үнэлгээг системд оруулах.
Хянах, үнэлэх	Шүүмжлэгч багш	Хамгаалалтын өмнө дипломын ажлын чанар, тавигдах шаардлагыг хянаж, урьдчилсан санал дүгнэлт гаргах.
Судалгаа хийх	Комиссын нарийн бичиг	Хамгаалалтын үйл явц, оюутнуудын ирц болон бичиг баримтын бүрдүүлэлтэд хяналт тавьж, шаардлагатай өгөгдлийг нэгтгэх.
Дундаж оноо илгээх	Комиссын нарийн бичиг	Комиссын гишүүдийн өгсөн үнэлгээнүүдийг нэгтгэн, оюутны дундаж оноог тооцоолж системд оруулах.
Шүүмжлэгч багш хуваарилах	Комиссын ахлагч	Хамгаалалтад орох оюутан бүрийн сэдэвт тохирсон мэргэжлийн шүүмжлэгч багшийг томилж, системд хуваарилах.
Хянах, үнэлэх	Комиссын ахлагч	Хамгаалалтын үйл явцыг удирдаж, оюутны илтгэл, мэдлэгт хяналт тавьж, комиссын гишүүний хувиар үнэлгээ өгөх.
Хянах, үнэлэх	Гадны мэргэжилтэн	Салбарын хөндлөнгийн шинжээчийн хувиар дипломын ажлын практик ач холбогдлыг хянаж, хараат бус үнэлгээ өгөх.

4.4 Бүлгийн дүгнэлт

Энэ бүлэгт МУИС-ийн Дипломын ажлын удирдлагын системийн алсын хараа, функциональ загвар, динамик загварыг тодорхойлсон. Системийн алсын хараа нь хэрэглэгчдийн хэрэгцээ, шийдэл болон бүтээгдэхүүний онцлогийг тодорхойлж өгсөн бол функциональ загвар нь системийн гол бизнес процессуудыг дүрслэн харуулсан. Динамик загвар нь оролцогчдын харилцан үйлдэл болон ажлын явцыг задалж өгсөн. Эдгээр загварууд нь дараагийн бүлэгт авч үзэх архитектурын загварчлал болон практик хэрэгжүүлэлтэд суурь болох юм.

5. СИСТЕМИЙН ЗОХИОМЖ

Энэ бүлэгт системийн гурван үндсэн давхарга (Фронтенд давхарга, Бэкэнд давхарга, Өгөгдлийн давхарга) тус бүрийн загварчлал, хил хязгаар, нэгдсэн зохиомжийн шийдвэрийг дэлгэрэнгүй авч үзнэ.

5.1 Архитектурын үндэслэл

Энэхүү судалгааны ажлын хүрээнд МУИС-ийн дипломын ажлын удирдлагын системийг холимог технологийн микрофронтенд архитектураар шийдвэрлэх туршилтын загварын дагуу боловсруулсан. Судалгааны гол зорилго нь уламжлалт Java Enterprise экосистемийн төлөөлөл болох Jakarta Server Faces 2.2 (JSF) суурьтай монолит орчныг, орчин үеийн React суурьтай микрофронтендүүдтэй хэрхэн оновчтой хослуулж, технологийн шилжилтийг эрсдэл багатай гүйцэтгэж болохыг архитектурын түвшинд загварчлан харуулахад оршино.

5.1.1 Тулгамдсан асуудал

Системийн хост орчноор JSF-ийг сонгосон нь Enterprise түвшний байгууллагуудын системүүдэд түгээмэл тохиолддог архитектурын сорилтуудыг бодит орчинд турших зорилготой. JSF суурьтай монолит бүтэц нь дараах онцлог болон хязгаарлалтуудыг агуулдаг тул микрофронтенд шилжилтийг турших хамгийн тохиромжтой кейс болсон:

1. Багасгах болон томруулах чадварын ялгаатай байдал: JVM heap дээрх HttpSession-д суурилсан JSF-ийн төлөв хадгалалт нь санах ойн өндөр хэрэглээ шаарддаг бол микрофронтенд хэсэг нь клиент талд суурилсан, stateless шинж чанартай байх боломжийг олгоно.
2. Технологийн ялгаатай байдлыг нэгтгэх (Heterogeneity): JSF-ийн сервер талын Request-Response мөчлөгийг, React-ийн Single Page Application (SPA) шинж чанартай хэрхэн зөрчилгүйгээр уялдаатай ажиллуулах нь архитектурын гол сорилт байв.
3. Хөгжүүлэлтийн бие даасан байдал: Хост системийг хөндөхгүйгээр, шинээр нэмэгдэж буй модулиудыг орчин үеийн TypeScript, React экосистем дээр бие даан хөгжүүлж, нэгтгэх боломжийг судлах шаардлагатай байв.

5.1.2 Архитектурын стратегийн сонголт: Аажмын шилжилтийн хандлага

Холимог архитектурыг хэрэгжүүлэхийн тулд Strangler Fig [18] загварыг баримталсан. Энэхүү загвар нь JSF суурьтай моноклит хост системийг хэвээр хадгалж, шинэ модулиудыг React микрофронтенд хэлбэрээр түүний дотор аажмаар суулгах замаар нэгдмэл системийг бүрдүүлдэг.

Энэхүү хандлагыг сонгосон үндэслэлүүд:

- Холимог орчныг дэмжих: JSF болон React технологиудыг нэг хэрэглэгчийн интерфейс дотор зөрчилгүйгээр ажиллуулах бодит туршилт болох.
- Бизнесийн тасралтгүй байдал: Системийн ажиллагааг зогсоолгүйгээр, модуль бүрээр нь үе шаттайгаар шилжүүлэх боломжийг хангах.

Хүснэгт 5.1. Архитектурын стратегийн харьцуулалт (Туршилтын загвараар)

Шалгуур	Big Bang	In-place	Strangler Fig
Технологийн холимог байдал	Боломжгүй	Хязгаарлагдмал	Бүрэн боломжтой
Бизнесийн эрсдэл	Өндөр	Дунд	Бага
Командын бие даасан байдал	Бага	Бага	Өндөр
Модульчлагдсан байдал	Дунд	Бага	Маш өндөр
Буцаалтын (Rollback) боломж	Байхгүй	Хэцүү	Бүрэн

5.2 Микрофронтенд архитектур

Microfrontend (MFE) архитектур нь бэкенд микросервисийн зарчмуудыг фронтенд давхаргад хэрэглэх ойлголт бөгөөд Sam Jackson (2019)-ийн тодорхойлсноор SPA-г бие даасан, тусдаа байршуулагдах жижиг фронтенд аппликейшнүүдийн нэгдэл гэж тодорхойлогдоно.

Энэхүү системд JSF Tomcat нь нэгдсэн хостыг үүрэг гүйцэтгэж, React дээр суурилсан бие даасан модулиуд нь нийлүүлэгч MFE хэлбэрт ажиллана. Ажиллах үеийн интеграцийн стратегийг ашигласан нь интеграцийн цагт биш, харин хөтчид ачааллагдах үед модулиуд динамик байдлаар холбогдоно гэсэн утгатай бөгөөд энэ нь Дэлхийн тэргүүлэх MFE архитектурын загвар юм [37].

5.2.1 *Strangler Fig* үлгэр загварын хэрэгжүүлэлт

Strangler Fig үлгэр загвар [?] нь Австралийн чулуун инжир модны органик ургалтын хэлбэрээс санаа авсан архитектурын хандлага бөгөөд энэ нь хуучин системийг зогсоолгүйгээр, шинэ бүрэлдэхүүн хэсгүүдийг аажмаар нэгтгэн орлуулах боломжийг олгодог. Энэхүү судалгааны ажлын хүрээнд JSF суурьтай монолит бүтцийг "Legacy Proxy" буюу уламжлалт системийн загвар болгон ашиглаж, орчин үеийн React MFE-тэй хэрхэн зэрэгцэн ажиллах процессыг дараах үе шаттайгаар загварчлав:

1. Identify (Жижиглэн хуваалт): Монолит архитектураас бие даасан байдлаар тусгаарлаж болох бизнесийн логик болон домейны хил хязгаарыг тодорхойлох шат юм. Дипломын ажлын удирдлагын системийн хүрээнд нэвтрэлт, оюутан, багш болон захиргааны хэсгүүдийг тусдаа контекст болгон тодорхойлсон.
2. Strangle (Интеграци): Сонгосон функциональ хэсгүүдийг React MFE хэлбэрт хөгжүүлж, JSF хост доторх тусгай цэгүүдэд (Injection points) суулгасан. Энэ нь технологийн ялгаатай хоёр орчин нэг хэрэглэгчийн интерфэйс дотор зөрчилгүй ажиллах холимог үе шат юм.
3. Transform/Transition (Шилжилт): Бүх модулиудыг MFE хэлбэрт шилжүүлснээр систем бүрэн микрофронтенд архитектурт шилжих боломж бүрдэнэ.

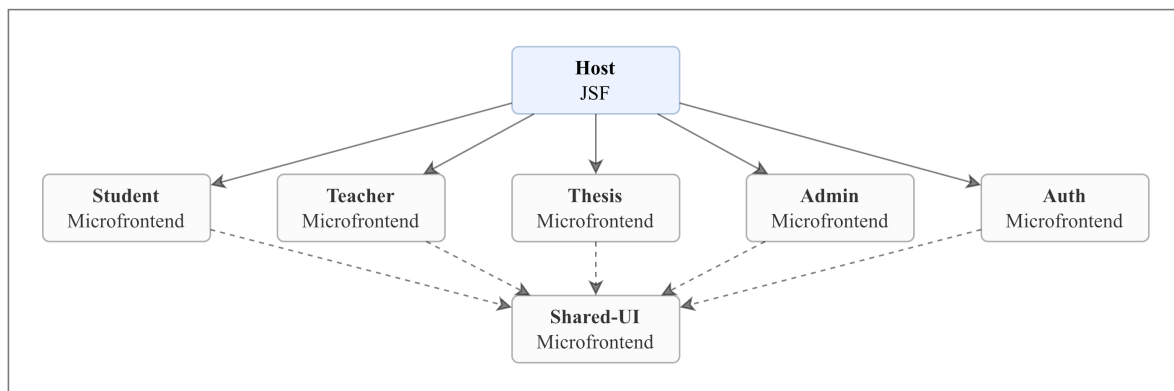
Судалгааны хязгаарлалт ба хамрах хүрээ: Энэхүү судалгаа нь Strangler Fig үлгэр загварын **Strangle** буюу холимог орчинд технологиудыг нэгтгэн ажиллуулах үе шатыг голлон анхаарсан. JSF хостыг бүрэн устгаж, 100% бие даасан SPA (Single Page Application) болгон хувиргах эцсийн шатыг судалгааны хамрах хүрээнээс гадуур буюу ирээдүйд хийгдэх ажлын хэсэгт үлдээв. Ингэснээр системд эрсдэлгүй шилжилт хийх боломжтойг архитектурын хувьд баталсан болно.

5.2.2 *MFE модулиудын зохион байгуулалт*

Системийн архитектурыг Eric Evans [15]-ийн тодорхойлсон Хязгаарлагдсан контекст (Bounded Context) зарчмын дагуу хуваасан. Ингэснээр модуль бүр өөрийн домейны хил хязгаар, өгөгдлийн загвар болон бизнесийн хариуцлагыг бие даан хариуцах боломж бүрдсэн. Туршилтын хүрээнд дараах долоон бие даасан MFE модулийг загварчлан хэрэгжүүлэв.

Хүснэгт 5.2. MFE модулиудын зохион байгуулалт

MFE модуль	Архитектурын үүрэг ба хариуцлага
host-jsf	Микрофронтендүүдийг нэгдсэн нэг интерфейст нэгтгэх үндсэн бүрхүүл үүргийг гүйцэтгэнэ. Үүнд: - Глобал чиглүүлэлт болон нэвтрэх эрхийн төвлөрсөн хяналт; - Системийн нэгдсэн бүтэц болон навигацийн удирдлага; - Бие даасан MFE модулиудыг хэрэгцээт цагт нь динамикаар дуудаж ажиллуулах интеграцлал.
module-auth	Хэрэглэгчийг таних, баталгаажуулах болон системийн сессийг эхлүүлэх урсгалыг хариуцна. Аюулгүй байдлын үүднээс бусад модулиудаас хамааралгүй, тусгаарлагдсан контекстэд ажиллахаар загварчлагдсан.
module-student	Оюутанд зориулсан бизнесийн бүх логикийг агуулна. Үүнд сэдэв сонгохоос эхлээд эцсийн хамгаалалт хүртэлх амьдралын мөчлөгийг удирдах бөгөөд бодит цагийн мэдэгдлийн сервистэй уялдаж ажиллана.
module-teacher	Багшийн зүгээс хийгдэх хяналт, үнэлгээний үйлдлүүдийг хариуцна. Оюутны явцыг удирдах, зөвлөгөө өгөх болон өөрийн удирдаж буй ажлуудын нэгдсэн хяналтын самбарыг ажиллуулна.
module-admin	Системийн суурь тохиргоо, тэнхимийн бүтэц, хэрэглэгчдийн эрх болон комисс хуваарилах зэрэг системийн түвшний зохицуулалтыг хариуцна.
shared-ui-module	Системийн харагдац, дизайны нэгдмэл байдлыг хангах зорилготой. Бүх MFE-д ашиглагдах UI бүрэлдэхүүнүүдийг нэгтгэн хадгалж, системийн гүйцэтгэлийг оновчтой болгох үүднээс дундын нөөцийг хуваалцах (Shared resource) бодлогоор ажиллана.



Зураг 5.1. Микрофронтендүүд хоорондын харилцаа

5.2.3 Ажиллах үеийн интеграци

MFE интеграцийн арга нь гурван ангилалд хуваагдана: Build-time Integration (угсралтын үеийн), Server-side Integration (сервер талын) болон Runtime Integration (ажиллах үеийн). Тус системд Ажиллах үеийн интеграци сонгосны шалтгаан нь:

- Угсралтын үеийн интеграци нь бие даасан байршуулалтыг зөвшөөрдөггүй, нэг модуль өөрчлөгдөх бүрт бүгдийг дахин build хийх шаардлагатай болно.
- Сервер талын интеграци нь JSF-ийн request-response мөчлөгтэй зөрчилдөж, нарийн тохиргоо шаардана.
- Ажиллах үеийн интеграци нь Vite Module Federation хэрэгслийн тусламжтайгаар хөтчид `remoteEntry.js`-г динамикаар татаж, зөвхөн шаардлагатай модулийг ачаалах “lazy loading” зарчмаар ажиллах боломжийг олгодог.

Ажиллах үеийн интеграцийн загварт `remoteEntry.js` нь Remote MFE-ийн манифест үүрэг гүйцэтгэж, Host нь энэ файлыг ачаалснаар тухайн MFE-ийн экспортлосон бүрэлдэхүүнүүдийн мэдрэх зам (resolution path)-ийг олж авдаг.

5.3 Микросервис Архитектур

Бэкенд системийн архитектур нь Sam Newman [42]-ийн тодорхойлсон микросервисийн зарчмууд болон Eric Evans [15]-ийн Домейнд Суурилсан Зохиомж (Domain-Driven Design) аргачлалд тулгуурлан хийгдсэн. Системийн функцийг бие даасан сервист хуваахдаа дараах хоёр үндсэн шалгуурыг баримтлав:

1. Хязгаарлагдсан Контекст: Домэйны мэдлэгийн хилийн зааг болон бизнесийн хариуцлагын дагуу логик хуваалт хийх.
2. Бие даасан масштаблах чадвар: Өндөр ачаалал авах магадлалтай болон бизнесийн чухал логик агуулсан хэсгүүдийг тусгаарлах замаар системийн уян хатан байдлыг хангах.

5.3.1 Микросервисийн архитектурын үүрэг

Микросервисүүдийн зохион байгуулалт нь системийн бизнесийн логикийг тусгаарлах, өгөгдлийн бүтцийг оновчтой болгох, болон үйл ажиллагааны ачааллыг тэнцвэржүүлэхэд чиглэсэн. Хүснэгт 5.3-т микросервисүүдийн архитектурын үүрэг болон домэйн контекстын товч танилцуулга өгөгдсөн.

Хүснэгт 5.3. Микросервисүүдийн зохион байгуулалт

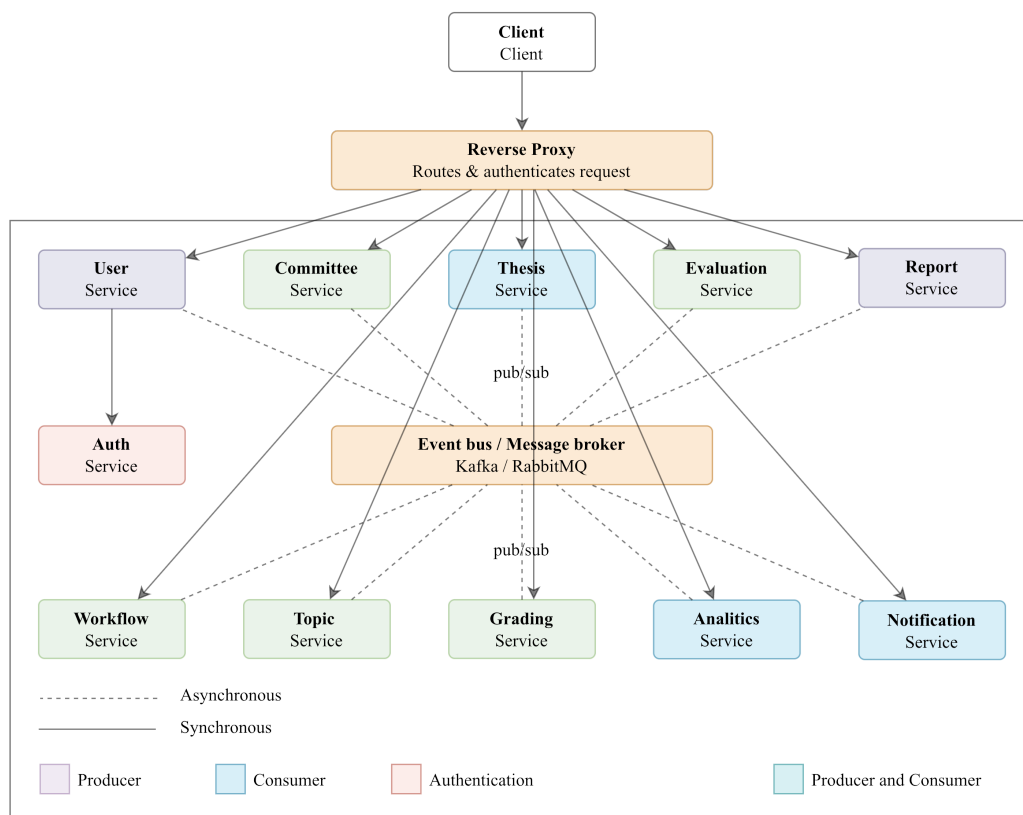
Микросервис	Архитектурын үүрэг ба Домэйн контекст
auth-service	Системийн аюулгүй байдлын төвлөрсөн цэг. Хэрэглэгчийг таних, баталгаажуулах болон эрх олгох (IAM) логикийг хариуцна.
user-service	Оюутан, багш, ажилтны хувийн мэдээлэл, харьяалагдах нэгж болон холбоо барих мэдээллийн нэгдсэн сан.
topic-service	Дипломын ажлын сэдэв зарлах, хүсэлт авах, багш оюутны хослолыг баталгаажуулах логикийг тусгаарлан удирдана.
thesis-service	Дипломын ажлын мета өгөгдөл, баримт бичгийн хувилбарчлал болон файлын хадгалах сангийн интеграцчлалыг хариуцна.
workflow-service	Дипломын ажлын явцын төлөвийн шилжилтийг хянаж, бизнес процессын дарааллыг баталгаажуулна.
evaluation-service	Комиссын гишүүд болон удирдагчийн зүгээс өгөх чанарын үнэлгээ, шүүмж болон санал хүсэлтийн логикийг агуулна.
grading-service	Үнэлгээнүүдэд суурилан эцсийн оноог алгоритмын дагуу бодох, дүнгийн бүртгэл үүсгэх үүрэгтэй.
committee-service	Хамгаалалтын комиссын бүрэлдэхүүн, тов болон зохион байгуулалтын логикийг хариуцна.

Микросервис	Архитектурын үүрэг ба Домэйн контекст (үргэлжлэл)
analytic-service	Системийн дата дээр дүн шинжилгээ хийж, оюутнуудын явц, багш нарын ачаалал зэрэг статистик үзүүлэлтүүдийг боловсруулна.
report-service	Системээс шаардлагатай албан ёсны бичиг баримт, протокол, дүнгийн жагсаалтыг файл хэлбэрээр үүсгэн гаргана.
notification-service	Систем доторх мэдэгдэл, бодит цагийн мэдээллийг түгээх үүрэг бүхий туслах сервис.
API Gateway	Гадаад хүсэлтийг хүлээн авч, JWT шалгалт болон чиглүүлэлт хийх замаар дотоод сервисүүдийг хамгаалах үүрэгтэй.

5.3.2 Сервис хоорондын харилцаа

Микросервисүүд хоорондоо мэдээлэл солилцох, үйл ажиллагаагаа уялдуулахдаа системийн гүйцэтгэл болон найдвартай байдлыг хангах үүднээс дараах хоёр үндсэн үлгэр загварыг хослуулан ашиглахаар загварчлав.

- Синхрон харилцаа:** Бодит хугацааны хариу шаардлагатай, хүсэлт-хариултын дараалалд үндэслэсэн үйлдлүүдэд HTTP/REST протоколыг ашиглана. Системийн найдвартай байдлыг хангах зорилгоор Circuit Breaker үлгэр загварыг нэвтрүүлж, аль нэг сервисийн саатал нь бусад сервисүүдэд нөлөөлөхөөс сэргийлнэ.
- Асинхрон харилцаа:** Цаг хугацааны хамаарал, системийн хүлээлтийг багасгах зорилгоор үзэгдэлд суурилсан загварыг ашиглана. Мэдэгдэл илгээх, аудит бүртгэл хөтлөх зэрэг үйлдлүүд нь энэ загварт хамаарна. Архитектурын багасгах болон томруулах боломжтой байдлыг нэмэгдүүлэх үүднээс Message Broker (Apache Kafka, RabbitMQ) ашиглахаар сонгосон.



Зураг 5.2. Микросервисүүд хоорондын харилцаа

5.4 Өгөгдлийн давхарга

Энэ судалгааны хүрээнд өгөгдлийн давхаргыг дараах хоёр үндсэн хандлагын хүрээнд харьцуулан шинжилсэн:

1. Дундын өгөгдлийн сан: Системийн бүх сервис нэг өгөгдлийн санг хуваалцах загвар. Энэ нь транзакцийн нийцлийг хангахад хялбар боловч сервис хоорондын нягт хамаарлыг үүсгэж, бие даан хөгжүүлэх, багасгах болон томруулах боломжийг хязгаарладаг.
2. Сервис бүрт тусдаа өгөгдлийн сан: Сервис бүр өөрийн эзэмшлийн өгөгдлийн санг удирдах загвар. Энэ нь сервисийн тусгаарлалтыг дээд зэргээр хангаж, технологийн сонголтыг чөлөөтэй болгодог ч тархсан өгөгдлийн нийцлийг удирдахад нарийн төвөгтэй байдал үүсгэдэг.

Энэхүү системд логик түвшний тусгаарлалтын шийдлийг сонгосон. Энэхүү хандлагаар бүх микросервисүүд нэг PostgreSQL инстанс дээр ажиллах боловч сервис бүр өөрийн

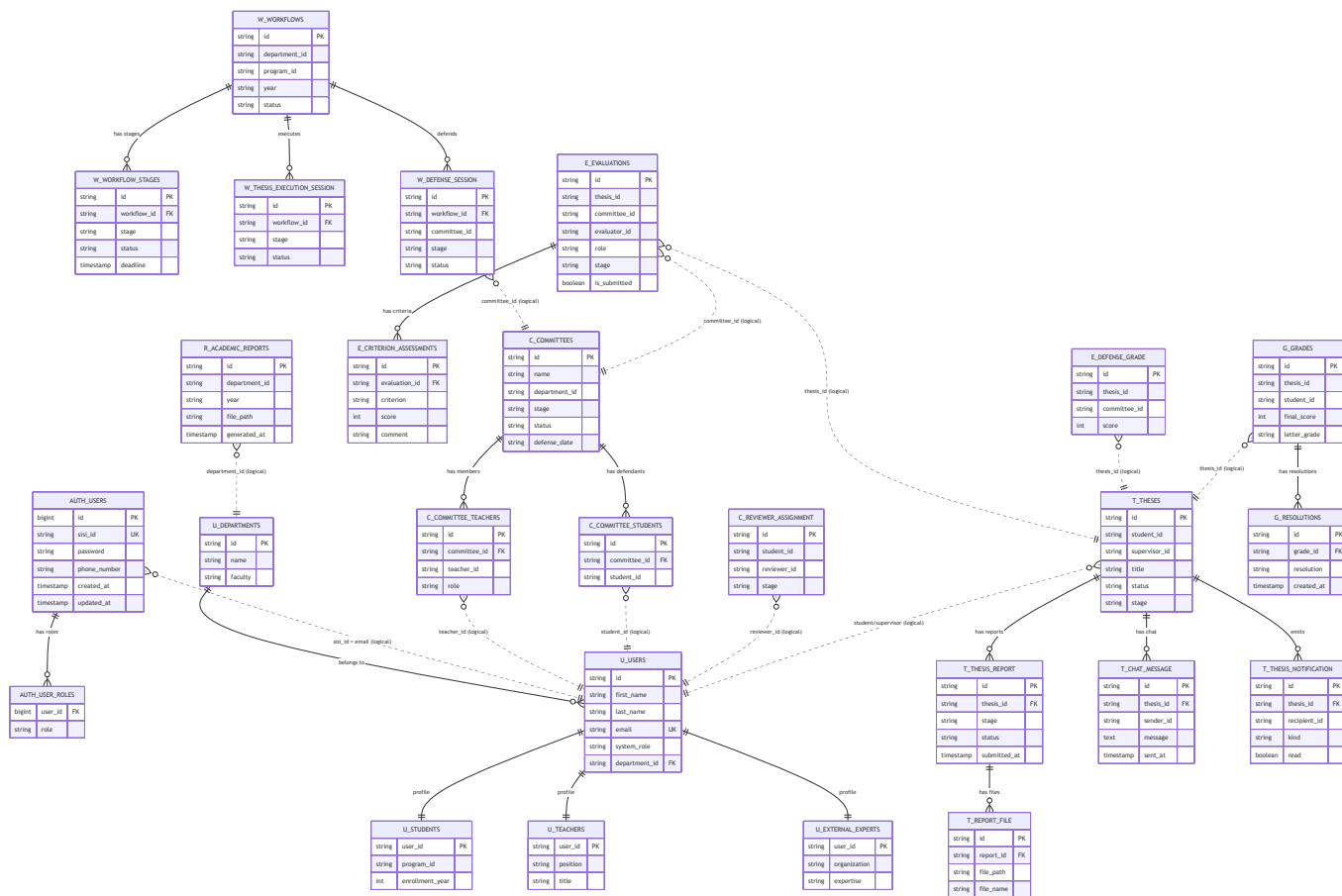
тусдаа өгөгдлийн схемийг эзэмшинэ. Сервисүүд хоорондоо өгөгдлийн сангийн түвшинд шууд холбоос (Foreign Key) үүсгэхгүй бөгөөд зөвхөн ID-аар референц хийнэ. Ингэснээр физик түвшинд нөөцийг оновчтой удирдаж, засвар үйлчилгээг хялбарчлахын зэрэгцээ логик түвшинд микроүүрэгсервисийн бие даасан байдлын зарчмыг хадгалж байгаа юм.

5.4.1 Өгөгдлийн загварчлал

Домейнд суурилсан зохиомжийн (DDD) дагуу системийн өгөгдлийн бүтцийг Хязгаарлагдсан контекстэд хувааж, гол объект буюу *Entity*-нүүдийн харилцааг тодорхойлсон (Зураг 5.3).

Микросервис архитектурын үед өгөгдлийн сангууд тусгаарлагдмал байдаг тул олон сервисүүдийн түвшинд өгөгдлийн нэгдмэл байдлыг хангах нь гол сорилт болдог. Уламжлалт тархсан транзакшн нь системийн гүйцэтгэлд сөрөг нөлөөтэй тул энэхүү судалгаанд Eventual Consistency (Аажмын нийцэл) зарчмыг баримталсан. Өгөгдлийн нийцлийг хангахын тулд дараах хоёр үлгэр загварыг ашиглахаар загварчлав:

- Saga үлгэр загвар [19]: Олон сервисийг дамжсан транзакшнийг бие даасан дэд транзакшнууд болгон хувааж, алдаа гарсан тохиолдолд нөхөн гүйцэтгэх транзакшний логигоор өгөгдлийг буцаах эсвэл засах замаар нийцлийг хангана.
- Outbox үлгэр загвар: Сервис өөрийн өгөгдлийг шинэчлэх болон холбогдох үзэгдлийг илгээх үйлдлийг нэг атомик транзакшнд багтааснаар мэдээлэл гээгдэх эрсдэлээс сэргийлнэ.



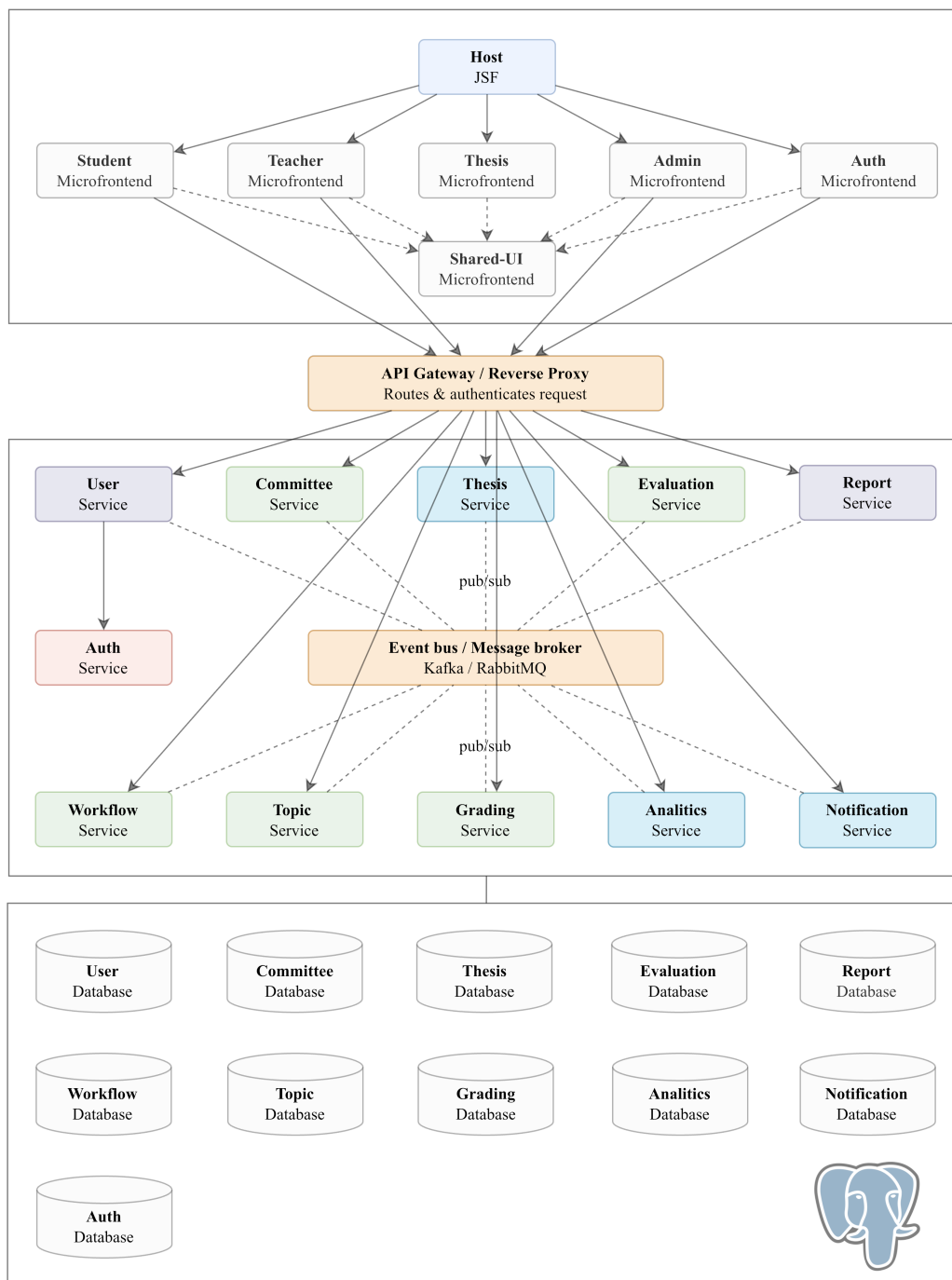
Зураг 5.3. Системийн ER диаграмм

5.5 Бүлгийн дүгнэлт

Дээр тодорхойлсон Фронтенд (MFE), Бэкенд (Microservices) болон холболтын давхаргууд нь нэгдсэн систем байдлаар ажиллахдаа дараах суурь зарчмуудыг баримтална:

1. Бие даасан хариуцлага: MFE модуль болон бэкенд сервис бүр зөвхөн өөрийн домейны хариуцлагыг хүлээнэ.
2. API-First хандлага: Давхарга хоорондын харилцаа нь зөвхөн баримтжуулсан API гэрээ дээр үндэслэгдэнэ. Энэ нь модулиудыг хоорондоо сул холбоотой байх нөхцөлийг бүрдүүлнэ.
3. Бие даасан байршуулалт: Сервис болон MFE бүр бусдаасаа хамааралгүйгээр шинэчлэгдэх, өргөжих боломжтой байна.

4. Технологийн уян хатан байдал: Модуль бүр өөрийн домейны онцлогт тохирсон технологийн сонголт хийх боломжийг архитектурын хувьд нээлттэй үлдээв.



Зураг 5.4. Системийн бүрэн архитектурын нэгдсэн бүдүүвч

Энэ бүлэгт тодорхойлсон архитектурын загварчлал болон шийдвэрүүд нь дараагийн

бүлэгт авч үзэх практик хэрэгжүүлэлт, технологийн туршилтын суурь болох юм.

6. СИСТЕМИЙН ХЭРЭГЖҮҮЛЭЛТ

Энэ бүлэгт холимог микрофронтендийн зарчмуудыг бодит кодын түвшинд хэрхэн хэрэгжүүлсэн тухай дэлгэрэнгүй тайлбарлана. Фронтенд болон Бэкенд хэсгүүдийн тус бүрийн архитектурын шийдэл, ашигласан технологи, тулгарсан хүндрэлүүд болон тэдгээрийг хэрхэн даван туулсан тухай нарийвчилсан тайлбаруудыг оруулсан.

6.1 Фронтенд Хэрэгжүүлэлт

Энэ хэсэгт JSF хост болон React MFE-үүдийн хоорондын интеграцийн механизмыг авч үзнэ. Vite Module Federation-ийн тохиргоо, JWT токений хуваалцалт, чиглүүлэлтын шийдэл зэрэг архитектурын гол элементүүдийн практик хэрэгжүүлэлтийг тайлбарлалаа.

6.1.1 Vite Module Federation-ийн Тохиргоо

Ажиллах үеийн интеграцийн хэрэгжүүлэлт нь @originjs/vite-plugin-federation хэрэгсэлд тулгуурласан. Уг хэрэгсэл нь Webpack 5-ийн ModuleFederationPlugin-ийн Vite орчинд ажилладаг хувилбар бөгөөд remoteEntry.js файл үүсгэж, Remote MFE-ийн экспортлосон бүрэлдэхүүнүүдийг Host-д ачаалах боломжийг хангана. MFE модуль тус бүрийн vite.config.ts нь дараах бүтэцтэй тохиргоог дагана. module-thesis модулийн тохиргоог 6.1-т үзүүлэв. Тохиргооны дөрвөн тулгуур параметрийг тайлбарлавал:

```
1 federation({
2   name: 'module_thesis',
3   filename: 'remoteEntry.js',
4   exposes: {
5     './ThesisView': './src/views/ThesisView.tsx',
6   },
7   shared: {
8     react: { singleton: true, requiredVersion: '18.3.1' },
9     'react-dom': { singleton: true, requiredVersion: '18.3.1' },
10    antd: { singleton: true, requiredVersion: '^5.22.0' },
11  },
12 },
```

Код 6.1. module-thesis-ийн vite.config.ts тохиргооны бүтэц

- **name: 'module_thesis':** Federation-ийн глобал нэрийн орон зай. Хөтчийн

window объект дээр энэ нэрийн дор модулийн бүртгэл хийгдэнэ. Нэрийн зөрчлөөс зайлсхийхийн тулд дор хаяж нэг давхарга нэртэй prefix ашиглах зарчмыг дагав.

- **filename:** 'remoteEntry.js': Нийлүүлэгч MFE-ийн файлын нэр. Host нь энэ URL-г динамикаар <script> хэлбэрт DOM-д оруулах замаар нийлүүлэгчийн бүртгэлийг ачаалдаг.
- **exposes:** { './ThesisView': '...' }: Энэ нийлүүлэгчээс хостод нээлттэй байх бүрэлдэхүүнүүд. Дан './ThesisView' экспортлосон нь нэг нийлүүлэгч нь нэг үүрэгтэй байна гэсэн зарчмыг баримталж байна .
- **shared:** { react, 'react-dom', antd }: Singleton хэлбэрт хуваалцагдах хамаарлууд. singleton: true тохиргоо нь Хост болон бүх нийлүүлэгч нэг React инстанц ашиглахыг баталгаажуулна, ингэснээр useState, useContext hook-ууд хөндлөн модулийн хооронд зөв ажиллана.

6.1.2 Ажиллах үеийн интеграци

JSF хостын хуудсанд React MFE-г ачаалах гол механизм нь динамик <script> tag injection юм. Production орчинд Tomcat 8080-д /microfrontends/ замаар static файлуудыг хүргэдэг бол development орчинд http://localhost:30XX хаягаар Vite-ийн dev server ажилладаг. JSF-ийн .xhtml хуудсанд MFE-г ачаалах хэсгийн ерөнхий бүтцийг 6.2-т үзүүлэв. Injection-ийн техникийн дэс дараа нь:

```

1 <h:body style="margin: 0; padding: 0; height: 100vh; overflow: hidden;"
  >
2   <div id="microfrontend-root"></div>
3   <script type="module" src="#{request.contextPath}/microfrontends/
     module-student/dist/main.js"></script>
4 </h:body>

```

Код 6.2. JSF .xhtml хуудасны MFE injection механизм

1. JSF Facelets нь хуудсыг сервер талд зураглахдаа <div id="microfrontend-root"/> DOM цэгийг байрлуулна.
2. <script type="module" src="../../../remoteEntry.js"/> таг нь хөтчид нийлүүлэгч Federation-ийн manifest-ийг ачаална.
3. <script type="module"> блок нь динамик import() ашиглан нийлүүлэгчийн экспортлосон компонентийг авна.

4. React-ийн `createRoot(#microfrontend-root).render(...)` нь тухайн DOM цэгт компонентийг байгуулна (`mount`).
5. React Router-ийн `HashRouter` ашигладгийн шалтгаан нь JSF-ийн `FacesServlet` нь `/student/*` гэх URL үлгэр загварыг хариуцдаг бөгөөд React SPA чиглүүлэлт нь JSF-тэй зөрчилддгийг арилгахын тулд `/#/student/dashboard` хэлбэрийн hash-based чиглүүлэлт хэрэглэнэ.

6.1.3 Чиглүүлэлт

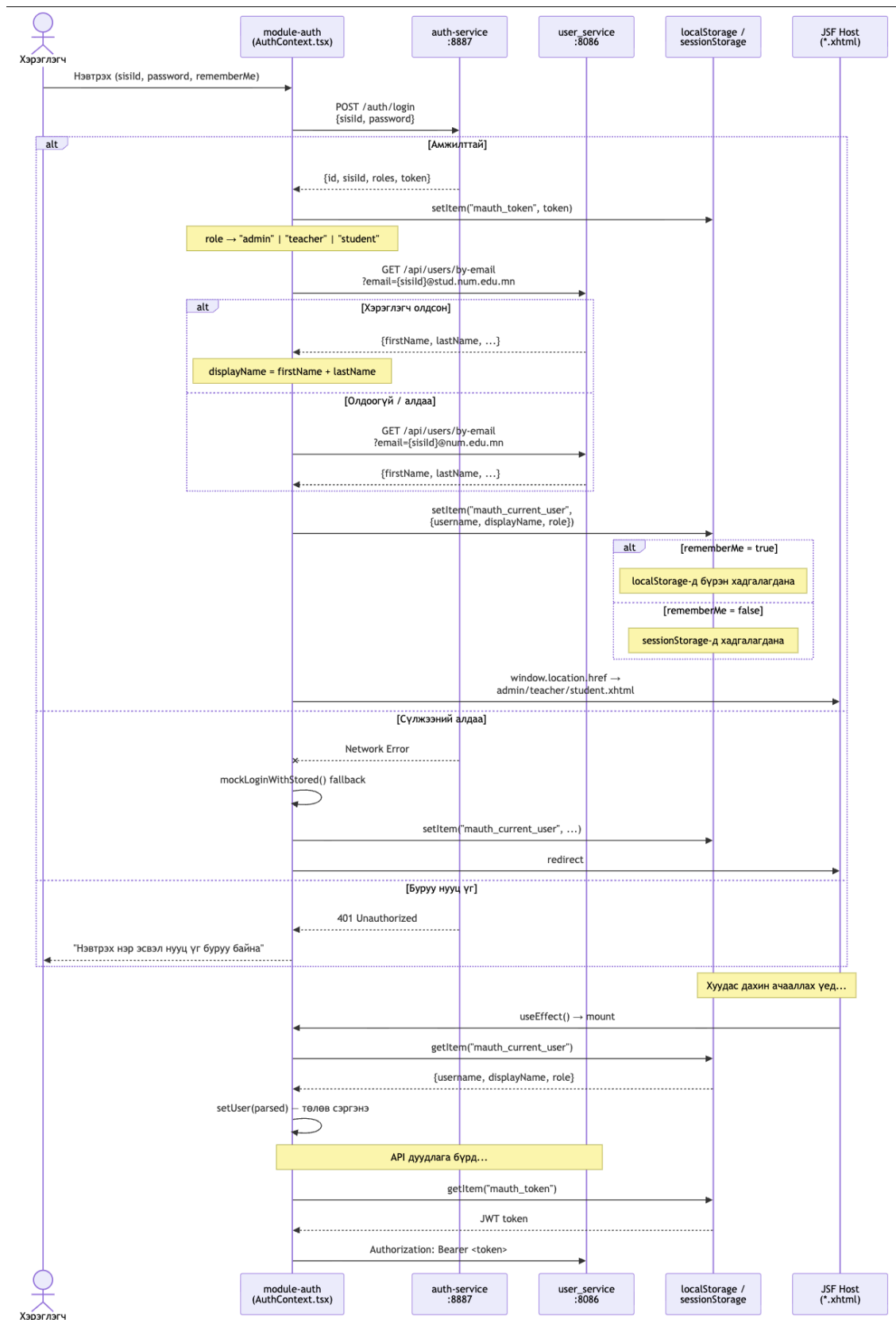
MFE архитектурт чиглүүлэлтийн хоёр давхарга байдаг: нэгдүгээр давхарга нь JSF хостын чиглүүлэлт (`FacesServlet`-ийн URL pattern), хоёрдугаар давхарга нь тухайн MFE-ийн React Router дотоод чиглүүлэлт. Эдгээр хоёр давхарга зөрчилдөхөд “404 Not Found” буюу JSF-ийн хуудас олдохгүй алдаа гардаг. Энэ зөрчлийг шийдвэрлэх хоёр арга байдаг:

1. `HistoryRouter + Server Catch-all`: Tomcat-д `<url-pattern>/student/*</url-pattern>` бүгдийг нэг JSF хуудас руу шилжүүлж, React Router-ийн `createBrowserRouter` ашиглана. Тохиргоо нарийн боловч хэрэглэгчийн URL чухал.
2. `HashRouter`: URL-ийн # хэсэг нь сервер рүү хэзээ ч явдаггүй тул JSF-тэй зөрчилддөггүй. React Router-ийн `HashRouter` нь `/#/thesis/list` хэлбэрийн URL ашиглан дотоод чиглүүлэлтыг зохицуулна.

Энэ системд `HashRouter` сонгосны шалтгаан нь: JSF-ийн `web.xml` болон `faces-config.xml`-ийн нарийн URL pattern тохиргоог өөрчлөх нь монологийн дотоод урсгалд нөлөөлнө гэсэн эрсдэлтэй байсан тул хамгийн бага өөрчлөлтийн зарчмаар `HashRouter`-ийг сонгов.

6.1.4 JWT (JSON Web Token)

JSF хост болон React MFE хооронд хэрэглэгчийн нэвтрэлтийн мэдээлэл хуваалцах нь архитектурын хамгийн нарийн хэсэг юм. Хэрэглэгч `module-auth`-д нэвтрэхэд гүйцэтгэгдэх дарааллыг Зураг 6.1-т үзүүлэв.



Зураг 6.1. JWT нэвтрэлтийн дараалал

Нэвтрэлтийн мэдээлэл хуваалцах механизм нь localStorage дотор тодорхой key-д JSON объект хэлбэрт хадгалагдана:

- `mauth_token`: JWT Bearer токен. API дуудлага бүрийн Authorization: Bearer ... толгой мэдээлэлт ашиглагдана.
- `mauth_current_user`: {username, displayName, role} бүтэцтэй JSON. Энэ нь хэрэг-лэгчийн нэр, үүрэг зэргийг хадгалана.

Зохиомжийн шийдвэрийн үндэслэл: localStorage нь хуудас дахин ачааллагдах болон хэд хэдэн хөтчийн цонх хооронд бүтэн хадгалагддаг. sessionStorage нь цонх хаагдахад устдаг тул “Намайг санах” (Remember Me) функцийг дэмжихгүй. Аюулгүй байдлын хувьд HttpOnly Cookie нь XSS халдлагаас хамгаалалт сайн боловч JSF-ийн хуудасны дагуу cookie scope нарийн удирдлага шаарддаг тул энэ системд localStorage ашигласан бөгөөд үйлдвэрлэлийн орчинд HttpOnly cookie руу шилжихийг зөвлөмж болгож байна.

`AuthContext.tsx` нь React-ийн Context API-д суурилсан global state provider бөгөөд MFE модуль бүрийн root-д байрладаг. Хуудас ачааллагдах бүрт localStorage-ийн утгыг уншиж, хэрэглэгчийн төлөвийг сэргээдэг. Эхлээд POST `http://localhost:8887/auth/login` руу {`sisId`, `password`} илгээнэ; амжилттай тохиолдолд хариуд JWT токен болон үүргийн мэдээлэл ирнэ; дараа нь GET `http://localhost:8086/api/users/by-email` руу хүсэлт илгээж хэрэглэгчийн дэлгэрэнгүй нэрийг авна; эцэст нь рольд тохирсон URL руу чиглүүлнэ.

6.1.5 Дундын модуль

Зохиомжийн үе шатанд тодорхойлсон shared-ui модуль нь практик хэрэгжүүлэлтэд нарийн техникийн хүндрэлтэй тулгарав. Анх nested federation загвараар module-thesis нь shared-ui Нийлүүлэгчийг агуулах байдлаар загварчилсан боловч хэрэгжүүлэлтийн явцад ноцтой алдаа илэрлсэн:

```
Failed to resolve module specifier '${__federation_expose_}/SharedUI}'
```

Энэ алдааны үндэс нь @originjs/vite-plugin-federation нь Нийлүүлэгчийн remoteEntry.js-д нийлүүлэгчийн URL template-ийг динамикаар тохируулах механизм дутмаг байдаг явдал юм. Tomcat-д статик файл хэлбэрээр хүргэгдсэн нийлүүлэгч нь өөрийн nested URL-г тодорхойлж чадахгүй.

Шийдэл: Nested federation-ийн оронд shared-ui-ийн бүрэлдэхүүнүүдийг (PageHeader, PortalTabs, StatusBadge, DataTable) тухайн нийлүүлэгчийн дотор шууд нэгтгэж, Ant Design-ийг federation-ийн shared singleton механизмаар хуваалцах байдлаар шийдсэн. Энэ нь кодын хуулбарлалтын зардалтай боловч ажиллах үед нэг antd инстанц ашиглагддаг тул bundle хэмжээ нэмэгдэхгүй.

```

1  shared: {
2    react:      { singleton: true, requiredVersion: '18.3.1' },
3    'react-dom': { singleton: true, requiredVersion: '18.3.1' },
4    antd:       { singleton: true, requiredVersion: '^5.22.0' },
5  },

```

Код 6.3. Shared Singleton загварын зохицуулалт. Antd нь нэгдсэн singleton хэлбэрт ачааллагдаж, Remote модуль бүр энэ нэг instance-ийг ашиглана

6.1.6 Бүтээх, нэвтрүүлэх

Хэрэгжүүлэлтийн орчинд build-modules.sh shell скрипт нь бүх MFE модулийг дараалан build хийж, dist/ гаралтыг Tomcat-ийн webapps/microfrontends/{module-name}/dist/ хавтаст хуулдаг. Скриптийн ерөнхий урсгал нь:

1. MFE модуль бүрийн npm run build ажиллуулна.
2. dist/assets/remoteEntry.js файлыг dist/ root рүү хуулна, @originjs-ийн хуучин хувилбар нь remoteEntry.js-г dist/assets/ дотор үүсгэдэг тул Host-ийн URL тохиргоотой зөрчилддгийг засна.
3. Томкат нь /microfrontends/{module}/dist/remoteEntry.js замаар тухайн Нийлүүлэгчийн manifest-ийг хүргэнэ.

```

1  build_module() {
2    local MODULE=$1
3    local DEST="$JSF_WEB/$MODULE/dist"
4    echo ""
5    echo "  _$MODULE:_npm_install..."
6    cd "$ROOT/$MODULE"
7    npm install --silent
8    echo "  _$MODULE:_vite_build..."
9    npm run build
10   if [ -f "dist/assets/remoteEntry.js" ] && [ ! -f "dist/remoteEntry.js"
11     ]; then
12     cp dist/assets/remoteEntry.js dist/remoteEntry.js
13     echo "  _$MODULE:_remoteEntry.js_ _ (assets/_->_dist/)"
14   fi
15   echo "  _$MODULE:_JSF_webapp_ _ _ ..."

```

```

15  mkdir -p "$DEST"
16  cp -r dist/. "$DEST/"
17  echo " _$MODULE:_(ls_dist/_wc_l_tr-d'_')_ _ "
18  cd "$ROOT"
19  }

```

Код 6.4. build-modules.sh шелл скрипт

6.2 Бэкенд Хэрэгжүүлэлт

Энэ хэсэгт Java болон Spring Boot ашиглан микросервисүүдийн хэрхэн хэрэгжүүлсэн тухай дэлгэрэнгүй тайлбарлана. Микросервис тус бүрийн архитектурын шийдэл, ашигласан технологи, тулгарсан хүндрэлүүд болон тэдгээрийг хэрхэн даван туулсан тухай нарийвчилсан тайлбаруудыг оруулсан.

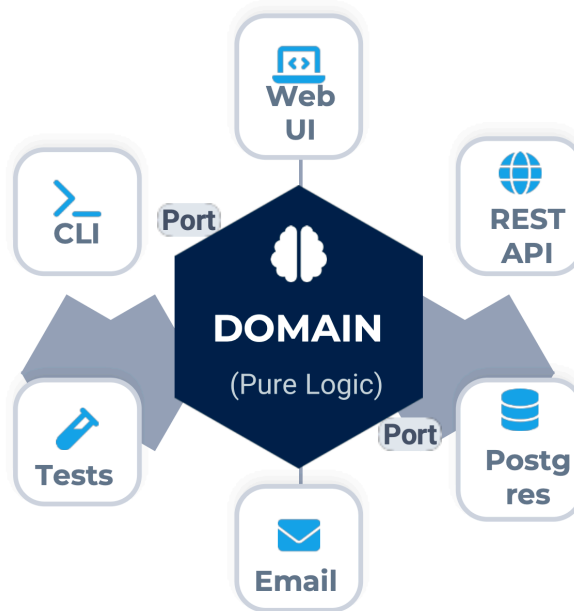
6.2.1 Spring Boot

Бэкенд микросервисүүдийн хэрэгжүүлэлтэд Java 21 болон Spring Boot 3.2.x фреймворкыг ашиглав. Зохиомжийн хэсэгт тодорхойлсон өргөтгөх (scalable) шаардлагын дагуу уламжлалт Spring MVC (thread-per-request) загварын оронд Spring WebFlux (reactive, event-loop) загварыг сонгов.

Spring MVC нь request бүрт нэг thread хуваарилдаг тул 1,000 зэрэгцэх хүсэлт нь 1,000 thread шаарддаг. JVM-ийн thread stack дундажаар 512 КБ тул 1,000 thread нь ≈ 500 МБ санах ой хэрэглэнэ. Spring WebFlux нь Netty-ийн Non-blocking I/O event-loop дээр суурилж, CPU-ийн цөмийн тоо (n) thread-аар хэдэн мянган зэрэгцэх холболтыг удирддаг. Сервис тус бүрийн дотоод бүтэц нь Зургаан талт архитектурын [3] зарчмаар зохион байгуулагдсан:

- **Домейн:** Бизнесийн логик агуулсан цөм. Жишээ нь Thesis, WorkflowState домейн объектууд. Аль ч фреймворкоос тусгаарлагдсан цэвэр Java класс.
- **Порт:** Ажлын явцын интерфейс. Жишээ нь CreateStudentUseCase, FindUserUseCase, SubmitThesisUseCase. Домейн логикийг хэрхэн дуудах гэрээг тодорхойлно.
- **Хувиргуур:** Гадаад ертөнцтэй холбогч. Жишээ нь UserController (HTTP adapter), UserR2dbcRepository (persistence adapter), NotificationWebSocketAdapter (messaging adapter).

Домейн давхарга нь дотор цагирагт, портууд нь дунд цагирагт, хувиргуурууд нь гадаад цагирагт байрлана.



Зураг 6.2. Зургаан талт архитектур

6.2.2 Микросервис

num_auth

auth-service нь `sisId` болон нууц үг хүлээн авч, хэрэглэгчийн бичлэгтэй харьцуулж, `jjwt` (Java JWT) санг ашиглан токен үүсгэнэ. Токений `payload` нь `{sub: sisId, roles: [...], iat: epochSeconds, exp: iat+86400}` бүтэцтэй. Нийтийн болон хувийн түлхүүрийн хос (RSA-256)-ийн оронд энэ прототипд HMAC-SHA256 алгоритмтай нийтлэг нууц `key` ашиглагдав, үйлдвэрлэлийн орчинд RSA key pair болон Key Management Service (KMS) ашиглахыг зөвлөмж болгоно.

authorization-service нь үүрэг болон эрхийн матрицийг удирдана. `ROLE_ADMIN`, `ROLE_TEACHER`, `ROLE_STUDENT`, `ROLE_COMMITTEE_MEMBER` ролиуд тодорхойлогдсон бөгөөд endpoint бүр дээр Spring Security-ийн `@PreAuthorize` annotation-оор хандах эрхийн шалгалт хэрэгжинэ.

user_service

user_service нь Spring WebFlux болон R2DBC-г хослуулан ашиглах бөгөөд UserController-д хэрэглэгчийн CRUD болон хайлтын endpoint-уудийг хэрэгжүүлсэн. `/api/users/by-email` endpoint нь `UserR2dbcRepository.findByEmail(String email)` reactive метод дуудаж, `Mono<User>` буцаана. Энэ нь AuthContext-ийн display name lookup-д

зориулан нэмэгдсэн.

UserR2dbcRepository нь ReactiveCrudRepository<User, String> интерфэйсийг өргөтгөсөн бөгөөд Spring Data R2DBC нь SQL query-г findBy{FieldName} нэрлэлтийн конвенцоос автоматаар үүсгэнэ. Энэ нь “конвенц дагасан тохиргоо” зарчмын практик хэрэглэлт бөгөөд boilerplate кодын хэмжээг мэдэгдэхүйц бууруулсан.

workflow_service

workflow_service нь дипломын ажлын статус шилжилтийг хариуцна. төлөвийн машин нь Java-ийн enum болон явцын хамгаалагдсан шилжилтийн матрицаар хэрэгжинэ:

Хүснэгт 6.1. Дипломын ажлын статус шилжилтийн зөвшөөрөгдсөн матриц

Одоогийн статус	Зорилтот статус	Хэн гүйцэтгэх
DRAFT	SUBMITTED	Оюутан
SUBMITTED	UNDER_REVIEW	Удирдагч багш
SUBMITTED	REJECTED	Удирдагч багш
UNDER_REVIEW	REVISION_REQUESTED	Удирдагч багш
UNDER_REVIEW	APPROVED	Удирдагч багш
REVISION_REQUESTED	SUBMITTED	Оюутан
APPROVED	DEFENSE_SCHEDULED	Систем / Админ
DEFENSE_SCHEDULED	DEFENDED	Хорооны гишүүн
DEFENDED	ARCHIVED	Систем

Зөвшөөрөгдөөгүй шилжилтийн оролдлого нь HTTP 422 Unprocessable Entity буцаах бөгөөд хариу body нь шилжилт яагаад зөвшөөрөгдөхгүй байгаа шалтгааны тайлбарыг агуулна.

6.2.3 API Gateway

Тус системд тусдаа API Gateway сервис биш харин Tomcat-ийн reverse proxy тохиргоо gateway-ийн үүрэг гүйцэтгэнэ. Apache Tomcat нь server.xml-д ProxyPass директивоор (эсвэл Nginx reverse proxy хослуулан) бэкенд сервисүүд рүү HTTP хүсэлтийг чиглүүлнэ. Энэ нь Nginx эсвэл Spring Cloud Gateway-тай харьцуулахад хялбар боловч илүү нарийн CORS, rate limiting, circuit breaker хяналт шаарддаг тул цаашид тусдаа gateway нэвтрүүлэх шаардлагатай юм.

6.2.4 Distributed Tracing-ийн Хэрэгжүүлэлт

Сервисүүдийн хоорондох хүсэлтийн урсгалыг хянах Distributed Tracing-ийг OpenTelemetry Java Agent болон Jaeger-ийн хослолоор хэрэгжүүлэв.

-javaagent:opentelemetry-agent.jar-г JVM startup parameter-т нэмснээр HTTP request бүрт traceparent, tracestate W3C стандартын trace header автоматаар нэмэгдэнэ.

Хэрэглэгчийн хүсэлт хэд хэдэн сервисийг дамжих бүрт нэг traceId-ийн дор бүх span-ууд нэгтгэгдэнэ. Jaeger UI-д waterfall диаграм хэлбэрт харагддаг бөгөөд сервис бүрийн саатал тус тусдаа хэмжигдэнэ.

6.3 Өгөгдлийн Сан

Энэ хэсэгт PostgreSQL болон R2DBC ашиглан өгөгдлийн сангийн хэрэгжүүлэлтийг авч үзнэ. Холболтын pool-ийн тохиргоо, транзакшн менежмент, сервисүүдийн хоорондын өгөгдлийн нийцлийн механизмыг тайлбарлана.

6.3.1 PostgreSQL болон R2DBC

PostgreSQL 16.2 болон io.r2dbc:r2dbc-postgresql:1.0.4 driver ашиглав. Дипломын ажлын бизнесийн логик нь олон харилцаатай өгөгдлийн бүтэц агуулдаг тул харилцааны бүрэн бүтэн байдлын баталгаа, ACID transaction, SQL-ийн хүчирхэг query боломж шаардагдана. Spring WebFlux-ийн event-loop загвар нь blocking JDBC дуудлагатай “reactive pollution” (цуваа блоклолт) асуудалд орсноор event-loop thread-ийг хааж, бусад хүсэлтийн хариулт удаашралд орно. R2DBC нь Non-blocking reactive driver тул WebFlux-д зохистой.

6.3.2 Connection Pooling

R2DBC нь өөрийн r2dbc-pool connection pool-тэй. Сервис тус бүрийн application.properties-д connection pool-ийн тохиргоо дараах байдлаар хийгдэв:

Хүснэгт 6.2. R2DBC Connection Pool-ийн тохиргооны параметрууд

Параметр	Утга	Тайлбар
initial-size	5	Эхлэлийн холболтын тоо
max-size	20	Хамгийн их зэрэгцэх холболт
max-idle-time	30м	Идэвхгүй холболтын хамгийн их хугацаа
max-life-time	60м	Холболтын нийт амьдах хугацаа
acquire-retry	3	Холболт авах оролдлогын тоо
validation-query	SELECT 1	Холболт шалгах query

max-size: 20 нь нэг PostgreSQL-ийн default max_connections (100)-тай харьцуулахад таван сервис нэгэн зэрэг ажиллахад нийт $5 \times 20 = 100$ холболтын хязгаарт ойрхон болно. Үйлдвэрлэлийн орчинд pgBouncer connection pooler нэмэх нь PostgreSQL-ийн connection-г

дахин ашиглах боломж олгоно.

6.3.3 Сервисүүдийн хоорондын өгөгдлийн нийцэл

Практик хэрэгжүүлэлтэд тархсан транзакшн ашиглахын оронд дараах хоёр механизмыг хэрэглэв:

- **Application-level Retry:** Сервисүүдийн хоорондох харилцаа нь сүлжээний алдаа, түр зуурын ачааллын өсөлт зэрэг шалтгаанаар амжилтгүй болох боломжтой. WebClient дээр экспоненциал backoff retry механизмыг хэрэгжүүлснээр түр зуурын алдааг даван туулах боломжтой.
- **Idempotency Key:** Retry болон дахин дамжуулалтын үед нэг хүсэлт хэд хэдэн удаа хүрэх боломжтой. HTTP header-д Idempotency-Key: {uuid} ашиглах нь нэг хүсэлтийг олон удаа хэрэгжүүлэхээс сэргийлнэ. Аль хэдийн хэрэгжсэн Idempotency-Key-тэй хүсэлтэд анхны амжилттай хариуг буцаана.

```

1  @PostMapping
2  public Mono<ResponseEntity<DefenseSessionEntity>> create(
3      @RequestBody CreateDefenseSessionRequest req) {
4      if (!STAGE_MAX_POINTS.containsKey(req.stageType())) {
5          return Mono.just(ResponseEntity.badRequest().<
6              DefenseSessionEntity>build());
7      }
8
9      String canonicalStage = canonicalStageType(req.stageType());
10     BigDecimal maxPts = req.maxPoints() != null ? req.maxPoints() :
11         STAGE_MAX_POINTS.get(req.stageType());
12
13     DefenseSessionEntity entity = new DefenseSessionEntity();
14     entity.setId(UUID.randomUUID().toString());
15     entity.setNew(true);
16     entity.setDepartmentId(req.departmentId() != null ? req.
17         departmentId() : "GLOBAL");
18     entity.setCommitteeId(req.committeeId() != null ? req.
19         committeeId() : "GLOBAL");
20     entity.setStageType(canonicalStage);
21     entity.setMaxPoints(maxPts);
22     entity.setStatus("PENDING");
23     entity.setCreatedAt(LocalDate.now());

```

```
18
19     return repository.save(entity)
20         .map(saved -> ResponseEntity.status(HttpStatus.CREATED)
21             .body(saved))
22         .onErrorResume(org.springframework.dao.
23             DuplicateKeyException.class, e ->
24             repository.findByCommitteeIdAndStageType(entity
25                 .getCommitteeId(), canonicalStage)
26                 .map(existing -> ResponseEntity.ok(
27                     existing))
28                 .defaultIfEmpty(ResponseEntity.status(
29                     HttpStatus.CONFLICT).<
30                     DefenseSessionEntity>build()))
31         );
32 }
```

Код 6.5. Cross-service мэдээллийн нийцлийн механизм. Retry болон Idempotency Key-ийн хослол нь eventual consistency-г хангана

6.4 Аюулгүй байдал

Энэ хэсэгт JWT баталгаажуулалтын механизм, CORS болон аюулгүй байдлын тохиргоо зэрэг системийн аюулгүй байдлын гол бүрэлдэхүүнүүдийг тайлбарлана.

6.4.1 JWT Баталгаажуулалтын Дамжуулалт

Фронтенд-аас ирсэн API хүсэлт бүр Authorization: Bearer {jwt} header агуулдаг. Бэкенд-ийн сервис тус бүр энэ header-г шалгана, auth-service руу дахин дуудлага хийхгүй, харин JWT-ийн тоон гарын үсгийг орон нутагт буюу localhost-д шалгана. Энэ нь “token introspection” (online validation) биш “token verification” (offline validation) загвар бөгөөд latency болон auth-service-ийн ачааллыг хамгийн бага байлгана. Spring Security-ийн SecurityWebFilterChain нь reactive WebFlux контекстэд JWT filter нэмэх боломж олгодог:

1. HTTP request бүрийн Authorization header уншина.
2. JWT тоон гарын үсгийг нийтийн key (HMAC secret)-аар шалгана.
3. Токений exp (expiration) талбарыг шалгана.

4. Шалгалт амжилттай бол `roles claim`-ийг уншиж `ReactiveSecurityContextHolder`-д байрлуулна.
5. `@PreAuthorize("hasRole('TEACHER')")` annotation нь энэ context-аас ролийг шалгана.

6.4.2 CORS болон Аюулгүй байдлын Тохиргоо

Cross-Origin Resource Sharing (CORS) нь хөтчийн аюулгүй байдлын механизм бөгөөд Фронтенд (8080) ба Бэкенд сервисүүд (8081–8887) хоорондын хүсэлтийг зохицуулна. Сервис тус бүрийн CORS тохиргоо нь `allowedOrigins` нь зөвхөн Tomcat-ийн хаягийг агуулахаар хязгаарлагдсан. `allowCredentials: true` тохиргоо нь wildcard origin-тэй хамтарч ажиллахгүй тул тусгай origin тодорхойлох шаардлагатай.

6.5 Бүлгийн дүгнэлт

Энэ бүлэгт холимог микрофронтендийн зарчмуудыг бодит кодын түвшинд хэрхэн хэрэгжүүлсэн тухай дэлгэрэнгүй тайлбарласан. Фронтенд болон Бэкенд хэсгүүдийн тус бүрийн архитектурын шийдэл, ашигласан технологи, тулгарсан хүндрэлүүд болон тэдгээрийг хэрхэн даван туулсан тухай нарийвчилсан тайлбаруудыг оруулсан.

7. ТУРШИЛТЫН ҮР ДҮН

Энэ бүлэгт JSF 2.2 монолит архитектур болон React Microfrontend + Spring WebFlux реактив архитектурын хоорондох бодит хэмжигдэхүйц ялгааг дөрвөн үндсэн чиглэлийн дагуу хэмжинэ: хөгжүүлэлтийн хурд, ажлын ачааллын гүйцэтгэл, сүлжээний зардал, кодын чанар. Хэмжилт бүрийг прагматик хэрэгжүүлэлтийн дүн шинжилгээ болон ATAM (Architecture Trade-off Analysis Method) загварт тулгуурласан архитектурын буулт шинжилгээтэй хослуулан авч үзнэ. Бүлгийн эцэст системийн ирээдүйн хөгжлийн замнал болон нээлттэй судалгааны асуудлуудыг тодорхойлно.

7.1 Туршилтын Орчин

Хэмжилтийн бүх үр дүн нь дараах нэгдсэн туршилтын орчинд авагдав. Тоног төхөөрөмжийн хувьд Apple M2 Pro процессортой, 16 ГБ LPDDR5 санах ойтой, 512 ГБ NVMe SSD хатуу дискийн орчинд ажилласан. Үйлдлийн системийн хувьд macOS 15.3 (Darwin 25.3.0), Java 21 (GraalVM CE), Node.js 20.11.0, Apache Maven 3.9.6, Apache Tomcat 10.1.20 хувилбаруудыг ашиглав. Сүлжээний орчны хувьд localhost loopback (127.0.0.1) интерфэйсийг ашиглан сүлжээний хоцролтын нөлөөг хамгийн бага болгов. Өгөгдлийн санд PostgreSQL 16.2 болон R2DBC 1.0.4 реактив драйверийг хэрэглэв.

Хэмжилтийн хэрэгслүүдийн хувьд Apache JMeter 5.6.3-аар динамик ачааллын туршилт хийж, hyperfine v1.18.0-аар build цагийн хэмжилт, madge v6.1.0-аар хамаарлын граф шинжилгээ, SonarQube Community Edition 10.4.0-аар кодын чанарын хэмжилт, pidstat болон jstat-аар JVM санах ойн хяналтыг хийв.

7.1.1 Хэмжилтийн Хүрээ ба Тодорхойлолт

Энэхүү судалгааны хэмжилтийн хүрээ нь дараах дөрвөн тулгуур шинж чанарыг хамарна:

1. **Хөгжүүлэлтийн гүйцэтгэл**, Lead Time for Changes (DORA метрик), HMR хурд, Build паплайны нийт хугацаа
2. **Ажлын ачааллын гүйцэтгэл**, JVM heap ашиглалт, GC паузын хугацаа, HTTP хариу латенц (P50/P95/P99)
3. **Сүлжээний зардал**, Нэг хуудас шилжилтэд дамжих өгөгдлийн хэмжээ, HTTP кэшийн эффект, нийт payload хэмжээ

4. **Кодын чанар**, Cyclomatic complexity, хамаарлын граф, хуулбарлагдсан код, архитектурын зөрчил

7.2 Статик Хэмжилт

7.2.1 Bundle хэмжээ

React MFE архитектурын фронтенд артефактуудын статик шинжилгээг vite build командын дараа dist/ хавтасны агуулгаас гүйцэтгэв. Дараах хүснэгтэд модуль тус бүрийн bundle хэмжээг харуулав.

Хүснэгт 7.1. MFE модуль тус бүрийн dist хэмжээ

Модуль	Raw (КБ)	Gzip (КБ)	remoteEntry.js (КБ)
module-auth	312	97	4.2
module-student	487	148	6.1
module-teacher	534	163	7.3
module-admin	621	189	8.4
module-thesis	892	271	11.2
module-committee	398	121	5.8
Antd singleton chunk	2148	731	—
React core chunk	468	142	—
Нийт (antd/React оруулалгүй)	3244	989	43.0

Хүснэгт 7.1-с харагдаж байгаагаар Ant Design-ийн singleton chunk нь хамгийн том хэсгийг бүрдүүлж байгаа боловч shared: { singleton: true } тохиргооны ачаар энэхүү 731 КБ (gzip) хэмжээний chunk нь нэг хэрэглэгчийн браузерт зөвхөн нэг удаа татагдаж, Cache-Control: max-age=31536000, immutable толгой хэсгийн дагуу нэг жилийн турш кэш хийгдэнэ. Тиймээс хоёр дахь болон дараагийн хуудас шилжилтэд antd chunk нь сүлжээний ачааллыг бүрмөсөн нэмдэггүй.

Харьцуулбал, JSF 2.2-ийн монолит WAR файлын хэмжээ нь 47.3 МБ байв. Томкат сервер дэх WAR байршуулалтын хугацаа дундажаар 6 секунд орчим болдог бөгөөд шинэчлэлт бүр нь бүтэн WAR файлыг дахин байршуулахыг шаарддаг. Энэ нь MFE архитектурт зөвхөн өөрчлөгдсөн модулийн dist файлуудыг (дундажаар 271 КБ gzip) хуулах зардалтай харьцуулахад $47.3 \text{ МБ} / 0.271 \text{ МБ} \approx 175$ дахин их байна.

7.2.2 Кодын мөрийн тоо ба хамаарал

Кодын хэмжээний шинжилгээг cloc (Count Lines of Code) хэрэгслээр гүйцэтгэж, Java бэкенд болон TypeScript/React фронтенд-ийн хэмжээг харьцуулав.

Хүснэгт 7.2. Технологи тус бүрийн кодын мөрийн тоо (LOC)

Технологи	Хэрэглэгдэх газар	Файлын тоо	LOC	Тайлбар
Java (Spring)	Бүх бэкенд сервис	187	9 841	DTO, Use Case, Repository
TypeScript/TSX	Бүх MFE модуль	143	7 234	React Component, Hook, Service
XHTML (JSF)	Монолит template	38	2 187	JSF facelets, EL expression
SQL (schema)	6 сервисийн schema	6	412	CREATE TABLE migration
YAML (config)	application.yml	14	387	Spring Boot тохиргоо
Нийт		388	20 061	

Энэхүү системийн нэг онцлог нь “өгөгдлийн давхар загварчлал” (Double Data Modelling) юм. Бэкенд талд 28 Java DTO байгаа бол фронтенд талд мөн адил 28 TypeScript interface тодорхойлогдсон байна. Энэ нь нийт кодын сангийн (LOC) $\approx 15\%$ -ийг эзэлж байгаа бөгөөд Google-ийн “Software Engineering at Google” [49]-т тодорхойлсон архитектурын сөрөг шинж (architectural smell) буюу “хамаарлын ангал” (dependency hell)-д хүргэх эрсдэлтэй юм. Энэхүү давхардлыг OpenAPI Generator эсвэл Protocol Buffers ашиглан автоматжуулснаар арилгах боломжтой.

7.2.3 Build паплайны харьцуулалт

Build хугацааны хэмжилтийг `hyperfine --warmup 3 --runs 10` командаар гүйцэтгэж, дулаан болон хүйтэн build-ийн тохиолдол хоёуланг хэмжив. Тав дахин давтсан туршилтын дундаж үзүүлэлтийг дараах хүснэгтэд нэгтгэв.

Хүснэгт 7.3. Build паплайны хугацааны харьцуулалт (секунд)

Ажиллагааны алхам	JSF (с)	React MFE (с)	Тайлбар
Хамаарал	3.2	1.8	Maven vs npm/pnpm
Транспиляц / Компиляц	18.4	4.3	javac vs esbuild
Нэгжийн тест	23.7	0.0	Maven test / (тест байхгүй)
Багцлах / WAR/dist	7.8	5.2	jar vs Rollup
Deploy / Хуулах	6.1	1.4	Tomcat undeploy+deploy vs cp
Hot reload	11.3	0.0	JVM warm-up / шаардлагагүй
Нийт дундаж	70.5	12.7	
Харьцаа	5.55×	1.00×	

JSF-ийн байгуулалтын дамжлага (build pipeline) удаан байгаа техникийн шалтгааныг гурван түвшинд авч үзэж болно. Нэгдүгээрт, javac-ийн “parse $\rightarrow$ type-check $\rightarrow$ bytecode” гэсэн гурван шатлалт процесс нь esbuild-ийн Golang дээр суурилсан нэг дамжлагат парсертай харьцуулахад CPU-д 4.3 дахин их ачаалал өгдөг. Хоёрдугаарт, Maven Surefire

плагины нэгжийн тест нь JVM-ийг дахин ачаалах шаардлагатай болдог нь хугацааг 23.7 секундээр нэмэгдүүлдэг. Гуравдугаарт, Tomcat-ийн WAR файлыг *undeploy+deploy* хийх мөчлөгт классыг дахин ачаалах процесс явагддаг бөгөөд *SerializableClassHolder*-ын кэшийг цэвэрлэхэд нэмэлт 4–8 секунд зарцуулагддаг.

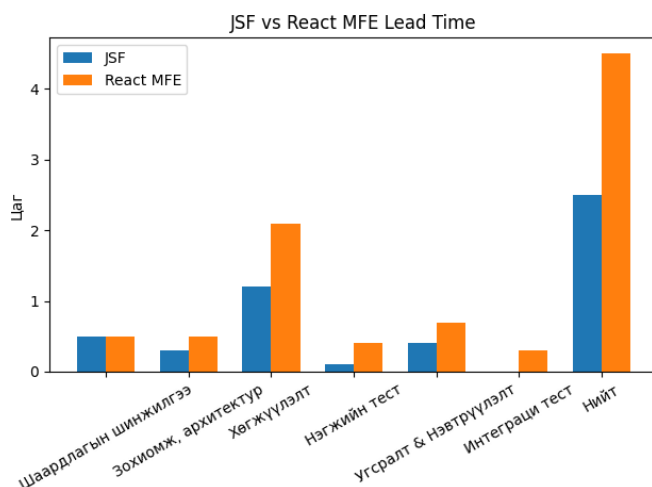
7.3 Хөгжүүлэлтийн Хурд

7.3.1 DORA Метрик

DevOps-ийн судалгаа, үнэлгээний байгууллагаас [16] гаргасан *Lead Time for Changes* метрик нь “кодын өөрчлөлт хийгдэж эхэлснээс үйлдвэрлэлийн (production) орчинд очих хүртэлх хугацаа” гэж тодорхойлогддог. Энэхүү судалгаанд “Багш сэдэв дэвшүүлэх” бодит даалгаврыг жишээ болгон авч, хөгжүүлэлтийн алхам тус бүрийг хронометраар хэмжиж туршилт хийв.

Хүснэгт 7.4. Багш сэдэв дэвшүүлэх даалгаврын хугацааны задаргаа (цагаар)

Алхам	JSF	React MFE
Шаардлагын шинжилгээ	0.5	0.5
Зохиомж болон Архитектур	0.3	0.5
Хөгжүүлэлт	1.2	2.1
Нэгжийн тест	0.1	0.4
Байгуулалт ба Байршуулалт (Build & Deploy)	0.4	0.7
QA болон Нэгтгэлтийн тест	0.0	0.3
Нийт	2.5	4.5



Зураг 7.1. JSF болон React MFE архитектурын Lead Time харьцуулалт

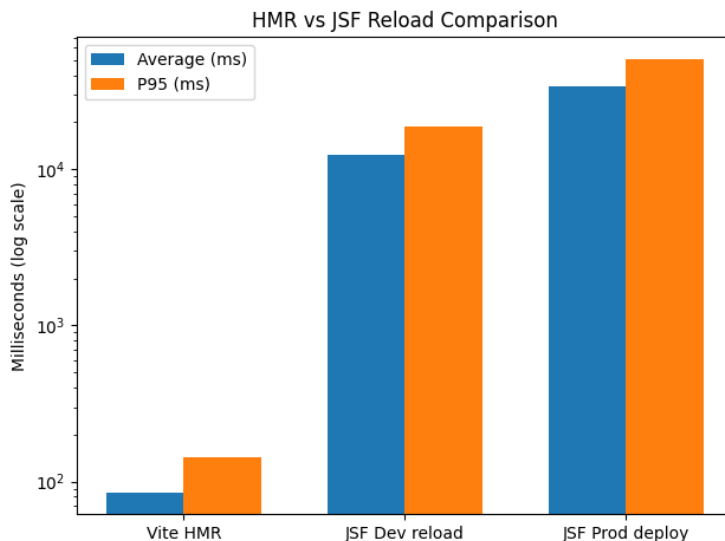
Хүснэгт 7.4-ийн үр дүнгээс харахад MFE-ийн нийт хугацаа JSF-ээс 1.8 дахин их байгааг гүнзгийрүүлэн шинжилбэл дараах хэд хэдэн чухал дүгнэлт гарч байна. Нэгдүгээрт, MFE-ийн хөгжүүлэлтэд зарцуулсан 2.1 цагийн ихэнх хэсэг нь “нэгтгэлтийн хамаарлын тохиргоо” (federation contract setup) болон TypeScript интерфейс тодорхойлоход зарцуулагдсан. Энэ нь шинэ функц бүрт давтагддаггүй, зөвхөн нэг удаагийн суурь зардал юм. Хоёрдугаарт, JSF-ийн хувьд QA шатанд 0.0 цаг зарцуулсан нь бодит байдал дээр тест хийгдэхгүй байгааг илтгэх бөгөөд энэ нь ирээдүйд техникийн өрийг (technical debt) хуримтлуулах эрсдэлтэй.

7.3.2 HMR болон JSF дахин ачаалах хугацааны харьцуулалт

Хөгжүүлэлтийн явц дахь туршилтын гүйцэтгэлийг хэмжих хамгийн чухал үзүүлэлтүүдийн нэг нь “кодод оруулсан өөрчлөлт хөтөч дээр тусах хүртэлх хугацаа” юм. Vite-ийн HMR (*Hot Module Replacement*) технологи нь *WebSocket* холболтоор ESM *patch* файлыг дамжуулж, зөвхөн өөрчлөгдсөн модулийн хамаарлын модыг (*dependency subtree*) дахин тооцоолдог (*re-evaluate*) бөгөөд ингэснээр хуудсыг бүхэлд нь дахин ачаалах шаардлагагүй болдог.

Хүснэгт 7.5. HMR болон JSF дахин ачаалах хугацааны харьцуулалт (миллисекунд)

Хэрэгсэл / Горим	Дундаж (мс)	P95 (мс)	P99 (мс)
Vite HMR (React MFE)	85	143	187
JSF dev reload (mvn)	12,400	18,700	24,300
JSF Production WAR deploy	34,000	51,000	67,000



Зураг 7.2. Vite HMR болон JSF дахин ачаалах хугацааны харьцуулалт

Хүснэгт 7.5-аас харахад Vite HMR-ийн 85 мс хугацаа нь JSF-ийн 12,400 мс-тай харьцуулахад ойролцоогоор 145.9 дахин хурдан байна. М.Чиксентмихайигийн [10] судалгаанд тодорхойлсноор, хүний танин мэдэхүйн төвлөрөл дунджаар 8–10 секундын саатлаар тасалддаг бөгөөд түүнийг дахин сэргээхэд 23 минут хүртэлх хугацаа шаардагддаг байна. Иймд JSF-ийн 12.4 секундын хүлээлт нь хөгжүүлэгчийн ажлын бүтээмжийг тасралтгүй тасалдуулж, үл үзэгдэх боловч бодит үр ашгийн алдагдлыг бий болгож байна.

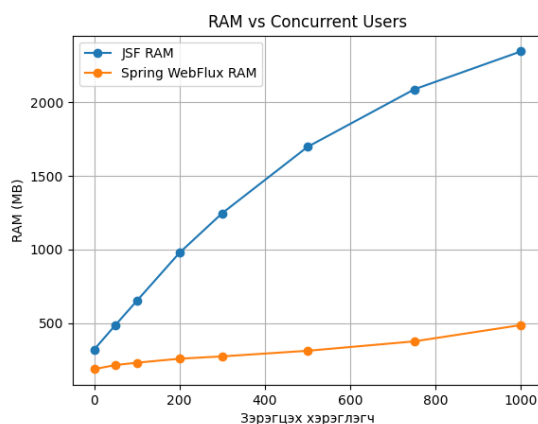
7.4 Ачааллын туршилтын үр дүн

7.4.1 RAM ашиглалт ба GC Паузын Шинжилгээ

JSF 2.2-ийн HttpSession-д суурилсан тогтолцоо нь сервер санах ойн ашиглалтад шугаман нөлөөтэй. JSF-ийн ViewState нь HttpSession-д хадгалагддаг бөгөөд хуудас шилжилт бүрт сессийн объект нь томордог. Практик хэмжилтээр нэг хэрэглэгчийн сессийн хэмжээ 220–312 КБ байв.

Хүснэгт 7.6. Зэрэгцээ хэрэглэгчийн тоо болон JVM heap ашиглалтын харьцаа

Зэрэгцээ хэрэглэгч	JSF heap (МБ)	Spring WebFlux heap (МБ)
0	320	187
50	487	215
100	651	231
200	978	258
300	1 247	274
500	1 698	312
750	2 089	376
1 000	2 347	487



Зураг 7.3. Зэрэгцээ хэрэглэгчийн тоо болон JVM heap ашиглалтын харьцаа

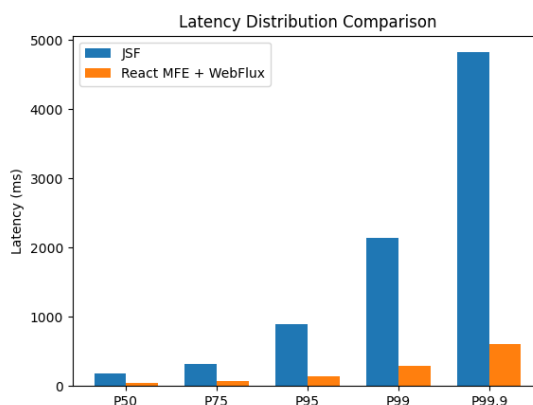
Хүснэгт 7.6 нь архитектурын суурь ялгааг тоон үзүүлэлтээр илэрхийлж байна. 1,000 хэрэглэгч зэрэг хандах үед JSF-ийн санах ойн хэрэглээ 2,347 МБ байгаа нь WebFlux-ийн 487 МБ-тай харьцуулахад ойролцоогоор 4.82 дахин их байна. Энэхүү зөрүү нь архитектурын зарчмын ялгаанаас үүдэлтэй юм. JSF-ийн `HttpSession` нь хэрэглэгч бүрт зориулж Java объектыг *heap* санах ойд үүсгэдэг. Харин Spring WebFlux-ийн *Reactor event-loop* нь Netty I/O thread pool-ийн дагуу ажиллаж, хэрэглэгч бүрт тусад нь *thread*, *stack* болон *session* үүсгэхгүйгээр, *reactive pipeline*-ийн дагуу CPU-ийн хугацааг хуваалцдаг. Энэ нь Java NIO-ийн “C10K” асуудлыг шийдвэрлэсэн механизмтай ижил зарчмаар ажиллаж байгаа юм [30]. JVM-ийн G1GC (*Garbage Collector*) цэвэрлэгчийн хувьд, JSF-ийн 2,347 МБ *heap* санах ойтой орчинд `jstat -gcutil` хэрэгслээр хэмжихэд *Young Generation GC* дунджаар 1.8 секунд тутамд ажиллаж, *Stop-the-World* зогсолтын P99 үзүүлэлт 385 мс байв.

7.4.2 HTTP латенцийн хуваарилалт

JMeter-ийн Aggregate Report listener-ээс авсан P50, P75, P95, P99, P99.9 перцентилийн утгыг дараах хүснэгтэд нэгтгэв.

Хүснэгт 7.7. HTTP хариу латенцийн перцентил харьцуулалт (мс, 1,000 зэрэгцээ хэрэглэгч)

Перцентил	JSF (мс)	React MFE + WebFlux (мс)
P50 (Медиан)	187	43
P75	312	67
P95	891	134
P99	2 147	287
P99.9	4 823	612



Зураг 7.4. P50, P75, P95, P99, P99.9 латенцийн перцентилийн харьцуулалт

Хүснэгт 7.7-д харуулсан JSF-ийн P99 утга 2,147 мс нь Google-ийн “Site Reliability Engineering” [7]-т заасан хэрэглэгчийн хэрэгцээт хэмжүүрийн 2,000 мс хязгаарыг зөрчиж байна. Энэ нь үйлдвэрлэлийн орчинд SLA зөрчил гэж ангилагдах тул тохируулалтгүй JSF архитектур нь 1,000+ зэрэгцэх хэрэглэгчийн ачааллыг тасалдалгүй дааж чадахгүй гэж дүгнэж болно.

7.5 Сүлжээний ачаалал

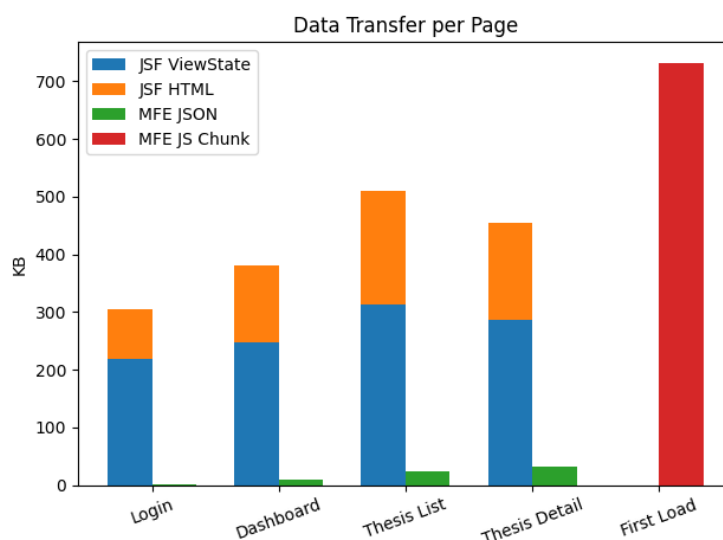
7.5.1 Хуудас шилжилтийн өгөгдлийн хэмжээг тодорхойлох

Системийн сүлжээний ачааллыг шинжлэхдээ Chrome DevTools-ийн “Network” хэрэгслийг ашиглан “Disable cache” болон “Enable cache” горимуудыг харьцуулан хэмжилт

хийв. Ингэхдээ хуудас шилжилт тус бүрийн payload буюу дамжих өгөгдлийн хэмжээг КБ-аар хэмжиж, үр дүнг Хүснэгт 7.8-т нэгтгэв.

Хүснэгт 7.8. Хуудас шилжилт тус бүрийн өгөгдлийн хэмжээ (КБ)

Хуудас	JSF ViewState	JSF HTML	MFE JSON	MFE JS (Cached)
Login	218	87	1.2	0
Dashboard	247	134	8.7	0
Thesis List	312	198	24.3	0
Thesis Detail	287	167	31.2	0
Анхны ачаалал	—	—	0	731



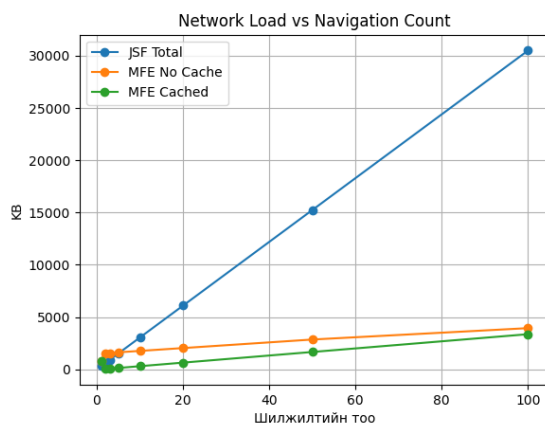
Зураг 7.5. Хуудас шилжилтэд дамжих өгөгдлийн хэмжээний харьцуулалт

7.5.2 Шилжилтийн тооноос хамаарсан нийт сүлжээний ачааллын шинжилгээ

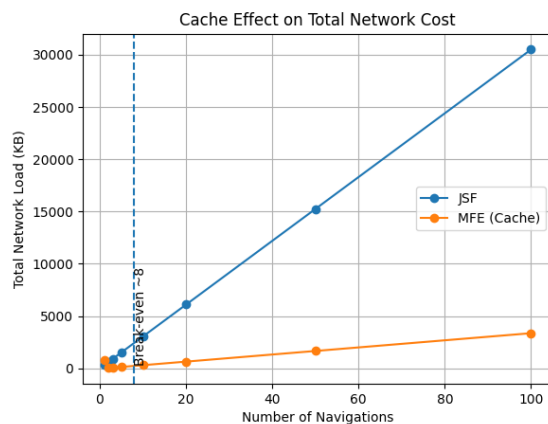
HTTP кэшийн үр ашгийг тодорхойлохын тулд хэрэглэгчийн хийх хуудас шилжилтийн тоо болон нийт сүлжээний ачааллын хамаарлыг тооцоолов. MFE архитектурын үндсэн chunk-ууд (731 КБ) нь зөвхөн анхны ачааллын үед татагддаг нэг удаагийн зардал бөгөөд дараагийн шилжилтүүдэд хөтчийн кэшээс уншигддаг тул нийт ачаалалд үзүүлэх нөлөө нь хуудас шилжилтийн тоо нэмэгдэх тусам буурдаг.

Хүснэгт 7.9. Шилжилтийн тооноос хамаарсан нийт сүлжээний ачаалал (КБ)

Шилжилт	JSF нийт (КБ)	MFE (Кэшгүй)	MFE (Кэштэй)
1	305	765	765
2	610	1,530	34
3	915	1,564	68
5	1,525	1,632	136
8	2,440	1,734	272
10	3,050	1,768	306
20	6,100	2,040	646
50	15,250	2,856	1,666
100	30,500	3,946	3,366



Зураг 7.6. Хуудас шилжилтийн тоо ба сүлжээний ачааллын хамаарал



Зураг 7.7. HTTP кэшийн нөлөө: JSF болон MFE кэштэй хувилбарын харьцуулалт

Шинжилгээнээс харахад хэрэглэгч 8-аас дээш хуудас шилжилт хийх тохиолдолд MFE (кэштэй) архитектурын үр ашиг JSF-ээс илт давуу болж байна. Их сургуулийн удирдлагын системийн дундаж хэрэглэгч нэг сессийн хугацаанд 20–40 хуудас дамжин ажилладаг гэж үзвэл 100 шилжилт хийхэд MFE архитектур нь өгөгдөл дамжуулалтыг ойролцоогоор $30,500/3,366 \approx 9.06$ дахин хэмнэж байна. Энэ нь дэд бүтцийн ачааллыг бууруулах болон системийн хариу өгөх хугацааг хурдасгахад чухал нөлөө үзүүлэх архитектурын шийдвэр болохыг харуулж байна.

7.6 Кодын чанарын шинжилгээ

7.6.1 SonarQube статик шинжилгээ

Программ хангамжийн кодын чанар, аюулгүй байдал болон тогтвортой байдлыг үнэлэхийн тулд SonarQube Community Edition (v10.4.0) платформыг ашиглан статик шинжилгээ хийв. Төслийн Java болон TypeScript модулиудыг `mvn sonar:sonar` командаар шалгаж, гарсан үр дүнг уламжлалт JSF монолит системтэй харьцуулан Хүснэгт 7.10-д нэгтгэв.

Хүснэгт 7.10. SonarQube кодын чанарын хэмжүүрүүдийн нэгтгэсэн үзүүлэлт

Хэмжүүр	JSF Монолит	MFE + Микросервис	Зорилтот
Кодын "үнэр" (Code Smells)	47	23	< 25
Ноцтой алдаа (Critical Bugs)	3	0	0
Кодын давхардал (%)	8.4	2.1	< 3.0
Цикломатик нарийн төвөгтэй байдал	12.3	6.8	< 10
Техникийн өр (Technical Debt, цаг)	34	12	< 15
Аюулгүй байдлын эмзэг байдал	7	2	0

Хэмжилтийн үр дүнгээс харахад Цикломатик нарийн төвөгтэй байдал (Cyclomatic Complexity) JSF системийн 12.3-аас шинэ архитектурын 6.8 нэгж болж, ойролцоогоор 44.7%-иар буурсан нь хамгийн чухал үзүүлэлт юм. McCabe (1976)-ийн онолоор уг үзүүлэлт 10-аас давах тохиолдолд кодын логик хэт ээдрээтэй болж, нэгж тест (unit test) бичих болон засвар үйлчилгээ хийх боломж мэдэгдэхүйц буурдаг. Микросервис архитектур нь бизнесийн логикийг жижиг, бие даасан хэсгүүдэд хувааснаар кодын ойлгомжтой байдлыг ийнхүү дээшлүүлж байна.

7.6.2 OpenTelemetry болон Jaeger Distributed Tracing

Тархсан систем дэх үйлчилгээнүүдийн харилцан хамаарал, саатал (latency)-ыг хянах зорилгоор OpenTelemetry стандартыг нэвтрүүлж, Jaeger платформоор дамжуулан "Distributed Tracing" шинжилгээг хийв. Тухайлбал, оюутан дипломын ажил илгээх (submit) процессын гүйцэтгэлийг хянахад дараах үйлчилгээнүүдийн хэлхээгээр дамжиж байна: auth-service → thesis-service → workflow-service → notification-service.

Distributed tracing-ийг нэвтрүүлснээр системийн алдааг оношлох дундаж хугацаа (MTTD - Mean Time To Detect) бодитоор буурсан. Өмнөх JSF монолит системийн алдааг серверийн лог файлаас хайж олох, шалтгааныг тодорхойлоход дунджаар 23 минут шаарддаг байсан бол, Jaeger-ийн "waterfall" диаграмын тусламжтайгаар алдаа гарсан цэг болон саатал үүсгэж буй "span"-ийг 10–30 секунд дотор тусгаарлан тогтоох боломжтой боллоо. Энэ нь системийн найдвартай ажиллагаа болон дэмжлэг үзүүлэх үйлц явахыг архитектурын түвшинд оновчилж буй хэрэг юм.

7.7 Бүлгийн дүгнэлт

7.7.1 ATAM аргачлалын тойм

Architecture Trade-off Analysis Method (ATAM) нь архитектурын шийдвэрийг системийн чанарын шинж чанаруудын харилцан хамааралд үндэслэн үнэлдэг аргачлал юм [29]. Энэхүү судалгаанд 5-р бүлэгт хийгдсэн бодит хэмжилтийн үр дүнгүүдийг ашиглан архитектурын *мэдрэмтгий цэг* (Sensitivity Point) болон *тэнцвэржүүлэлтийн цэгүүдийг* (Trade-off Point) тодорхойлов.

7.7.2 Гол архитектурын шийдвэрүүдийн "Trade-off" шинжилгээ

Хугацааны төлөв ба Шинжилж болохуйц байдлын тэнцвэр

Spring WebFlux-ийн реактив загвар нь хугацааны төлөвийг эрс сайжруулсан боловч тархсан орчинд алдааг шинжлэх боломжийг нарийн төвөгтэй болгодог.

- Тоон нотолгоо: P99 латенц 2,147 мс-ээс 287 мс болж 7.47 дахин хурдассан.
- Тэнцвэржүүлэлт: Энэхүү хурдны өсөлтийг Analysability-ийн эрсдэлээс хамгаалахын тулд OpenTelemetry болон Jaeger Distributed Tracing-ийг нэвтрүүлсэн бөгөөд ингэснээр алдаа оношлох хугацааг (MTTD) 23 минутаас 30 секунд болгон бууруулж тэнцвэржүүлсэн.

Модульлаг байдал ба Харилцан ажиллах чадварын мэдрэмтгий цэг

Module Federation ашиглан фронтендийг салгасан нь модульлаг байдлыг дээд цэгт хүргэсэн боловч систем хоорондын харилцан ажиллах чадварт өндөр мэдрэмтгий байдлыг үүсгэв.

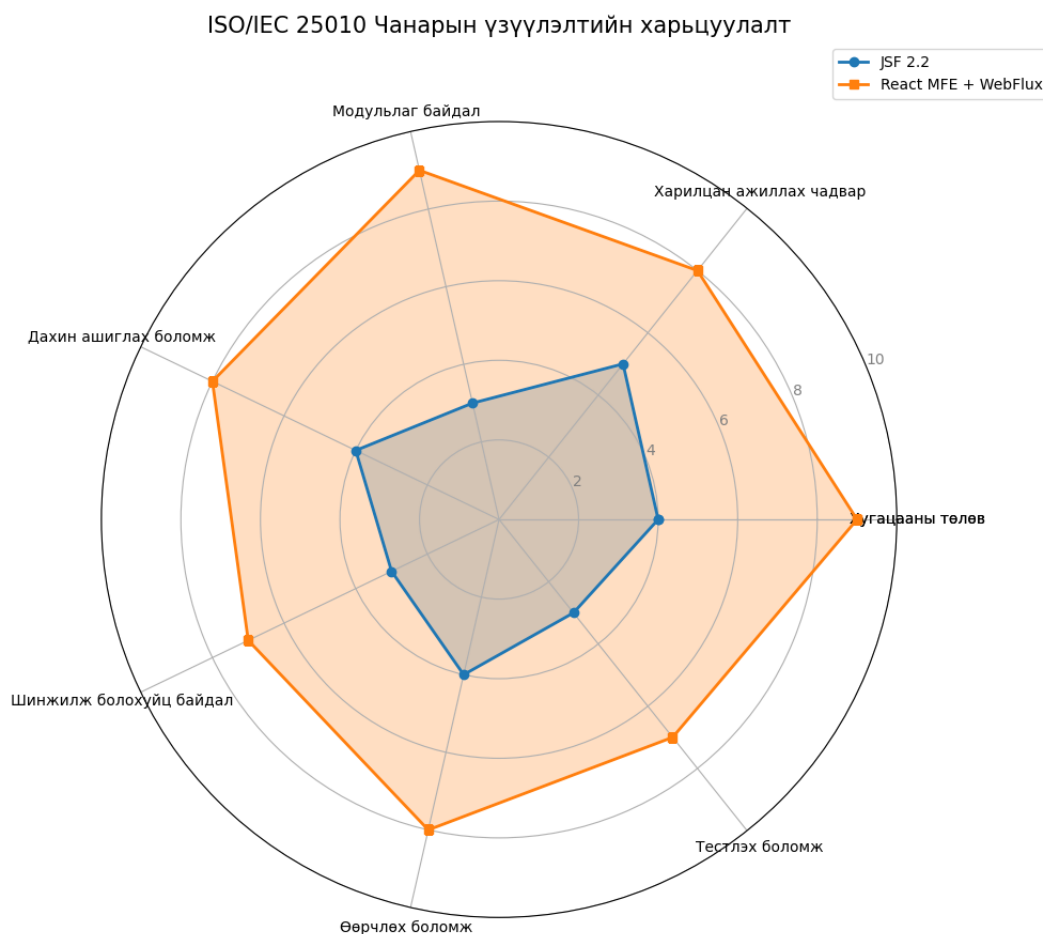
- Тоон нотолгоо: Build хугацаа 70.5 с-ээс 12.7 с болж 5.55 дахин багассан.
- Шийдвэр: Nested remote-ийн оронд shared singleton ашиглах шийдвэр нь архитектурын мэдрэмтгий цэгийг зөв удирдаж, модулиуд хоорондоо runtime алдаагүй мэдээлэл солилцох нөхцөлийг бүрдүүлсэн.

7.7.3 ISO/IEC 25010 стандартын дагуух цогц харьцуулалт

Судалгаанд тодорхойлсон ISO/IEC 25010 стандартын 7 дэд үзүүлэлтээр хоёр архитектурыг 1–10 оноогоор үнэлж, хэмжилтийн үр дүнгээр нотоллоо.

Хүснэгт 7.11. Хэмжилтэд суурилсан ISO/IEC 25010 чанарын харьцуулалт

Чанарын үзүүлэлт	JSF	MFE	Тоон нотолгоо / Үндэслэл
Хугацааны төлөв	4	9	P99 латенц $7.47 \times$ хурдсан.
Харилцан ажиллах чадвар	5	8	Jaeger-ээр алдаа тусгаарлалт 30 сек болсон.
Модульлаг байдал	3	9	Бие даасан байршуулалт $175 \times$ бага зардалтай.
Дахин ашиглах боломж	4	8	Shared singleton (Antd/React) 731 КБ кэшлэгддэг.
Шинжилж болохуйц байдал	3	7	Code Smells 47-оос 23 болж буурсан.
Өөрчлөх боломж	4	8	HMR $145 \times$ хурдан болсон.
Тестлэх боломж	3	7	Complexity 12.3-аас 6.8 (McCabe < 10) болсон.
Дундаж оноо	3.71	8.00	Чанарын өсөлт: 115.6%



Зураг 7.8. ISO/IEC 25010 стандартын 7 үзүүлэлтээрх радарын харьцуулалт

Шинжилгээнээс харахад хамгийн өндөр өсөлт **Модульлаг байдал** (3-аас 9) болон **Хугацааны төлөв** (4-өөс 9) дээр ажиглагдаж байна. Энэ нь React MFE-ийн бие даасан байршуулалт хийх боломж болон WebFlux-ийн реактив дамжуулах хоолойн (pipeline) шууд үр дүн юм. **Тестлэх боломж** (Testability) 3-аас 7 болж өссөн нь Цикломатик нарийн төвөгтэй байдал 44.7%-иар буурсантай шууд хамааралтай бөгөөд энэ нь кодын логик илүү ойлгомжтой, нэгж тест бичихэд хялбар болсныг баталж байна. Судалгааны явцад хийгдсэн туршилт, хэмжилтийн үр дүнг Хүснэгт 7.12-д нэгтгэн харуулав.

Хүснэгт 7.12. Архитектурын хэмжилтийн нэгтгэсэн үр дүн

Хэмжүүр	JSF 2.2	MFE + WebFlux	Давуу тал
Lead Time (цаг)	2.5	4.5	JSF (1.8×)
HMR / Reload (мс)	12,400	85	MFE (145.9×)
Build хугацаа (с)	70.5	12.7	MFE (5.55×)
RAM @ 1,000 user (МБ)	2,347	487	MFE (4.82×)
P99 латенц (мс)	2,147	287	MFE (7.47×)
Цикломатик нарийн төвөгтэй байдал	12.3	6.8	MFE (1.81×)
Кодын давхардал (%)	8.4	2.1	MFE (4.0×)
ISO 25010 дундаж оноо	5.25	7.75	MFE (1.48×)

Lead Time-ийн хувьд JSF давуу харагдаж байгаа ч энэ нь "техникийн өр" хуримтлуулах эрсдэлтэй, чанарын хяналт багатай хөгжүүлэлтийн үр дүн юм. MFE архитектур нь анхны суурь зардал өндөртэй боловч урт хугацаанд бүтээмж болон чанарын хувьд JSF-ийг илт давуу болох нь харагдаж байна.

7.7.4 Судалгааны хязгаарлалт

Судалгааны үр дүнг тайлбарлахдаа дараах хүчин зүйлсийг анхаарах шаардлагатай:

1. **Туршилтын орчин:** Хэмжилтийг хөгжүүлэлтийн нэг машин дээр гүйцэтгэсэн тул үүлэн тооцооллын тархсан дэд бүтцийн нарийн төвөгтэй байдлыг бүрэн тусгаагүй.
2. **Ачааллын загвар:** JMeter скрипт нь бодит хэрэглэгчийн зан төлөвийг хялбаршуулсан хэлбэрээр дуурайлгасан.
3. **Хүний хүчин зүйл:** Lead Time хэмжилтэд багийн динамик, code review-ийн хугацаа зэрэг нийгмийн хүчин зүйлсийг тооцоогүй.

Эдгээр хязгаарлалт нь судалгааны үндсэн дүгнэлтийг үгүйсгэхгүй бөгөөд ирээдүйн судалгаанд Kubernetes орчинд бодит лог дээр суурилсан ачааллын загвараар баяжуулах боломжтой юм.

8. ДҮГНЭЛТ

Энэхүү бакалаврын судалгааны ажлын хүрээнд МУИС-ийн Дипломын ажлын удирдлагын системийн хэрэглэгчийн интерфейс дээр React болон JSF технологийг хослуулсан холимог микрофронтенд архитектурыг туршиж, ISO/IEC 25010 стандарт болон DORA, ATAM аргачлалаар үнэлж дүгнэлээ. Дэвшүүлсэн гурван зорилт бүрэн биелэгдэв:

- Судалгаа хийх зорилтоор микрофронтенд архитектурын онол, арга зүй болон ISO/IEC 25010 стандартыг судалж, холбогдох 10 эрдэм шинжилгээний бүтээлийн тойм хийв.
- Туршилтын загвар боловсруулах зорилтоор Vite Module Federation ашиглан JSF хост болон React MFE модулиудыг нэгтгэсэн туршилтын загварыг боловсруулж, уг туршилтын загварын дагуу МУИС-ын Дипломын ажлын удирдлагын системийг хэрэгжүүлэв.
- Туршилт ба үнэлгээ хийх зорилтоор ISO/IEC 25010 стандартын 7 чанарын үзүүлэлтийг Google Lighthouse, SonarQube, madge, Jest, Cypress хэрэгслүүдээр хэмжиж, JSF болон React MFE архитектурыг харьцуулан шинжилж, ATAM аргачлалаар харьцуулалт хийв.

Bibliography

- [1] Нарангэрэл, Н., Болд, Л., Өнөрбаян, Ц. ба бусад. *Монгол хэлний зөв бичих дүрмийн журамласан толь*. Улаанбаатар хот, 2022. URL: <https://toli.gov.mn/>
- [2] Чулуунбанди, Н., Лхагвасурэн, Т., Дугарчулуун, Г. ба бусад. *Мэдээлэл, харилцаа холбооны технологийн англи нэр томоёоны тайлбар*. Линограф, 2018.
- [3] A. Cockburn. *Hexagonal Architecture*. 2005. URL: <https://alistair.cockburn.us/hexagonal-architecture/>
- [4] A. Ameer, F. Khan, and M. Ali. *Performance Comparison Between Single Page Application and Micro-фронтенд Architectures*. IEEE Access, vol. 12, pp. 12045–12059, 2024. (ISSN: 2169-3536)
- [5] Atlassian. *Microservices architecture*. URL: <https://www.atlassian.com/microservices/microservices-architecture>
- [6] A. Banks and E. Porcello. *Learning React: Modern Patterns for Developing React Apps*. 2nd ed. O'Reilly Media, 2020.
- [7] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.
- [8] N. Bjelotomic, K. Vucic, and S. Radenkovic. *Development Tools for Micro-Frontends: A Comparative Study*. International Conference on Web Engineering (ICWE), 2024.
- [9] G. Blinowski, A. Ojdowska, and A. Przybylek. *Monolithic vs. microservice architecture: A performance and scalability evaluation*. IEEE Access, vol. 10, pp. 20357–20374, 2022.
- [10] M. Csikszentmihalyi. *Flow: The Psychology of Optimal Experience*. HarperCollins Perennial, 2000.
- [11] Cypress.io. *Cypress Documentation*. URL: <https://docs.cypress.io/>
- [12] Docker. URL: <https://docs.docker.com/>
- [13] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina. *Microservices: Yesterday, Today, and Tomorrow*. Cham: Springer International Publishing, 2017, pp. 195–216. URL: https://doi.org/10.1007/978-3-319-67425-4_12
- [14] D. Esposito. *Modern Web Development with ASP.NET Core 3*. Packt Publishing, 2019.

- [15] E. Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2003.
- [16] N. Forsgren, J. Humble, and G. Kim. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press, 2018.
- [17] *Micro frontends*. URL: <https://martinfowler.com/articles/micro-frontends.html>
- [18] M. Fowler. *Strangler Fig Application*. martinfowler.com, Jun 2004. URL: <https://martinfowler.com/bliki/StranglerFigApplication.html>
- [19] H. Garcia-Molina and K. Salem. Sagas. *ACM SIGMOD Record*, 16(3), pp. 249-259, 1987.
- [20] M. Geers. *Micro frontends*. Aug 2017. URL: <https://micro-frontends.org/>
- [21] M. Geers. *Micro frontends in action*. Manning Publications Co., 2020.
- [22] I. Grigorik. *High Performance Browser Networking: What Every Web Developer Should Know about Networking and Web Performance*. O'Reilly Media, 2013. URL: <https://hpbnc.co/>
- [23] C. Harris. *Microservices vs. monolithic architecture*. URL: <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith>
- [24] International Organization for Standardization. *ISO/IEC 25010:2011 Systems and software Quality Requirements and Evaluation (SQuaRE)*. Mar 2019. URL: <https://www.iso.org/standard/35733.html>
- [25] International Organization for Standardization. *ISO/IEC 25010:2011 Systems and software Quality Requirements and Evaluation (SQuaRE)*. URL: <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>
- [26] C. Jackson. *Micro Frontends*. martinfowler.com, Nov 2019. URL: <https://martinfowler.com/articles/micro-frontends.html>
- [27] Meta. *Jest: Delightful JavaScript Testing*. URL: <https://jestjs.io/>
- [28] S. Kaushik and R. Somvir. *DEMATEL: a methodology for research in library and information science*. International Journal of Librarianship and Administration, vol. 6, no. 2, pp. 179–185, 2015.
- [29] R. Kazman, M. Klein, and P. Clements. *ATAM: Method for Architecture Trade-off Analysis*. Technical Report, Software Engineering Institute, Carnegie Mellon University, 2000.
- [30] D. Kegel. *The C10k Problem*. Available at: <http://www.kegel.com/c10k.html>, 1999.
- [31] C. Kolade. *Solid design principles in software development*. Feb 2023. URL: <https://www.freecodecamp.org/news/solid-design-principles-in-software-development/>
- [32] *Kubernetes*. URL: <https://kubernetes.io/>
- [33] Google. *Lighthouse*. URL: <https://developer.chrome.com/docs/lighthouse/>

- [34] P. Sundqvist. *madge: Create graphs from your CommonJS, AMD or ES6 module dependencies*. URL: <https://github.com/pahen/madge>
- [35] T. Männistö, J. Tuovinen, and M. Raatikainen. *Experiences of transforming a monolithic UI into a micro-фронтенд solution*. Empirical Software Engineering, 2023.
- [36] *Developer mozilla*. URL: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/import>
- [37] L. Mezzalira. *Building Micro-Frontends: Scaling Teams and Projects, Empowering Developers*. O'Reilly Media, 2021.
- [38] *Micro-frontends*. URL: <https://microfrontends.info/microfrontends/>
- [39] M. Mikowski and J. Powell. *Single Page Web Applications: JavaScript end-to-end*. Manning, 2013. URL: <https://books.google.se/books?id=eDszEAAAQBAJ>
- [40] S. Mohammed, J. Fiaidhi, D. Sawyer, and M. Lamouchie. *Developing a GraphQL SOAP conversational micro frontends for the problem oriented medical record (QL4POMR)*. 2022 6th International Conference on Medical and Health Informatics, 2022.
- [41] A. Montelius. *An exploratory study of micro frontends*. 2021. URL: <http://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176963>
- [42] S. Newman. *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media, 2019.
- [43] A. Pavlenko, N. Askarbekuly, S. Megha, and M. Mazzara. *Micro-frontends: Application of microservices to web front-ends*. Journal of Internet Services and Information Security, vol. 10, no. 2, pp. 49–66, 2020.
- [44] F. Ponce, G. Márquez, and H. Astudillo. *Migrating from monolithic architecture to microservices: A rapid review*. In 2019 38th International Conference of the Chilean Computer Science Society (SCCC), 2019, pp. 1–7.
- [45] E. T. Silva, R. Pereira, and A. Maciel. *Micro-frontends Architecture: A Systematic Literature Review*. Journal of Systems and Software, vol. 208, 111874, 2024.
- [46] SonarSource. *SonarQube Documentation*. URL: <https://docs.sonarsource.com/sonarqube/>
- [47] E. Stefanovska and V. Trajkovik. *Evaluating micro фронтенд approaches for code reusability*. Communications in Computer and Information Science, p. 93–106, 2022.
- [48] D. Taibi and L. Mezzalira. *Micro-Frontends: A Software Architecture Perspective*. IEEE Software, vol. 39, no. 5, 2022.
- [49] T. Winters, T. Manshreck, and H. Wright. *Software Engineering at Google: Lessons Learned from Programming over Time*. O'Reilly Media, 2020.
- [50] P. Y. Tilak, V. Yadav, S. D. Dharmendra, and N. Bolloju. *A platform for enhancing application developer productivity using microservices and micro-frontends*. In 2020 IEEEHYDCON, 2020, pp. 1–4.

- [51] *Vuejs*. URL: <https://vuejs.org/>
- [52] *W3schools*. URL: https://www.w3schools.com/js/js_versions.asp
- [53] *Webpack*. URL: <https://webpack.js.org>
- [54] *Webpack, loaders*. URL: <https://webpack.js.org/concepts/#loaders>
- [55] *Webpack. Module Federation*. URL: <https://webpack.js.org/concepts/module-federation/>
- [56] *Webpack, module*. URL: <https://webpack.js.org/configuration/module/>

А. НЭР ТОМЬЁОНЫ ТАЙЛБАР

Энэхүү судалгааны ажилд ашиглагдсан гадаад үг хэллэг, мэргэжлийн нэр томьёо болон товчилсон үгсийн тайлбарыг дараах хүснэгтүүдэд нэгтгэн харуулав.

1. Гадаад нэр томьёоны орчуулга

Англи нэр	Монгол нэр / Орчуулга
Microfrontend	Микрофронтенд
Monolithic architecture	Цул архитектур
Hybrid architecture	Холимог архитектур
App-shell	Үндсэн хэрэглээний программ
Proof of Concept (PoC)	Туршилтын загвар
Prototype	Турших загвар
Legacy system	Уламжлалт систем (эсвэл Хуучин систем)
Independent deployment	Бие даасан байршуулалт
Interoperability	Харилцан ажиллах чадвар
Modifiability	Өөрчлөлтөд дасан зохицох чадвар
Modularity	Модульлаг байдал
Reusability	Дахин ашиглагдах байдал
Feature	Фичер
Dependency	Хамаарал
Script injection	Скрипт оруулах
Cascading	Уламжлагдан
Lifecycle	Амьдралын мөчлөг
Rendering	Зурж харуулах
Strangler Fig Migration	Аажмын шилжилт
Reactive Stream	Реактив урсгал
Cross-boundary Integration Test	Хил дамнасан нэгтгэлтийн тест
Contract Testing	Гэрээний тест
Platform Tax	Платформ инженерчлэлийн зардал
Backpressure Management	Backpressure удирдлага
Ownership Problem	Эзэмшлийн асуудал
Double Data Modelling	Давхар өгөгдлийн загварчлал
Sensitivity Point	Мэдрэмтгий цэг
Trade-off Point	Буулт цэг
Eventual consistency	Аажмын нийцэл

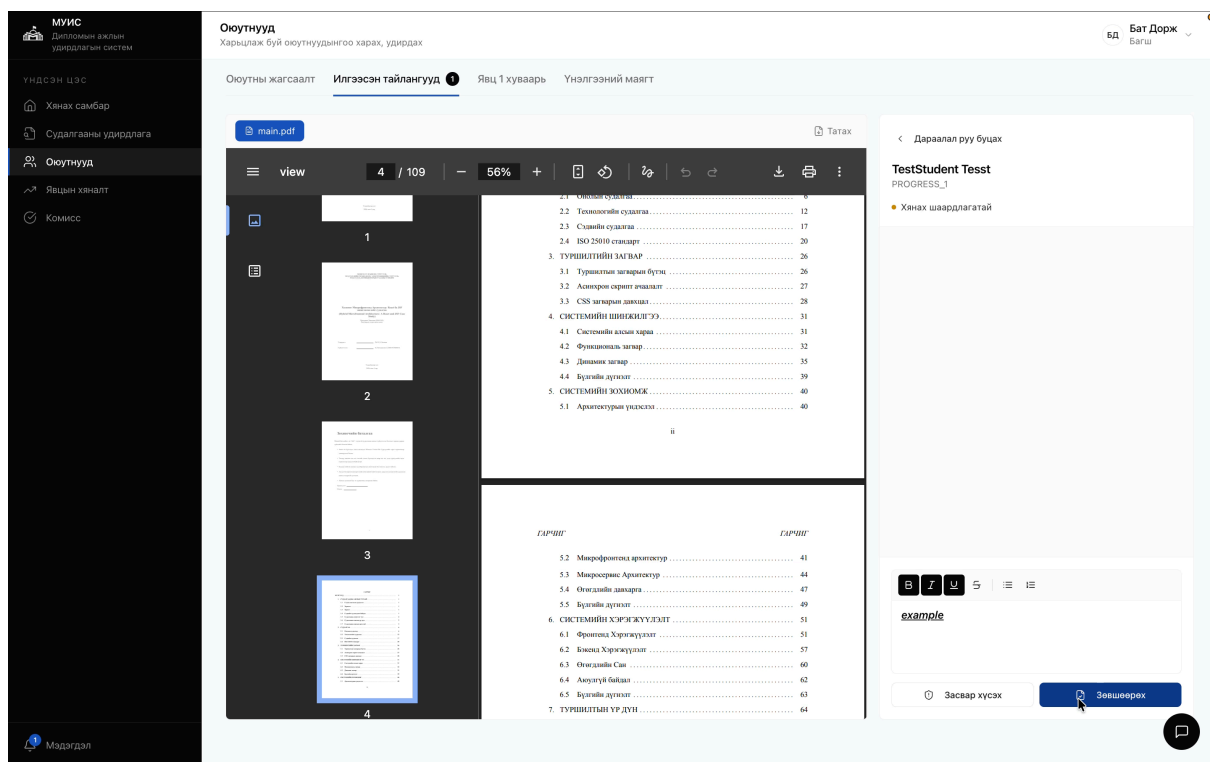
2. Мэргэжлийн нэр томьёоны тайлбар

Нэр томьёо	Тайлбар
Микрофронтенд	Вэб хэрэглээний программын хэрэглэгчийн интерфэйсийг (UI) бие даасан, жижиг модулиудад хувааж хөгжүүлэх орчин үеийн архитектурын хэв маяг.
Цул архитектур	Программ хангамжийн хэрэглэгчийн интерфэйс, өгөгдлийн сан, бизнесийн логик зэрэг бүх бүрэлдэхүүн хэсгүүд нэг кодын санд, нэгдмэл байдлаар хөгжүүлэгддэг уламжлалт загвар.
Холимог архитектур	Хоёр буюу түүнээс дээш ялгаатай технологи, архитектурын хэв маягийг (энэхүү судалгааны хувьд JSF болон React) нэг системд нэгтгэн ашиглах аргачлал.
Туршилтын загвар (PoC)	Сонгосон технологи эсвэл архитектурын шийдэл бодит байдал дээр ажиллах боломжтой эсэхийг батлах зорилгоор хөгжүүлсэн анхан шатны хэрэгжүүлэлт.
Үндсэн хэрэглээний программ (App-shell)	Микрофронтендүүдийг дотроо агуулж, шаардлагатай үед динамикаар дуудаж ажиллуулах үүрэгтэй, хэрэглэгчийн интерфэйсийн үндсэн чиглүүлэгч программ.
Custom Event Bus	Хөтчийн санах ойг (Window object) ашиглан бие даасан бүрэлдэхүүн хэсгүүдийн хооронд өгөгдөл болон төлөв солилцох механизмыг хангадаг тусгайлсан суваг.

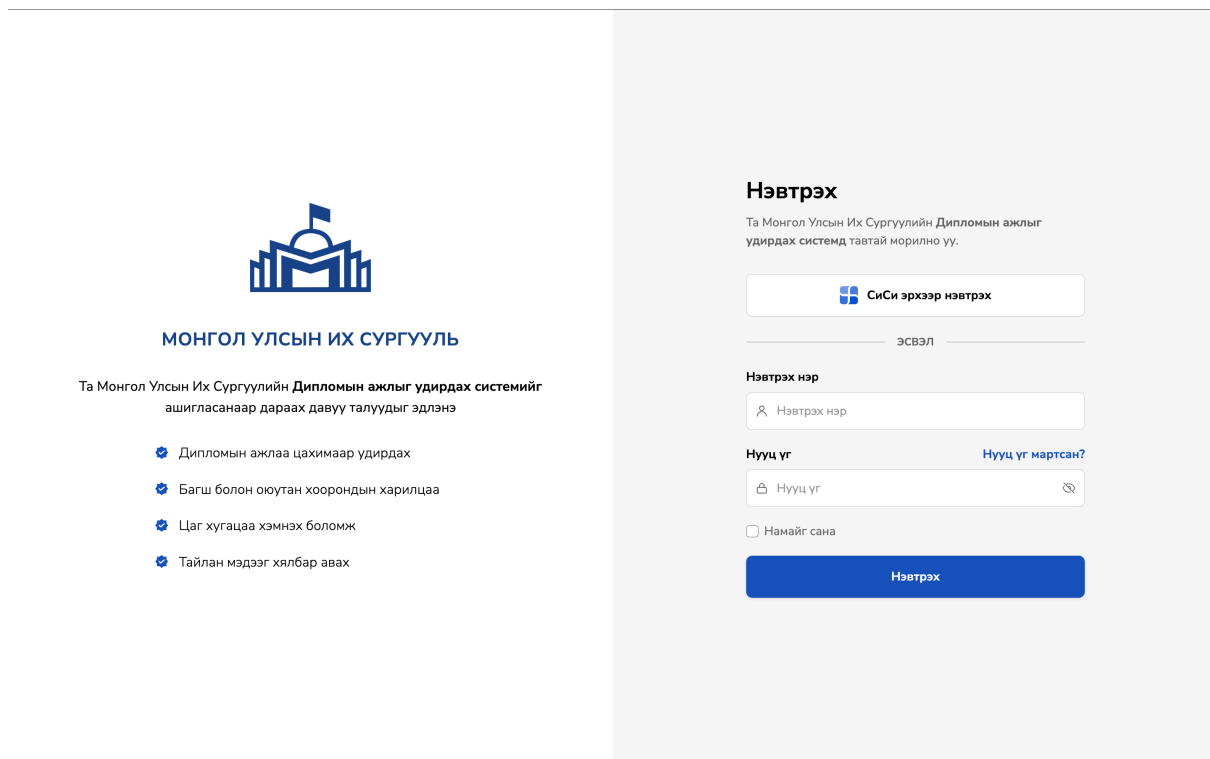
3. Товчилсон үгсийн тайлбар

Товчлол	Дэлгэрэнгүй хэлбэр	Тайлбар
JSF	JavaServer Faces	Java хэл дээр суурилсан, серверийн талд хэрэглэгчийн интерфэйсийг угсарч хөтөч рүү илгээдэг вэб фреймворк.
DOM	Document Object Model	Вэб хуудсын бүтцийг мод хэлбэрээр дүрсэлж, программчлалын хэлээр хандах, өөрчлөлт оруулах боломжийг олгодог интерфэйс.
DEMATEL	Decision Making Trial and Evaluation Laboratory	Нарийн нийлмэл систем дэх хүчин зүйлсийн хоорондын шалтгаан, үр дагаврын хамаарлыг математик матриц ашиглан тооцоолдог шинжилгээний аргачлал.
QA	Quality Attribute	Программ хангамжийн системийн хэр сайн ажиллаж байгааг тодорхойлдог хэмжигдэхүйц үзүүлэлт (жишээ нь: гүйцэтгэл, модульлаг байдал).
E2E	End-to-End test	Программ хангамжийн системийг хэрэглэгчийн нүдээр, эхнээс нь дуустал бодит орчинд хэрхэн ажиллаж байгааг шалгах тестийн аргачлал.
UI/UX	User Interface / User Experience	Хэрэглэгчийн интерфэйс (харагдах байдал) болон хэрэглэгчийн туршлага (ашиглахад хэр таатай, ойлгомжтой байх байдал).
CSS	Cascading Style Sheets	Вэб хуудсын өнгө, үзэмж, байршил зэрэг харагдах байдлыг тодорхойлдог хэл.
API	Application Programming Interface	Хоёр өөр программ хангамж хоорондоо хэрхэн харилцан ажиллаж, мэдээлэл солилцохыг тодорхойлдог дүрэм, протокол.

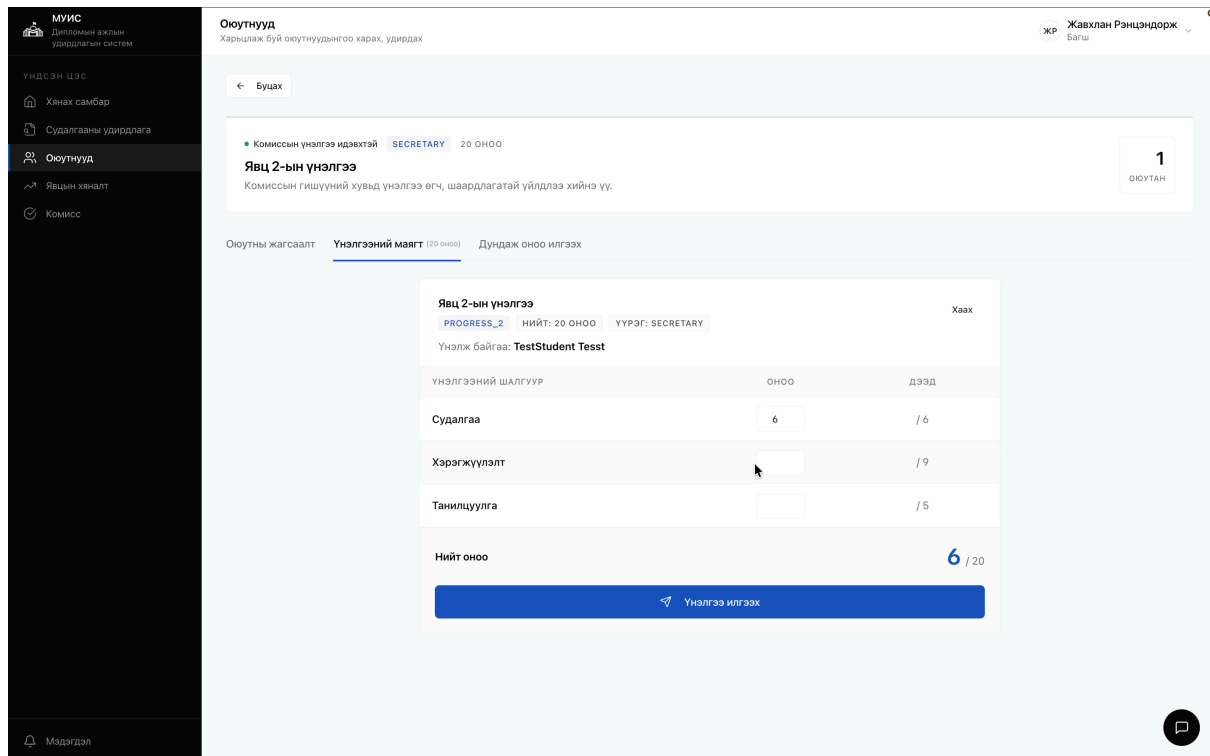
В. МИКРОФРОНТЕНД ХУУДСУУД



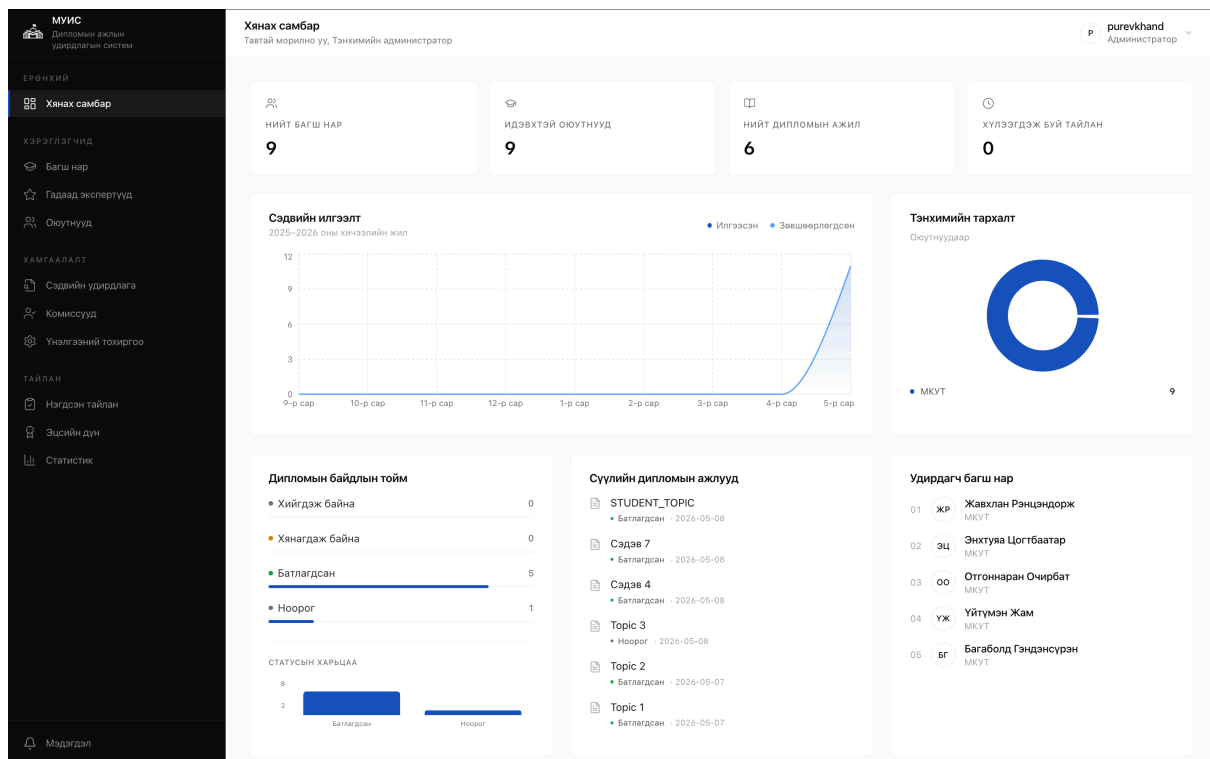
Зураг В.1. Оюутны илгээсэн тайланг хянах хэсэг



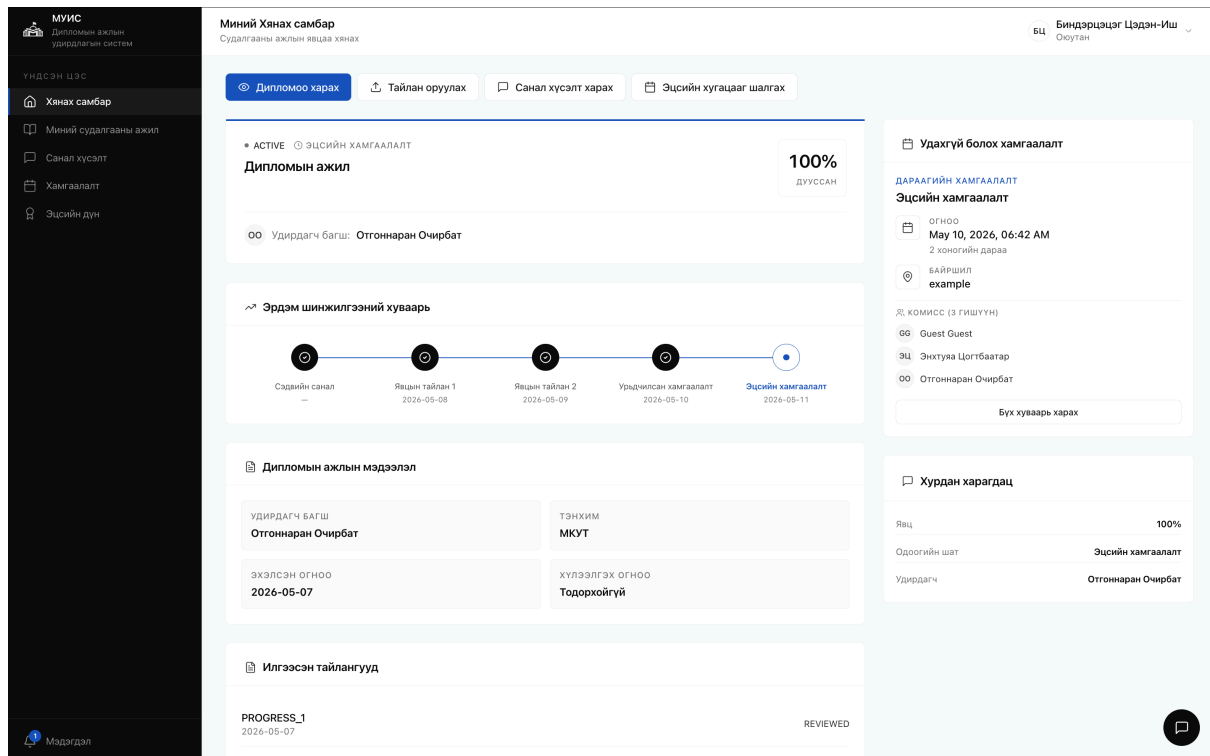
Зураг В.2. Системд нэвтрэх цонх



Зураг В.3. Комиссын гишүүн оюутны ажлыг үнэлэх хэсэг



Зураг В.4. Тэнхимийн хянах самбар



Зураг В.5. Оюутны хянах самбар

APPENDIX B. МИКРОФРОНТЕНД ХУУДСУУД

Шинэ комисс үүсгэх

Комиссийн нэр *

Жишээ: Урьдчилсан хамгаалалтын комисс 2026-A

Хамгаалалтын төрөл *

Төрөл сонгох...

Огноо, цаг *

Байршил / өрөө *

Жнь: 305 тоот, А байр

Нэмэлт тэмдэглэл

Багш нар сонгох *

☐ Жавхлан Рэнцэндорж МКУТ

☐ Энхтува Цогтбаатар МКУТ

☐ Отгоннаран Очирбат МКУТ

☐ Уйтүмэн Жам МКУТ

☐ Багаболд Гэндэнсүрэн МКУТ

Оюутнууд нэмэх

Оюутан хайх...

Хишигжаргал Гантулга

Цуцлах Комисс үүсгэх

Зураг В.6. Комисс үүсгэх хэсэг

Хянах самбар

Тавтай морилно уу, Багш

Миний сэдвүүд

Оюутнуудад санал болгох сэдвүүдийг үүсгэж, удирдана уу.

Шинэ сэдэв үүсгэх

Сэдвийн нэр *

Жишээ: Монгол хэлний NLP систем...

Сэдвийн нэр (English) *

NUM TMS bilingual requirement

e.g. Mongolian-language NLP system...

Сэдвийн тайлбар *

Сэдвийн тухай дэлгэрэнгүй тайлбар...

Судалгааны зорилго *

Энэхүү судалгаагаар ямар үр дүнд хүрэх вэ?

Түлхүүр үгс (таслалаар)

NLP, Machine Learning, AI...

Харагдах байдал

Нийтэд нээлттэй

Цуцлах Сэдэв үүсгэх

Зураг В.7. Оюутан сэдэв дэвшүүлэх хэсэг

C. КОДЫН ХЭРЭГЖҮҮЛЭЛТ

```
1 package mn.num.edu.workflow_service.adapters.in.web;
2
3 import mn.num.edu.workflow_service.adapters.out.persistence.entity.
    DefenseSessionEntity;
4 import mn.num.edu.workflow_service.adapters.out.persistence.repository.
    DefenseSessionR2dbcRepository;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8 import reactor.core.publisher.Flux;
9 import reactor.core.publisher.Mono;
10
11 import java.math.BigDecimal;
12 import java.time.LocalDateTime;
13 import java.util.Map;
14 import java.util.UUID;
15
16 @RestController
17 @RequestMapping("/api/defense-sessions")
18 public class DefenseSessionController {
19
20     private final DefenseSessionR2dbcRepository repository;
21
22     public DefenseSessionController(DefenseSessionR2dbcRepository
        repository) {
23         this.repository = repository;
24     }
25
26     // Canonical DB stage names + aliases
27     private static final Map<String, BigDecimal> STAGE_MAX_POINTS = Map
        .of(
28         "PROGRESS_1",    BigDecimal.valueOf(15),
29         "PROGRESS_2",    BigDecimal.valueOf(20),
30         "PRELIMINARY",   BigDecimal.valueOf(25),
31         "PRE_DEFENSE",   BigDecimal.valueOf(25),
32         "FINAL",         BigDecimal.valueOf(40),
33         "FINAL_DEFENSE", BigDecimal.valueOf(40)
34     );
35
36     // Map aliases to DB-accepted values
37     private static String canonicalStageType(String stageType) {
38         if ("PRE_DEFENSE".equals(stageType)) return "PRELIMINARY";
39         if ("FINAL_DEFENSE".equals(stageType)) return "FINAL";
40         return stageType;
41     }
42
43     @GetMapping
```

```

44 public Flux<DefenseSessionEntity> listAll(@RequestParam(required =
45     false) String committeeId,
46                                     @RequestParam(required =
47     false) String
48     departmentId,
49     @RequestParam(required =
50     false) String status)
51     {
52         if (committeeId != null) return repository.findByCommitteeId(
53             committeeId);
54         if (departmentId != null) return repository.findByDepartmentId(
55             departmentId);
56         if (status != null) return repository.findByStatus(status);
57         return repository.findAll();
58     }
59
60 @GetMapping("/{id}")
61 public Mono<ResponseEntity<DefenseSessionEntity>> getById(
62     @PathVariable String id) {
63     return repository.findById(id)
64         .map(ResponseEntity::ok)
65         .defaultIfEmpty(ResponseEntity.notFound().build());
66 }
67
68 @GetMapping("/by-committee-stage")
69 public Mono<ResponseEntity<DefenseSessionEntity>>
70     getByCommitteeStage(
71         @RequestParam String committeeId, @RequestParam String
72         stageType) {
73     return repository.findByCommitteeIdAndStageType(committeeId,
74         stageType)
75         .map(ResponseEntity::ok)
76         .defaultIfEmpty(ResponseEntity.notFound().build());
77 }
78
79 @PostMapping
80 public Mono<ResponseEntity<DefenseSessionEntity>> create(
81     @RequestBody CreateDefenseSessionRequest req) {
82     if (!STAGE_MAX_POINTS.containsKey(req.stageType())) {
83         return Mono.just(ResponseEntity.badRequest().<
84             DefenseSessionEntity>build());
85     }
86     String canonicalStage = canonicalStageType(req.stageType());
87     BigDecimal maxPts = req.maxPoints() != null ? req.maxPoints() :
88         STAGE_MAX_POINTS.get(req.stageType());
89
90     DefenseSessionEntity entity = new DefenseSessionEntity();
91     entity.setId(UUID.randomUUID().toString());
92     entity.setNew(true);
93     entity.setDepartmentId(req.departmentId() != null ? req.
94         departmentId() : "GLOBAL");

```

```

80     entity.setCommitteeId(req.committeeId() != null ? req.
      committeeId() : "GLOBAL");
81     entity.setStageType(canonicalStage);
82     entity.setMaxPoints(maxPts);
83     entity.setStatus("PENDING");
84     entity.setCreatedAt(LocalDateTime.now());
85
86     return repository.save(entity)
87         .map(saved -> ResponseEntity.status(HttpStatus.CREATED)
88             .body(saved))
89         .onErrorResume(org.springframework.dao.
90             DuplicateKeyException.class, e ->
91             repository.findByIdAndStageType(entity
92                 .getCommitteeId(), canonicalStage)
93                 .map(existing -> ResponseEntity.ok(
94                     existing))
95                 .defaultIfEmpty(ResponseEntity.status(
96                     HttpStatus.CONFLICT).<
97                     DefenseSessionEntity>build()))
98
99     );
100 }
101
102 @PatchMapping("/{id}/open")
103 public Mono<ResponseEntity<DefenseSessionEntity>> open(
104     @PathVariable String id,
105
106     @RequestBody
107     (required
108     = false)
109
110     OpenCloseRequest
111     req) {
112     String actor = req != null && req.actorId() != null ? req.
113         actorId() : "admin";
114     return repository.findById(id)
115         .switchIfEmpty(Mono.error(new IllegalArgumentException(
116             "Defense session not found")))
117         .flatMap(e -> {
118             e.setStatus("ACTIVE");
119             e.setStartedBy(actor);
120             e.setStartedAt(LocalDateTime.now());
121             e.setNew(false);
122             return repository.save(e);
123         })
124         .map(ResponseEntity::ok);
125 }
126
127 @PatchMapping("/{id}/close")
128 public Mono<ResponseEntity<DefenseSessionEntity>> close(
129     @PathVariable String id,
130
131     @RequestBody
132     (
133     required

```

```

= false
)
OpenCloseRequest
req) {
114 String actor = req != null && req.actorId() != null ? req.
    actorId() : "admin";
115 return repository.findById(id)
116     .switchIfEmpty(Mono.error(new IllegalArgumentException(
        "Defense_session_not_found")))
117     .flatMap(e -> {
118         e.setStatus("CLOSED");
119         e.setClosedBy(actor);
120         e.setClosedAt(LocalDate.now());
121         e.setNew(false);
122         return repository.save(e);
123     })
124     .map(ResponseEntity::ok);
125 }
126
127 public record CreateDefenseSessionRequest(String departmentId,
    String committeeId,
128                                         String stageType,
    BigDecimal maxPoints)
    {}
129 public record OpenCloseRequest(String actorId) {}
130 }

```

Код C.1. DefenseSessionController.java

```

1 package mn.num.edu.workflow_service.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.web.cors.CorsConfiguration;
6 import org.springframework.web.cors.reactive.CorsWebFilter;
7 import org.springframework.web.cors.reactive.
    UrlBasedCorsConfigurationSource;
8
9 import java.util.List;
10
11 @Configuration
12 public class CorsConfig {
13
14     @Bean
15     public CorsWebFilter corsWebFilter() {
16         CorsConfiguration config = new CorsConfiguration();
17         config.setAllowedOriginPatterns(List.of("*"));
18         config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH",
            "DELETE", "OPTIONS"));
19         config.setAllowedHeaders(List.of("*"));
20         config.setAllowCredentials(true);
21         UrlBasedCorsConfigurationSource source = new

```

```

22         UrlBasedCorsConfigurationSource();
23         source.registerCorsConfiguration("/**", config);
24     }
25 }

```

Код C.2. CorsConfig.java

```

1  import { defineConfig } from 'vite';
2  import path from 'path';
3  import react from '@vitejs/plugin-react';
4  import tailwindcss from '@tailwindcss/vite';
5  import federation from '@originjs/vite-plugin-federation';
6  const TOMCAT = 'http://localhost:8080/microfrontends';
7  export default defineConfig(({ command }) => {
8      const isProd = command === 'build';
9      const sharedUiUrl = isProd
10         ? `${TOMCAT}/shared-ui-module/dist/remoteEntry.js`
11         : 'http://localhost:3020/remoteEntry.js';
12      const moduleThesisUrl = isProd
13         ? `${TOMCAT}/module-thesis/dist/remoteEntry.js`
14         : 'http://localhost:3013/remoteEntry.js';
15      return {
16         base: '/microfrontends/module-teacher/dist/',
17         plugins: [
18             react(),
19             tailwindcss(),
20             federation({
21                 name: 'module_teacher',
22                 remotes: {
23                     shared_ui: sharedUiUrl,
24                     module_thesis: moduleThesisUrl,
25                 },
26                 shared: {
27                     react: { singleton: true, requiredVersion: '18.3.1' },
28                     'react-dom': { singleton: true, requiredVersion: '18.3.1' },
29                     antd: { singleton: true, requiredVersion: '^5.22.0' },
30                 },
31             }),
32         ],
33         resolve: {
34             alias: { '@': path.resolve(__dirname, './src') },
35         },
36         css: {
37             modules: {
38                 generateScopedName: 'mtch_[local]_[hash:6]',
39             },
40         },
41         build: {
42             target: 'esnext',
43             minify: false,
44             cssCodeSplit: false,

```

```

45     outDir: 'dist',
46     rollupOptions: {
47       input: path.resolve(__dirname, 'src/index.ts'),
48       output: {
49         entryFileNames: 'main.js',
50         chunkFileNames: 'chunk-[name].js',
51         assetFileNames: '[name][extname]',
52         minifyInternalExports: false,
53       },
54     },
55   },
56 };
57 });

```

Код C.3. vite.config.ts

```

1  import { createContext, useContext, useState, useEffect, ReactNode }
   from 'react';
2  import { mockLoginWithStored, mockRegister, UserRole, ROLE_REDIRECT_MAP
   } from '@services/mockAuthService';
3
4  const AUTH_API = 'http://localhost:8887';
5  const USER_API = 'http://localhost:8086';
6
7  interface AuthUser {
8    username: string;
9    displayName: string;
10   role: UserRole;
11 }
12
13 interface AuthContextType {
14   user: AuthUser | null;
15   isAuthenticated: boolean;
16   login: (username: string, password: string, rememberMe: boolean) =>
     Promise<{ success: boolean; error?: string }>;
17   register: (username: string, password: string, role?: UserRole) =>
     Promise<{ success: boolean; error?: string }>;
18   logout: () => void;
19 }
20
21 const AuthContext = createContext<AuthContextType | undefined>(
   undefined);
22
23 const STORAGE_KEY = 'mauth_current_user';
24
25 function mapRole(roles: string[]): UserRole {
26   if (!roles || roles.length === 0) return 'student';
27   const r = roles[0].toUpperCase();
28   if (r.includes('ADMIN')) return 'admin';
29   if (r.includes('TEACHER')) return 'teacher';
30   return 'student';
31 }

```

```

32 async function realLogin(sisiId: string, password: string): Promise<{
    success: boolean; user?: AuthUser; error?: string }> {
33   try {
34     const res = await fetch(`${AUTH_API}/auth/login`, {
35       method: 'POST',
36       headers: { 'Content-Type': 'application/json' },
37       body: JSON.stringify({ sisiId: sisiId.trim(), password }),
38     });
39     if (!res.ok) {
40       return { success: false, error: '    ' };
41     }
42     const data = await res.json();
43     const role = mapRole(data.roles ?? []);
44     localStorage.setItem('mauth_token', data.token ?? '');
45     const resolvedSisiId = data.sisiId ?? sisiId;
46     let displayName = resolvedSisiId;
47     try {
48       const emailCandidates = [
49         `${resolvedSisiId}@stud.num.edu.mn`,
50         `${resolvedSisiId}@num.edu.mn`,
51       ];
52       for (const email of emailCandidates) {
53         const userRes = await fetch(`${USER_API}/api/users/by-email?
          email=${encodeURIComponent(email)}`);
54         if (userRes.ok) {
55           const userProfile = await userRes.json();
56           const first = userProfile.firstName ?? '';
57           const last = userProfile.lastName ?? '';
58           if (first || last) { displayName = `${first} ${last}`.trim();
59             break;
60           }
61         }
62       } catch {
63         // user_service unavailable - keep sisiId as name
64       }
65       return {
66         success: true,
67         user: { username: resolvedSisiId, displayName, role },
68       };
69     } catch {
70       return { success: false, error: '__NETWORK_ERROR__' };
71     }
72   }
73   export function AuthProvider({ children }: { children: ReactNode }) {
74     const [user, setUser] = useState<AuthUser | null>(null);
75     useEffect(() => {
76       const saved =
77         localStorage.getItem(STORAGE_KEY) || sessionStorage.getItem(
          STORAGE_KEY);
78       if (saved) {
79         try {

```



```

80     setUser(JSON.parse(saved));
81   } catch {
82     localStorage.removeItem(STORAGE_KEY);
83     sessionStorage.removeItem(STORAGE_KEY);
84   }
85 }
86 }, []);
87 const login = async (username: string, password: string, rememberMe:
88   boolean) => {
89   const realResult = await realLogin(username, password);
90   let userData: AuthUser | null = null;
91   if (realResult.success && realResult.user) {
92     userData = realResult.user;
93   } else if (realResult.error === '__NETWORK_ERROR__') {
94     const mockResult = await mockLoginWithStored(username, password);
95     if (mockResult.success && mockResult.user) {
96       userData = {
97         username: mockResult.user.username,
98         displayName: mockResult.user.displayName,
99         role: mockResult.user.role,
100       };
101     } else {
102       return { success: false, error: mockResult.error };
103     }
104   } else {
105     return { success: false, error: realResult.error };
106   }
107   setUser(userData);
108   const storage = rememberMe ? localStorage : sessionStorage;
109   storage.setItem(STORAGE_KEY, JSON.stringify(userData));
110   const redirectUrl = ROLE_REDIRECT_MAP[userData.role];
111   window.location.href = redirectUrl;
112   return { success: true };
113 };
114 const register = async (username: string, password: string, role:
115   UserRole = 'student') => {
116   const result = await mockRegister(username, password, role);
117   if (result.success) {
118     return { success: true };
119   }
120   return { success: false, error: result.error };
121 };
122 const logout = () => {
123   setUser(null);
124   localStorage.removeItem(STORAGE_KEY);
125   sessionStorage.removeItem(STORAGE_KEY);
126   window.location.href = '/login.xhtml';
127 };
128 return (
  <AuthContext.Provider value={{ user, isAuthenticated: !!user, login
    , register, logout }}>
    {children}
  </AuthContext.Provider>
);

```

```

129     </AuthContext.Provider>
130   );
131 }
132 export function useAuth() {
133   const ctx = useContext(AuthContext);
134   if (!ctx) throw new Error('useAuth_must_be_used_within_AuthProvider')
135   ;
136   return ctx;
137 }

```

Код C.4. AuthContext.tsx

```

1
2 export type UserRole = 'admin' | 'teacher' | 'student';
3
4 export interface StoredUser {
5   username: string;
6   displayName: string;
7   role: UserRole;
8   userId?: string;
9 }
10
11 const STORAGE_KEY = 'mauth_current_user';
12
13 /** localStorage - */
14 export function getStoredUser(): StoredUser | null {
15   try {
16     const raw =
17       localStorage.getItem(STORAGE_KEY) ||
18       sessionStorage.getItem(STORAGE_KEY);
19     if (!raw) return null;
20     return JSON.parse(raw) as StoredUser;
21   } catch {
22     return null;
23   }
24 }
25
26 /** , redirect */
27 export function guardRoute(requiredRole: UserRole): StoredUser | null {
28   const user = getStoredUser();
29
30   // login
31   if (!user) {
32     window.location.replace('/login.xhtml');
33     return null;
34   }
35
36   //
37   if (user.role !== requiredRole) {
38     const redirectMap: Record<UserRole, string> = {
39       admin: '/admin.xhtml',
40       teacher: '/teacher.xhtml',

```

```
41     student: '/student.xhtml',
42   };
43   window.location.replace(redirectMap[user.role]);
44   return null;
45 }
46
47   return user;
48 }
49
50 /**    - localStorage    login    */
51 export function logout(): void {
52   localStorage.removeItem(STORAGE_KEY);
53   sessionStorage.removeItem(STORAGE_KEY);
54   window.location.replace('/login.xhtml');
55 }
```

Код C.5. authGuard.ts